

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ АГЕНСТВО ВОЗДУШНОГО ТРАНСПОРТА
ФГБОУ ВПО «САНКТ-ПЕТЕРБУРГСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ГРАЖДАНСКОЙ АВИАЦИИ»
КАФЕДРА ИНФОРМАТИКИ

Ю. В. Земсков

Программирование на языке C/C++

Часть I

Структурное программирование

Учебное пособие

Санкт-Петербург

2012

Земсков Ю.В. Программирование на языке C/C++. Часть I. Структурное программирование. Учебное пособие/ СПб университет ГА. СПб., 2012.

Рецензенты:

Содержание

1. Основные сведения о языке C/C++	5
1.1. Алгоритмические языки	5
1.2. Состав языка	9
1.3. Переменные и именованные константы	12
1.4. Стандартные типы данных	14
1.5. Интегрированная среда разработки программ	16
1.5.1. Microsoft Visual Studio	16
1.5.2. Dev-C++	18
1.6. Простейшая программа на языке C	19
1.7. Область действия переменных	23
1.8. Ввод-вывод числовых значений	26
1.9. Выражения и математические функции	29
1.10. Контрольные вопросы	37
2. Потоки ввода-вывода, условные операторы, циклы	42
2.1. Потоки ввода-вывода	42
2.2. Системы счисления. Кодирование чисел	45
2.3. Логические выражения	51
2.4. Условные операторы	55
2.5. Операторы цикла	65
2.6. Контрольные вопросы	80
3. Массивы и указатели	83
3.1. Одномерные статические массивы	83
3.2. Указатели. Динамические массивы	87
3.3. Сортировка элементов массива	95
3.4. Двумерные массивы	99
3.5. Контрольные вопросы	102

4. Модульное программирование	105
4.1. Функции	105
4.2. Параметры функций	115
4.3. Многофайловые проекты	127
4.4. Шаблоны функций	128
5. Символы, строки, структуры, файлы	129
5.1. Символы	129
5.2. Строки	133
5.3. Пользовательские типы данных	137
5.4. Работа с файлами	141
6. Основы объектно-ориентированного программирования	144
6.1. Классы	144
Список литературы	147
Книги по С	147
Книги по С++	147
Сборники задач	148
Предметный указатель	149

1. Основные сведения о языке C/C++

1.1. Алгоритмические языки

- ▶ *Алгоритм* — конечный набор правил, который определяет последовательность операций для решения конкретного множества задач.
- ▶ *Алгоритмический язык* — формальный язык, используемый для записи, реализации или изучения алгоритмов.
- ▶ *Язык программирования* — формальная знаковая система, предназначенная для записи компьютерных программ.
- ▶ *Программа* — совокупность данных и команд, предназначенных для реализации определённого алгоритма.
- ▶ *Программирование* — научная и практическая деятельность по созданию программ [ГОСТ 19781-90].

Язык программирования определяет набор лексических, синтаксических и семантических правил, задающих внешний вид программы и действия, которые выполнит компьютер под её управлением.

Синтаксис устанавливает правила построения всех элементов языка и описывает структуру программы как набор символов (обычно говорят: «безотносительно к содержанию»). Семантика определяет смысл и правила использования каждой синтаксической конструкции.

Любой язык программирования является алгоритмическим языком, но не всякий алгоритмический язык пригоден для использования в качестве языка программирования.

Машинный код. Программа, непосредственно выполняемая процессором компьютера, представляет собой последовательность нулей и единиц, одни участки которой кодируют команды, а другие — данные. Подобная последовательность называется *машинным кодом*. *Язык ассемблера* представляет собой удобную для восприятия человеком форму записи машинных команд. Между машинным кодом и кодом на языке ассемблера существует взаимно

однозначное соответствие. Программа, которая переводит текст с языка ассемблера в машинный код, называется *ассемблером*. Для каждого процессора имеется свой набор машинных команд и, следовательно, свой ассемблер.

Далеко не всегда для написания программы требуется иметь полный контроль над процессором. Чаще всего на первый план выходят такие задачи, как быстрое написание читаемого и переносимого кода. Для этих целей были созданы языки высокого уровня.

Язык программирования высокого уровня — язык программирования, разработанный для быстроты и удобства использования программистом. Основная черта высокоуровневых языков — это абстракция, то есть введение смысловых конструкций, кратко описывающих такие структуры данных и операции над ними, описания которых на машинном коде (или другом низкоуровневом языке программирования) очень длинны и сложны для понимания.

Программа, написанная на языке высокого уровня, состоит из *инструкций* (*операторов*). Каждый оператор языка высокого уровня соответствует последовательности из нескольких низкоуровневых инструкций (машинных команд).

Парадигмы программирования. Языки высокого уровня, предназначенные для решения сходных задач сходными методами принято разделять на группы, называемые *парадигмами*. *Парадигма программирования* — это совокупность идей и понятий, определяющих стиль написания программ. В рамках каждой парадигмы существуют свои соглашения о том, какие подходы применяются при решении различных типов задач, как оформлять программный код и т. д.

Все языки программирования делятся на две группы: *декларативные* и *императивные*. Программа на императивном языке программирования с математической точки зрения представляет собой общее решение поставленной

задачи, иными словами, ответ на вопрос «как делать?». Это последовательность команд, которые должен выполнить исполнитель.

Программа на декларативном языке программирования является сочетанием формализованной в рамках языка программирования задачей и всех необходимых для её решения теорем, проще говоря, ответ на вопрос «что делать?».

В рамках императивной парадигмы существуют несколько подходов к разбиению задач на подзадачи; основные из них: *процедурное программирование* и *объектно-ориентированное программирование*. Эти два подхода рассматриваются, соответственно, в первой и второй части настоящего пособия.

Структурное программирование — методология разработки программного обеспечения, в основе которой лежит представление программы в виде иерархической структуры блоков. В соответствии с данной методологией любая программа представляет собой структуру, построенную из трёх типов базовых конструкций (рис. 1.1):

- *последовательное исполнение* — однократное выполнение операций в том порядке, в котором они записаны в тексте программы;
- *ветвление* — однократное выполнение одной из двух или более операций, в зависимости от выполнения некоторого заданного условия;
- *цикл* — многократное исполнение одной и той же операции до тех пор, пока выполняется некоторое заданное условие.

В программе базовые конструкции могут быть вложены друг в друга произвольным образом.

Фрагменты программы, представляющие собой логически целостные вычислительные блоки (в том числе, повторяющиеся фрагменты) могут оформляться в виде так называемых *подпрограмм* (*процедур* или *функций*).

Язык С (Си) — императивный язык программирования. Основные сферы его применения — системное программирование (практически всё современное системное ПО написано на С) и создание прикладных программ.

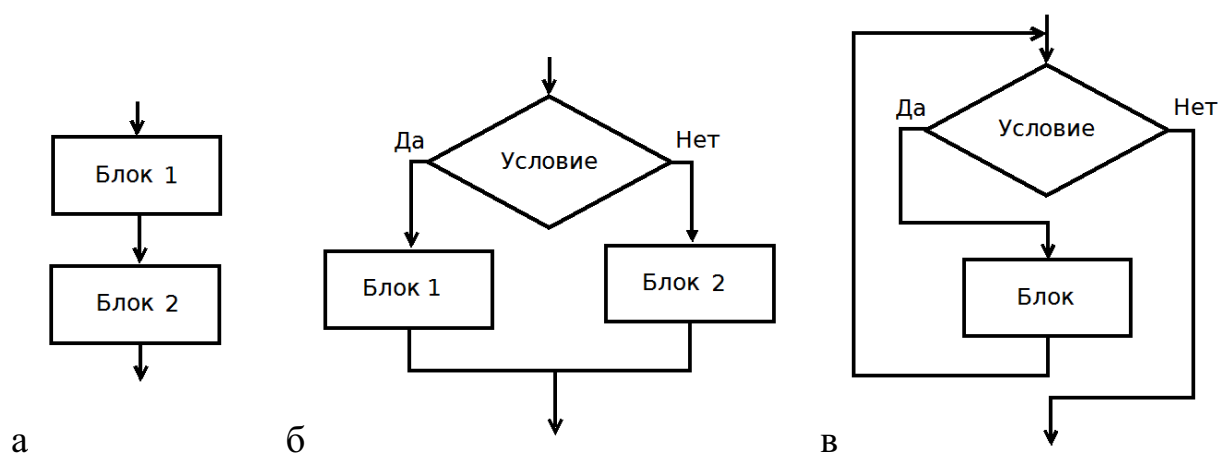


Рис. 1.1. Базовые конструкции структурного программирования: а) последовательное исполнение; б) ветвление; в) цикл с предусловием

С одной стороны, это язык программирования высокого уровня, поддерживающий методику структурного программирования. С другой стороны, этот язык обеспечивает возможность создавать системные программы, которые до него приходилось писать на языке ассемблера. Первым системным программным продуктом, разработанным с помощью С, стала операционная система UNIX. Поэтому нередко в литературе язык С называют языком среднего уровня. Стандарт языка С был утвержден в 1983 г. Американским национальным институтом стандартов (ANSI) и получил название ANSI C.

Трансляция: интерпретация и компиляция. Есть два способа выполнения программы компьютером: она может быть подвергнута компиляции или интерпретации. В простейшем случае *интерпретатор* читает исходный текст программы по строкам, выполняет инструкции, описанные в этой строке, и затем переходит к следующей. Так, например, работали ранние версии языка Basic. В языках типа Java исходный текст программы сначала конвертируется в промежуточную форму (байт-код), а затем уже интерпретируется при выполнении программы.

Компилятор конвертирует программу в *объектный код*, то есть транслирует исходный текст программы в форму, более пригодную для непосредственного выполнения компьютером. Объектный код также называют двоичным или машинным кодом; он зависит от того, для какой платформы (какого

процессора) производится компиляция программы. В общем случае интерпретируемая программа выполняется медленнее, чем скомпилированная.

Программа, написанная на любом языке программирования, может как компилироваться, так и интерпретироваться, однако многие языки изначально созданы для выполнения преимущественно одним из этих способов. При разработке языка C/C++ его конструкции оптимизировались для компиляции.

Трансляция — преобразование программы, представленной на одном языке программирования, в программу на другом языке, в определённом смысле равносильную первой. Таким образом, *компилятор* — это транслятор, который преобразует программы в машинный язык, а *интерпретатор* — это транслятор, осуществляющий пооператорную (покомандную) обработку и выполнение исходной программы.

1.2. Состав языка

Составными элементами исходного текста программы на языке C являются: *лексемы*, *выражения* и *операторы*. Для их написания используются символы, образующие *алфавит* языка. Каждый элемент языка определяется синтаксисом и семантикой. Как уже отмечалось, *синтаксические определения* устанавливают правила построения элементов языка, а *семантика* определяет их смысл и правила использования.

Алфавит. Алфавит языка C/C++ включает:

- прописные и строчные латинские буквы и знак подчёркивания;
- арабские цифры от 0 до 9;
- специальные знаки:

" { } | [] () + - / % * . \

' : ? < = > ! & # ~ ; ^ ,

- пробельные символы: пробел, символ табуляции, символ перехода на новую строку.

В комментариях, строках и символьных константах (см. ниже) допускаются любые другие символы, например, буквы русского алфавита.

Лексемы. Из символов алфавита формируются *лексемы* — минимальные (элементарные неделимые) единицы языка программирования, имеющие самостоятельный смысл:

- идентификаторы — имена программных объектов;
- ключевые (зарезервированные) слова;
- знаки операций;
- константы;
- разделители (скобки, точка, запятая, пробельные символы).

Идентификаторы. В идентификаторе (имени) программного объекта могут использовать только латинской буквы, символ подчёркивания и цифры. Первым символом идентификатора должна быть латинская буква или символ подчёркивания (но не цифра). Прописные и строчные буквы различаются, т.е. POS, Pos и pos — это три разных идентификатора. Пробелы внутри имён не допускаются.

Ключевые слова. *Ключевые слова* — это зарезервированные идентификаторы, которые имеют специальное значение. Эти имена должны использовать только в том смысле, в каком они определены стандартом языка C/C++:

asm	else	new	this
auto	enum	operator	throw
bool	explicit	private	true
break	export	protected	try
case	extern	public	typedef
catch	false	register	typeid
char	float	reinterpret_cast	typename
class	for	return	union
const	friend	short	unsigned
const_cast	goto	signed	using
continue	if	sizeof	virtual
default	inline	static	void

delete	int	static_cast	volatile
do	long	struct	wchar_t
double	mutable	switch	while
dynamic_cast	namespace	template	

В различных реализациях компилятора C/C++ список зарезервированных слов может быть расширен.

Константы (литералы). *Константы* служат для представления неизменяемых величин. Обычно их называют *литералами*, чтобы не путать с именованными константами (см. раздел 1.3). Различают целые, вещественные, символьные и строковые литералы.

Десятичные константы состоят из цифр 0, 1, 2, 3, 4, 5, 6, 7, 8 и 9; они не должны начинаться с нуля (кроме случая нулевой константы). Примеры: 21, 0.

Восьмеричные константы состоят из цифр от 0 до 7; они начинаются с цифры 0. Пример: 025.

Шестнадцатеричные константы состоят из цифр от 0 до 9 и из букв A, B, C, D, E, F (строчные и прописные не различаются); они начинаются символами 0x или 0X. Примеры: 0x15, 0xFE08.

Двоичные константы начинаются символами 0b или 0B, далее могут следовать только цифры 0 или 1. Примеры: 0b11011010, 0B00010110.

Вещественные десятичные константы имеют дробную часть, которая отделяется от целой части точкой. Они могут быть записаны в формате с фиксированной точкой или в экспоненциальной форме (в виде мантиссы и порядка). Примеры: 35.18, 3.518e1.

Символьные константы состоят из одного (или двух) символов, окружённого апострофами (одиночными кавычками), например: 'A'. Вместо символа можно писать его шестнадцатеричный код: '\x21'. Некоторые символы имеют специальное значение:

'\n' — переход на новую строку (перевод строки);

'\t' — горизонтальная табуляция (для построения таблиц);

' \' — апостроф (одиночная кавычка);

' \"' — символ двойной кавычки;

' \\' — символ «обратный слэш» (обратная косая черта);

и другие.

Строковые константы состоят из произвольного числа любых символов, окружённых двойными кавычками, например: "Привет!".

Комментарии. Комментарии либо начинаются знаком «двойной слэш» // и продолжаются до конца строки, либо заключаются символами-скобками /* и */.

Вложенные комментарии-скобки (т.е. комментарии внутри комментариев) в большинстве реализаций языка C не разрешены.

1.3. Переменные и именованные константы

Для хранения информации, обрабатываемой программой, используются *переменные и именованные константы*.

Переменная — это именованная область памяти, используемая для хранения данных, которые могут быть изменены программой. У переменной есть имя, тип и значение. *Тип* переменной определяет: 1) внутреннее представление данных в памяти компьютера, в том числе, размер памяти, отводимый для хранения значения; 2) множество допустимых значений; 3) набор допустимых операций, которые можно выполнять с переменной.

Именованная константа — это именованная область памяти, используемая для хранения данных, которые не могут быть изменены программой.

Все переменные и константы в языке C/C++ должны быть описаны перед первым использованием. *Описание* может быть сделано в форме объявления или определения. *Объявление* информирует компилятор о классе памяти и типе переменной; *определение* содержит, кроме этого, указание компилятору выделить память для хранения значения данной переменной. В наших первых

простых программах объявления переменных будут являться одновременно и определениями. Различие между ними мы увидим в разделе 4.

Формат описания переменной или константы:

`[класс_памяти] [const] тип идентификатор [инициализатор];`

Элементы описания, указанные в квадратных скобках, могут быть опущены.

Необязательный класс памяти может принимать значения `auto`, `extern`, `static`, `register`. Мы рассмотрим их позже (см. с. 24).

Модификатор `const` показывает, что значение переменной изменить нельзя (т. е. она является константой).

Правила именования программных объектов были описаны в разделе 1.2.

Инициализатор позволяет присвоить переменной начальное значение. Для констант наличие инициализатора обязательно. Инициализатор записывается после знака равенства

`= значение`

или в круглых скобках

`(значение)`

Подробно о типах переменных мы поговорим в следующем разделе, а пока нам достаточно знать, что в языке C/C++ могут использоваться переменные целого типа `int` или вещественного типа `double`.

Пример описания целой переменной:

`int i;`

Пример описания вещественной константы:

`const double a = 4.25;`

В одном операторе можно описывать (и, при необходимости, инициализировать) несколько переменных одного типа, разделяя их запятыми:

`int i, j = 5, k;`

`double x = 3.0, y;`

Ещё один способ определить именованную константу — директива препроцессора `#define`, например:

```
#define N 5
```

Знак присваивания в этом случае не используется. Точка с запятой после директивы не ставится. Константы, определённые через `#define`, принято писать заглавными буквами.

1.4. Стандартные типы данных

В языке C все типы делятся на простые (их также называют стандартными, базовыми или основными) и составные.

Для описания шести стандартных типов данных в языке C используются следующие ключевые слова:

- `int` — целое число (занимает в памяти 2 байта для 16-разрядных систем; 4 байта для 32-разрядных);
- `char` — символ (1 байт);
- `wchar_t` — расширенный символ (обычно 2 байта);
- `bool` — логическое значение (`true` или `false`, 1 байт);
- `float` — вещественное число (4 байта);
- `double` — вещественное число двойной точности (8 байт).

К имени типа может добавляться один из следующих модификаторов:

- `short` — короткий;
- `long` — длинный;
- `signed` — со знаком;
- `unsigned` — беззнаковый.

Например: `long int` (или просто `long`) — целое число двойной точности (4 байта); `signed short int` (или просто `signed short`) — короткое целое со знаком (2 байта).

В таблице 1.1 приведены сведения об основных типах языка C.

Таблица 1.1. Стандартные типы данных

Тип	Диапазон значений	Размер, байт
bool	true и false	1
signed char	−128...127	1
unsigned char	0...255	1
signed short int	−32 768...32 767	2
unsigned short int	0...65 535	2
signed long int	−2 147 483 648...2 147 483 647	4
unsigned long int	0...4 294 967 295	4
float	$\pm 3.4e - 38 \dots \pm 3.4e + 38$	4
double	$\pm 1.7e - 308 \dots \pm 1.7e + 308$	8
long double	$\pm 3.4e - 4932 \dots \pm 3.4e + 4932$	10

Чтобы узнать количество байт, занимаемых конкретным типом данных, используется функция `sizeof()`. Например, вызов `sizeof(float)` вернёт значение 4.

По умолчанию все целочисленные типы, а также тип `char` считаются знаковыми, т.е. модификатор `signed` можно опускать. Вещественные константы по умолчанию имеют тип `double`.

Числовым константам, встречающимся в программе, приписывается тот или иной тип в соответствии с их видом; тип можно указать явно с помощью *суффиксов*:

- L или l — long;
- U или u — unsigned;
- F или f — float.

Например, константа `15L` будет иметь тип `signed long int` и занимать 4 байта. Можно указывать несколько непересекающихся суффиксов

в произвольном порядке, например, 10UL или 10LU для константы типа `unsigned long int`.

1.5. Интегрированная среда разработки программ

Интегрированная среда разработки (IDE, Integrated development environment) — система программных средств, используемая программистами для разработки программного обеспечения (ПО). Включает в себя текстовый редактор, компилятор, средства автоматизации сборки, отладчик, систему справки (интерактивную документацию). Существуют несколько IDE для программирования на языке C/C++: коммерческие программы Microsoft Visual Studio (имеется также упрощённый и бесплатный вариант Microsoft Visual Studio Express); Borland C++ Builder; бесплатные программы Visual-MinGW, Dev-C++, Code::Blocks, Eclipse, Qt Creator.

1.5.1. Microsoft Visual Studio

Внешний вид экрана Microsoft Visual Studio 10 показан на рис. 1.2.

Цифрами обозначено:

- 1 — строка меню;
- 2 — панели инструментов;
- 3 — обозреватель решений с деревом проектов;
- 4 — окно с исходным текстом программы;
- 5 — окно вывода с результатами компиляции.

Запуск среды разработки программ: ⇨ Пуск ⇨ Программы
⇨ Microsoft Visual Studio 2010 ⇨ Microsoft Visual Studio 2010

Создание нового проекта (консольное приложение win32): ⇨ File (Файл) ⇨ New (Создать) ⇨ Project (Проект) или комбинация клавиш Ctrl+Shift+New.

⇨ Visual C++ ⇨ Win32 ⇨ Консольное приложение Win32

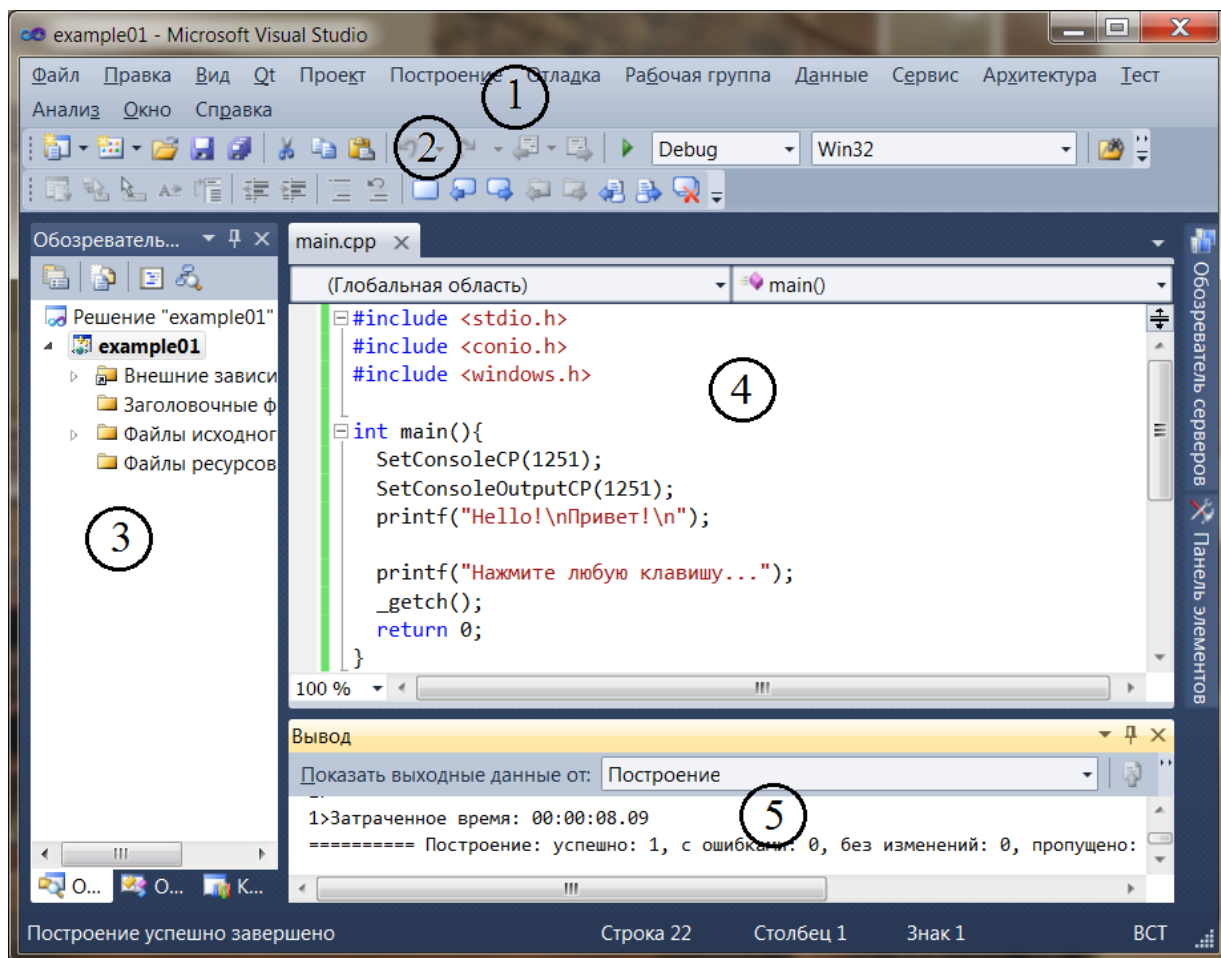


Рис. 1.2. Интегрированная среда разработки программ Microsoft Visual Studio 10

Ввести имя проекта, выбрать каталог для его размещения, задать имя решения (оно может совпадать с именем проекта).

▼ Создать каталог для решения ☐ OK ☐ Next (Далее)

▼ An empty project (Пустой проект) ☐ Finish (Готово)

Создание и добавление к проекту файла с исходным текстом программы: Если не отображается окно «Обозреватель решений», то выполнить команду меню ⇨ Вид ⇨ Обозреватель решения или нажать комбинацию клавиш Ctrl+Alt+L.

В обозревателе решений щёлкнуть по нужному проекту правой кнопкой мыши и выбрать команду ⇨ Add (Добавить) ⇨ Create (Создать элемент). Выбрать «Файл C++ (.cpp)». Ввести имя файла, например, main.cpp. Нажать ☐ Add (Добавить).

Сборка (компиляция и компоновка): ⇨ Build (Построение) ⇨ Build (Построить решение) или клавиша F7.

Отладка: ⇨ Debug (Отладка) ⇨ Start (Начать отладку) или клавиша F5.

Запуск программы без отладки: ⇨ Debug (Отладка) ⇨ Start without debugging (Запуск без отладки) или комбинация клавиш Ctrl+F5.

Выполнение программы по шагам: клавиша F10 (без захода внутрь функций) или F11 (с заходом).

1.5.2. Dev-C++

Внешний вид экрана Dev-C++ показан на рис. 1.3.

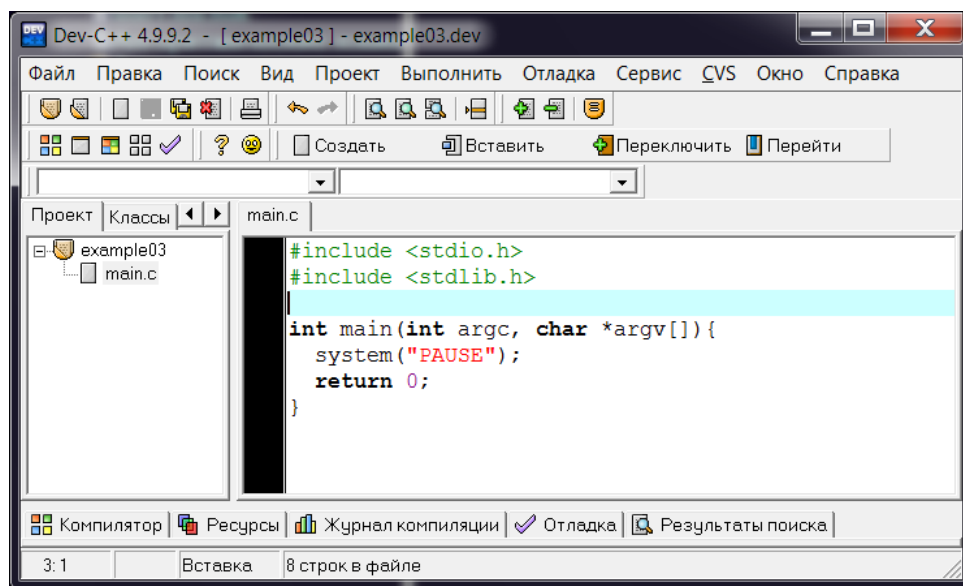


Рис. 1.3. Интегрированная среда разработки программ Dev-C++

Создание нового проекта (консольное приложение win32): ⇨ Файл ⇨ Создать ⇨ Проект ⇨ Console Application. Ввести имя проекта, выбрать используемый язык (C или C++), нажать ☐ ОК. Полный путь к файлам проекта не должен содержать пробелов или символов кириллицы.

Создание и добавление к проекту нового файла с исходным текстом программы: ⇨ Файл ⇨ Создать ⇨ Исходный файл ☐ Да

Сборка (компиляция и компоновка): ⇨ Выполнить ⇨ Скомпилировать или комбинация клавиш Ctrl+F9.

Выполнить (без отладки): комбинация клавиш Ctrl+F10.

Скомпилировать и выполнить: клавиша F9.

Компиляция проекта с возможностью отладки: ⇨ Проект ⇨ Параметры проекта  Компиляция  Компоновщик

Генерировать отладочную информацию := Yes (Да) ☐ OK

После этого перекомпилировать проект, нажав Ctrl+F11 (или выполнив пункт меню ⇨ Выполнить ⇨ Перестроить всё).

Отладка: ⇨ Отладка ⇨ Отладка или клавиша F8.

Задание (снятие) точки останова: щелчок мыши по левому полю или комбинация клавиш Ctrl+F5.

Выполнение программы по шагам: клавиша F7.

Отслеживание значения переменной в процессе отладки: нажать клавишу F4 и задать имя переменной.

1.6. Простейшая программа на языке C

Вывод текстовой строки. В листинге 1.1 приведена программа, которая выводит на консольный экран приветствие по-английски. Особенность *консольных приложений* в том, что они работают не в графическом, а в текстовом режиме.

Листинг 1.1: Простейшая программа на языке C (вывод текстовой строки)

```
#include <stdio.h>           ①
int main()                   ②
{                             ③
    printf("Hello!");        ④
    return 0;                ⑤
}                             ⑥
```

- ① — директива препроцессора: подключает заголовочный файл, в котором описаны функции ввода-вывода (в программе используется функция вывода `printf`);
- ② — программа на языке C/C++ всегда должна иметь главную функцию `main`; в простейшем случае входные параметры у этой функции отсутствуют (пустые круглые скобки), а возвращаемое значение — это код ошибки, имеет тип `int` (`integer` — целое число);
- ③, ⑥ — начало и конец функции (и любого другого блока кода) выделяются фигурными скобками, а текст блока для удобства чтения записывается с отступами;
- ④ — вывод строки на экран с помощью функции `printf()`, описанной в файле `stdio.h`; выводимый текст заключается в кавычки;
- ⑤ — выход и возврат кода ошибки в операционную систему (0 — ошибок нет).

Обратите внимание, что каждый оператор (в нашем примере — функция вывода `printf` и оператор возврата `return`) оканчивается точкой с запятой. После директивы препроцессора (`#include`) и заголовка функции (`main`) точка с запятой не ставится.

Компиляция и компоновка. После нажатия клавиши F7 (в среде Microsoft Visual Studio) или F9 (в Dev-C++) начинается процесс построения (`build`), который заключается в последовательном выполнении препроцессора, компилятора и компоновщика.

Препроцессор обрабатывает *директивы*, которые могут находиться в исходном тексте программы (пока нам известны только директивы `#include` и `#define`).

Получившийся в результате полный текст программы преобразуется *компилятором* в объектный модуль, который содержит машинные команды, определённые в программе данные и, возможно, символьные ссылки на библио-

теки каких-либо внешних функций, которые находятся в других объектных модулях.

Наконец, *компоновщик* (иначе называемый редактором связей) формирует из всех необходимых объектных модулей исполняемый файл (в системе Microsoft Windows исполняемые файлы имеют расширение `.exe`).

Символы национальных алфавитов. Изменим предыдущую программу, выведем на экран символы кириллицы:

```
printf("Hello!\nПривет!\n");    ④
```

Здесь используется `"\n"`, чтобы вставить в выводимое сообщение перевод строки.

К сожалению, вместо русских символов будут выведены непонятные значки. Чтобы вывод символов, код которых больше 128, работал правильно, необходимо указать номер кодовой страницы (способ кодировки символов). Кодировка вводимых символов устанавливается с помощью функции `SetConsoleCP()`; выводимых символов — с помощью функции `SetConsoleOutputCP()`. В обоих случаях в качестве параметра функции требуется указать номер кодовой страницы. Изменённая программа показана в листинге 1.2.

Листинг 1.2: Вывод в консоль символов кириллицы (способ 1)

```
#include <stdio.h>
#include <windows.h>    ①
int main() {
    SetConsoleCP(1251);    ②
    SetConsoleOutputCP(1251);    ③
    printf("Hello!\nПривет!\n");
    return 0;
}
```

① — подключение заголовочного файла, в котором определены функции, специфичные для системы Microsoft Windows;

- ② — установка кодовой страницы для ввода (можно опустить, т. к. в программе не предполагается чтение строковых данных);
- ③ — установка кодовой страницы для вывода.

Кроме того, в контекстном меню консольного окна (вызывается щелчком кнопки мыши по системному значку в левом верхнем углу окна) необходимо указать для консольных окон шрифт Lucida.

Для изменения кодировки также можно использовать функцию

```
setlocale(LC_ALL, "rus"); // кодировка для кириллицы
```

Ожидание в конце программы. Чтобы после выполнения программы (вне интегрированной среды) консольное окно не закрывалось, можно использовать функцию `_getch()`, которая ждёт нажатия клавиши.

Листинг 1.3: Ожидание нажатия любой клавиши

```
#include <stdio.h>
#include <conio.h>
#include <windows.h>
int main(){
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    printf("Hello!\nПривет!\n");
    printf("Нажмите любую клавишу...");
    _getch();
    return 0;
}
```

- ① — подключение заголовочного файла, в котором определена функция `_getch()`;
- ② — вывод на экран приглашения нажать любую клавишу;
- ③ — ожидание нажатия клавиши (в данном случае код нажатой клавиши нигде не используется).

Функция `system()`. С помощью функции `system()` можно выполнять команды операционной системы. Команда `chcp 1251` изменяет кодировку символов, а команда `pause` выводит на экран сообщение и ожидает нажатия любой клавиши. Поэтому программу из листинга 1.3 можно переписать по-другому (см. листинг 1.4). На рис. 1.4 показан результат работы программы.

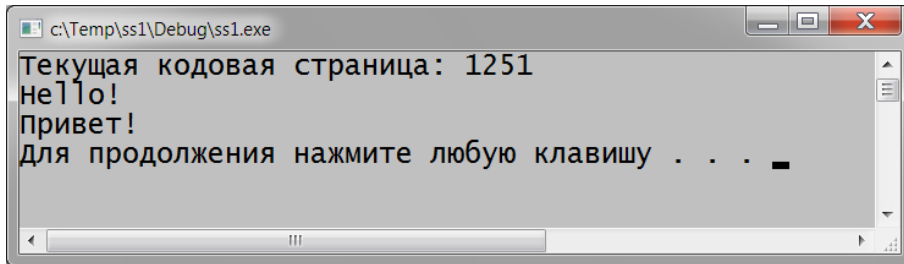


Рис. 1.4. Консольный экран программы из листинга 1.4

Листинг 1.4: Кодировка и ожидание нажатия клавиши (способ 2)

```
1 #include <stdio.h>    // Для функции printf().
2 #include <windows.h>  // Для функции system().
3 int main() {
4     system("chcp 1251"); // Кодировка (change code page).
5     printf("Hello!\nПривет!\n"); // Вывод текста.
6     system("pause");     // Ожидание нажатия клавиши.
7     return 0;
8 }
```

1.7. Область действия переменных

Описание переменной явно или по умолчанию задаёт область её действия. *Область действия* любого идентификатора — это та часть программы, где его можно использовать для доступа к связанной с ним области памяти. В зависимости от области действия переменная может быть *локальной* или *глобальной*.

Если переменная определена внутри блока (части текста программы, окруженной фигурными скобками), то она называется *локальной*; область её действия — от точки описания до конца блока (включая все вложенные блоки).

Если переменная определена вне какого-либо блока (и за пределами любой функции, в том числе `main`), она называется *глобальной*:

```
double x; // глобальная переменная
int main() {
    double y; // локальная переменная
    ...
}
```

Область действия глобальной переменной — от точки описания до конца файла. Использование глобальных переменных в общем случае не является хорошим стилем программирования: надо стремиться к тому, чтобы все необходимые данные функции получали через свои параметры (см. раздел 4.2).

Областью видимости идентификатора называется часть текста программы, из которой допустим обычный доступ к связанной с этим идентификатором области памяти. Обычно область видимости совпадает с областью действия. Исключением является ситуация, когда во вложенном блоке описана переменная с таким же именем; в этом случае внешняя переменная во вложенном блоке невидима, однако, если она глобальная, к ней можно обратиться, поставив перед её именем два двоеточия `::` (пример использования двойного двоеточия перед именем глобальной переменной см. в листинге 4.5 на с. 111); если она локальная, то обратиться к ней невозможно.

Время жизни переменной может быть постоянным (на всё время выполнения программы) или временным (пока выполняется данный блок).

Класс памяти определяет время жизни и область видимости программных объектов (в том числе, переменных). Для явного задания класса памяти используются спецификаторы:

`auto` — автоматическая переменная; память для неё выделяется в стеке и при необходимости инициализируется каждый раз при выполнении оператора, содержащего её определение. Освобождение памяти происходит при выходе из блока, в котором переменная описана. Поэтому автоматическая переменная не сохраняет своего значения при выходе из своего

блока и последующем возвращении в него. Время жизни автоматических переменных — от точки описания до конца блока. Для глобальных переменных спецификатор `auto` не используется, а для локальных — принимается по умолчанию, поэтому его обычно не пишут.

`extern` — превращает описание в объявление. Означает, что объект определён в другом месте программы (в другом файле или дальше по тексту). Используется, если переменная должна быть доступна в нескольких модулях (тогда она объявляется во всех модулях, где нужна, но определяется только в одном из них). Если переменная в том же операторе инициализируется, то спецификатор `extern` игнорируется (т. е. получается определение, а не объявление).

`static` — статическая переменная, имеющая постоянное время жизни. В зависимости от расположения описания статические переменные могут быть локальными (описаны внутри блока) или глобальными (описаны вне любого блока). Глобальные статические переменные видны только в том модуле (файле), где они описаны. Статические переменные создаются при первом выполнении оператора, где они определены, и, в отличие от автоматических переменных, продолжают существовать после выхода из своего блока; при возвращении в свой блок они не создаются заново, поэтому сохраняют своё последнее значение.

`register` — аналогично `auto`, но компилятор по возможности выделяет переменной место в регистрах процессора (для увеличения быстродействия).

Каждая глобальная или статическая переменная автоматически инициализируется нулевым значением (если в её определении не указан инициализатор). Автоматические переменные компилятором не инициализируются.

Имя переменной должно быть уникальным в пределах своей области действия; например, в одном блоке не может быть двух переменных с одинаковыми именами. Каждая переменная может быть многократно объявлена в

различных местах программы, но определена она может быть только в одном месте (поскольку при определении компилятор выделяет память для хранения значения переменной, а объявление лишь описывает свойства переменной). Разумеется, все описания (объявления и определение) одной и той же переменной не должны противоречить друг другу.

Константы, определённые через директиву препроцессора `#define`, доступны для использования во всех модулях, начиная с места своего определения. Если мы хотим отменить сделанное таким способом определение, то должны использовать директиву `#undef`.

1.8. Ввод-вывод числовых значений

Ввод и вывод чисел. Для ввода с клавиатуры используется функция `scanf()`:

```
scanf("спецификатор_формата", & имя_переменной);
```

Первый параметр — спецификатор формата, который задаёт тип вводимой информации (см. ниже). Второй параметр — адрес переменной, значение которой будет присвоено. Обратите внимание на знак `&` перед именем переменной (это операция взятия адреса, см. с. 88).

Например, чтобы ввести значение целой переменной `a`, имеющей тип `int`, надо написать:

```
scanf("%d", &a);
```

Вывод значений на экран осуществляется с помощью уже знакомой нам функции `printf`. Первым её параметром является строка, содержащая спецификаторы формата для всех выводимых значений. Переменные или константы, задающие сами выводимые значения, указываются в качестве всех остальных параметров функции `printf`:

```
printf("форматная_строка", имя1, имя2, ...);
```

Например, чтобы вывести значение двух переменных `x` и `y`, первая из которых имеет тип `long int`, а вторая — тип `double`, можно написать:

```
printf("Ответ: %ld и %lg", x, y);
```

Спецификатор формата — это специальная строка, содержащая условное обозначение формата вводимого или выводимого значения. Все спецификаторы формата начинаются с символа '% '.

Для целых чисел `int`:

`%d` — ввод/вывод в десятичной форме;

`%u` — в десятичной форме без знака;

`%o` — для целых чисел без знака в восьмеричной форме;

`%x` — для целых чисел без знака в шестнадцатеричной форме.

Пример 1.1. Выведем на экран одно и то же число в десятичном, шестнадцатеричном и восьмеричном виде (16, 10 и 20):

```
int a = 16;
printf("%d %x %o", a, a, a);
```

Для целых чисел `long int`:

`%ld` — в десятичной форме;

`%lu` — в десятичной форме без знака.

Для вещественных чисел `float`:

`%f` — в форме с фиксированной точкой (в виде целой и дробной части);

`%e` — в экспоненциальной форме (нормализованная мантисса и порядок);

`%g` — автоматически выбирается та форма вывода, которая короче (для конкретного выводимого значения).

Для вещественных чисел `double`:

`%lf` — с фиксированной точкой;

`%le` — в экспоненциальной форме;

`%lg` — форма выбирается автоматически.

Пример 1.2. Вывод на экран одного и того же числа в двух разных вещественных форматах (16.300000 и 1.630000e+01):

```
double a = 16.3;
printf("%lf; %le", a, a);
```

В форматной строке можно указать ширину поля (число символов), отводимых для вывода числа, а также число разрядов после десятичной точки (точность).

Пример 1.3. Вывод значения переменной типа `double` в поле шириной 7 символов, с точностью три знака после десятичной точки:

```
printf("Ответ: %7.3lf", x);
```

Т. к. знак процента в форматной строке используется как управляющий символ, то для вывода этого символа его надо записать дважды:

```
printf("Прибыль составила %5.1f%%", x);
```

Пример 1.4. В листинге 1.5 приведена программа, которая запрашивает ввод с клавиатуры целого числа и выводит на экран результат сложения введённого числа со значением, заданным в программе.

Листинг 1.5: Ввод целых чисел с клавиатуры

```
1 #include <stdio.h>
2 #include <windows.h>
3 int main() {
4     int x, y, z; // Три целые переменные.
5     SetConsoleCP(1251);
6     SetConsoleOutputCP(1251);
7     x = 5;        // Присваивание.
8     printf("\nВведите целое число: ");
9     scanf("%d", &y); // Ввели с клавиатуры значение y.
10    z = x + y;      // Сложили.
11    printf("Результат сложения %d и %d равен %d\n", x, y, z);
12    system("pause");
13    return 0;
14 }
```

1.9. Выражения и математические функции

Для преобразования данных используются операции: арифметические, логические, битовые. Если операция имеет один операнд, то она называется *унарной* (как, например логическая операция NOT или арифметическое отрицание). Операции, имеющие два операнда, называются *бинарными* (сложение, умножение и др.). В табл. 1.2 приведён список операций, используемых в языке C/C++.

Присваивание. Операция присваивания записывается в виде

```
операнд1 = операнд2
```

Первый операнд должен быть L-значением, второй — выражением. Сначала вычисляется выражение, стоящее в правой части, затем его результат записывается в область памяти, указанную в левой части (значение, хранившееся там до этого, теряется).

L-значением (от англ. *left* — левый) является любое выражение (в частном случае, переменная), адресуемое некоторый участок памяти, в который можно записать данные.

Допускаются множественные присваивания, когда одно и то же значение записывается сразу в несколько переменных, например:

```
x = y = z = 0;
```

Таблица 1.2. Основные операции языка C/C++

Операция	Описание	Пример
1	2	3
Унарные операции		
++	Инкремент (увеличение на 1)	i++;
--	Декремент (уменьшение на 1)	k--;
~	Поразрядная инверсия	a = ~b;
!	Логическое отрицание	x = ~y;
-	Унарный минус	z = -h;

(см. продолжение на следующей странице)

Продолжение таблицы 1.2

1	2	3
+	Унарный плюс	$z = +h;$
Бинарные операции		
*	Умножение	$a = x * y;$
/	Деление	$a = x / y;$
%	Остаток то деления	$a = x \% y;$
+	Сложение	$a = x + y;$
-	Вычитание	$a = x - y;$
>>	Сдвиг вправо	$a = x >> 2;$
<<	Сдвиг влево	$a = x << 1;$
<	Меньше	$f = x < y;$
<=	Меньше или равно	$f = x <= y;$
>	Больше	$f = x > y;$
>=	Больше или равно	$f = x >= y;$
==	Равно	$f = x == y;$
!=	Не равно	$f = x != y;$
&	Поразрядная конъюнкция (И)	$a = x \& y;$
^	Поразрядное исключающее ИЛИ	$a = x \wedge y;$
	Поразрядная дизъюнкция (ИЛИ)	$a = x y;$
&&	Логическое И	$a = x \&\& y;$
	Логическое ИЛИ	$a = x y;$
=	Присваивание	$a = b;$
*=	Умножение с присваиванием	$a *= b;$
/=	Деление с присваиванием	$a /= b;$
%=	Остаток от деления с присваиванием	$a \% = b;$
+=	Сложение с присваиванием	$a += b;$
-=	Вычитание с присваиванием	$a -= b;$
>>=	Сдвиг вправо с присваиванием	$a >> = 2;$
<<=	Сдвиг влево с присваиванием	$a << = 2;$
&=	Поразрядное И с присваиванием	$a \& = b;$
=	Поразрядное ИЛИ с присваиванием	$a = b;$
^=	Поразрядное исключающее ИЛИ с присваиванием	$a \wedge = b;$

(см. продолжение на следующей странице)

Продолжение таблицы 1.2

1	2	3
Операции с указателями		
<code>&</code>	Взятие адреса	<code>p = &a;</code>
<code>*</code>	Разыменование указателя	<code>a = *p;</code>
<code>new</code>	Выделение памяти	<code>p = new int;</code>
<code>delete</code>	Освобождение памяти	<code>delete p;</code>

Приведение типов. При выполнении операции присваивания значение, получившееся при вычислении правого операнда, автоматически приводится к тому типу, который имеет левый операнд. Например:

```
float x = 5;
int y = x / 2.0;
```

В первой строке целая константа преобразуется в вещественное значение 5.0. Во второй строке вещественный результат деления 2.5 преобразуется к целому числу путём отсечения дробной части (получается 2). Поскольку неявные преобразования служат причиной многочисленных ошибок, во втором случае лучше использовать явное приведение типов:

```
int y = (int)(x / 2.0);
```

В общем случае явное преобразование типа записывается в виде:

```
(тип) выражение
```

Арифметические операции. Для обозначения четырёх арифметических операций используются символы +, −, * и /. Результат арифметической операции будет целым, если целыми являются оба операнда. Результат деления двух целых операндов будет округлён до целого значения. Если хотя бы один из операндов вещественный, то результат также будет вещественным.

Пример 1.5. (целочисленное деление):

```
int n = 7, m = 2;
int k1 = n / m;           // результат 3
double k2 = n / m;        // результат 3.0
```

```
int k = n % m;    // остаток то деления, результат 1
double x = 5 / 2; // результат 2.0, т.к. оба операнда целые
```

Пример 1.6. Деление вещественных чисел:

```
int n = 7, m = 2;
double x1 = (double)n / m; // результат 3.5
double x2 = n / (double)m; // результат 3.5
double x3 = 5.0 / 2;    // результат 2.5
double x4 = 5 / 2.0;    // результат 2.5
double x5 = 5.0 / 2.0; // результат 2.5
```

Составное присваивание. Операция присваивания может быть совмещена с бинарными операциями:

```
x += y; // эквивалентно x = x + y;
x -= y; // эквивалентно x = x - y;
x *= y; // эквивалентно x = x * y;
x /= y; // эквивалентно x = x / y;
x %= y; // эквивалентно x = x % y; (остаток от деления)
x <<= y; // эквивалентно x = x << y; (сдвиг влево)
x >>= y; // эквивалентно x = x >> y; (сдвиг вправо)
x &= y; // эквивалентно x = x & y; (побитное И)
x |= y; // эквивалентно x = x | y; (побитное ИЛИ)
x ^= y; // эквивалентно x = x ^ y; (побитное XOR)
```

Инкремент и декремент. Часто используемые операции увеличения и уменьшения на 1 в языке C/C++ можно записывать в сокращённом виде. Инкремент:

```
i++; //эквивалентно i = i + 1;
```

Декремент:

```
i--; //эквивалентно i = i - 1;
```

Когда эти операции являются частью более сложного выражения, имеет значение, до или после идентификатора переменной (в *префиксной* или в *постфиксной форме*) указана операция инкремента или декремента.

Постинкремент и постдекремент (значение переменной участвует в вычислении выражения, а затем изменяется):

```
k = i++ / j--; //эквивалентно k = i*j; i = i+1; j = j-1;
```

Преинкремент и предекремент (значение переменной изменяется, после чего участвует в вычислении выражения):

```
k = ++i * --j; //эквивалентно i = i+1; j = j-1; k = i*j;
```

Операции инкремента и декремента могут быть применены только к L-значению. Например, нельзя написать: 5++.

Операции сдвига >> (вправо) и << (влево) применяются только к целочисленным операндам. Они сдвигают двоичное представление левого операнда на количество двоичных разрядов, заданное правым операндом. Пример: 5<<1 = 0b00000101 << 1 = 0b00001010 = 10.

При сдвиге вправо на один разряд самый младший (нулевой) бит теряется; самый старший бит принимает нулевое значение для беззнаковых (unsigned) типов, а для знаковых — принимает значение 1 для отрицательного числа и 0 для положительного. Пример: -2 >> 1 = 0b11111110 >> 1 = 0b11111111 = -1.

При сдвиге влево на один разряд теряется самый старший бит; младший бит принимает нулевое значение.

Приоритет операций. Для указания порядка выполнения операций в сложных выражениях используются скобки, например:

```
x = a * (b - c / d);
```

Когда скобки отсутствуют, порядок выполнения операций зависит от их *приоритета*. В приведённом примере при вычислении выражения внутри скобок сначала будет выполнена операция деления c / d, т. к. её приоритет выше, чем у операции вычитания.

В табл. 1.2 операции расположены в порядке уменьшения приоритета. Операции, имеющие одинаковый приоритет, расположены в одной группе; группы разделяются горизонтальной чертой.

Стандартные математические функции. В таблице 1.3 указаны некоторые математические функции, описанные в заголовочном файле `math.h` (или `cmath`). Более полный список можно найти в указанном файле, он находится в каталоге

`C:\Program Files\Microsoft Visual Studio x.x\VC\include`

Если та или иная функция в языке отсутствует, то используют известные соотношения:

1) для логарифмов:

$$\log_a b = \log_c b \cdot \log_a c = \frac{\log_c b}{\log_c a}; \quad (1.1a)$$

$$\log_a b = \frac{1}{\log_b a}; \quad (1.1b)$$

Таблица 1.3. Математические функции

Обозначение	Функция	Тип результата	Пример
<code>sqrt</code>	квадратный корень	<code>double</code>	<code>y = sqrt(x/2.0);</code>
<code>pow</code>	степень	<code>double</code>	<code>y = pow(x, a/3.0);</code>
<code>abs</code>	модуль	<code>int</code>	<code>k = abs(i);</code>
<code>labs</code>	модуль	<code>long int</code>	<code>j = labs(i);</code>
<code>fabs</code>	модуль	<code>double</code>	<code>y = abs(x);</code>
<code>ceil</code>	округление вверх	<code>double</code>	<code>y = ceil(x);</code>
<code>floor</code>	округление вниз	<code>double</code>	<code>y = floor(x);</code>
<code>sin</code>	синус	<code>double</code>	<code>y = sin(x);</code>
<code>cos</code>	косинус	<code>double</code>	<code>y = cos(x*5);</code>
<code>tan</code>	тангенс	<code>double</code>	<code>y = tan(x);</code>
<code>atan</code>	арктангенс	<code>double</code>	<code>y = atan(x);</code>
<code>exp</code>	экспонента e^x	<code>double</code>	<code>y = exp(x);</code>
<code>log</code>	натуральный логарифм $\ln$	<code>double</code>	<code>y = log(x);</code>
<code>log10</code>	десятичный логарифм $\log_{10}$	<code>double</code>	<code>y = log10(x);</code>

2) для показательных функций и корней:

$$\frac{1}{a^m} = a^{-m}; \quad \frac{a^n}{a^m} = a^{n-m}; \quad (1.2a)$$

$$\sqrt[n]{a^n} = (\sqrt[n]{a})^n = a^{n/n}; \quad (1.2b)$$

$$\sqrt[n]{\sqrt[m]{a}} = a^{\frac{1}{nm}} = \sqrt[nm]{a}; \quad (1.2c)$$

$$a^n \cdot a^m = a^{n+m}; \quad (1.2d)$$

3) для тригонометрических функций:

$$\operatorname{tg} x = \frac{\sin x}{\cos x} = \frac{1}{\operatorname{ctg} x}; \quad (1.3a)$$

$$\arcsin x = \operatorname{arctg} \left(\frac{x}{\sqrt{1-x^2}} \right); \quad (1.3b)$$

$$\arccos x = \frac{\pi}{2} - \arcsin x; \quad (1.3c)$$

$$\operatorname{arcctg} x = \frac{\pi}{2} - \operatorname{arctg} x. \quad (1.3d)$$

Обращаем внимание на то, что во всех случаях использования тригонометрических функций углы измеряются в радианах.

Пример 1.7. Вычисление математического выражения. Вычислим значение выражения:

$$\left(2 - \frac{2}{3} + \sqrt[3]{\frac{x}{2.7}} - \sqrt{e^{-3x}} \right) \cdot |\sin 5x|$$

при заданном значении x (вводится с клавиатуры).

В листинге 1.6 показан пример решения данной задачи. На рис. 1.5 показан результат работы программы.

Листинг 1.6: Вычисление математического выражения

```
1 #include <stdio.h>
2 #include <windows.h>
3 #include <math.h>
4
5 int main(){
6     double x, y;
7     SetConsoleCP(1251);
```

```

8  SetConsoleOutputCP(1251);
9  printf("\nВычисления по формулам\n");
10 printf("Введите x = ");           ②
11 scanf("%lg", &x);                 ③
12 y = ( 2.0 - 2.0/3 + pow(x/2.7, 1.0/3) - sqrt(exp(-3*x)) )
13      * fabs(sin(5*x));             ④
14 printf("Ответ = %lg\n", y);       ⑤
15 system("pause");                  ⑥
16 return 0;
17 }

```

- ① — описали две вещественные переменные;
- ② — вывели на экран приглашение;
- ③ — ввели с клавиатуры значение переменной x;
- ④ — вычислили значение выражения (выражения можно разрывать);
- ⑤ — вывели ответ на экран;
- ⑥ — ждём нажатие клавиши в конце программы.

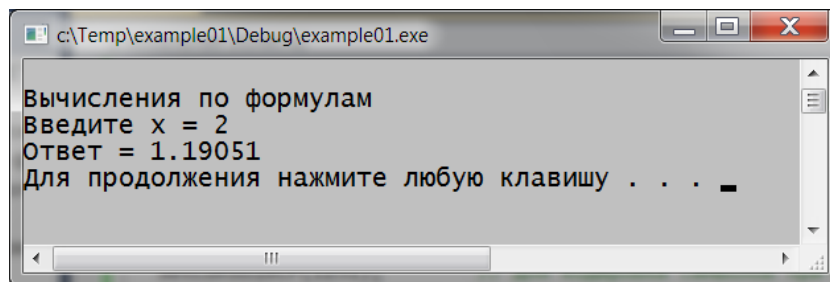


Рис. 1.5. Экран программы из листинга 1.6

$$x := 2$$

$$\left(2 - \frac{2}{3} + \sqrt[3]{\frac{x}{2.7}} - \sqrt{e^{-3 \cdot x}} \right) \cdot |\sin(5 \cdot x)| = 1.19051$$

Мате...

Арифметика

Рис. 1.6. Пример вычисления выражений в программе MathCAD

Обращаем ваше внимание на то, что следует избегать использования в нецелочисленных выражениях целых констант вместо вещественных, т.к. при делении целых операндов результат округляется до целого. Проверьте самостоятельно, что получается, например, если написать $2 - 2/3$ вместо $2 - 2.0 / 3$ или `row(x / 2.7, 1/3)` вместо `row(x / 2.7, 1.0 / 3)`.

Использование программы MathCAD. На рис. 1.6 показан пример вычисления того же самого выражения в среде MathCAD. Для ввода специальных операций и функций предназначена панель «Арифметические инструменты».

Для того, чтобы присвоить значение переменной, надо нажать на клавиатуре `Shift + :`, при этом на экране отобразится символ присваивания `:=`. Для ввода степени используется комбинация клавиш `Shift+6` (символ `"^"`). При вводе дробей и других сложных выражений приходится использовать клавишу пробел, чтобы задать, к какой части уже введенного выражения будет применён очередной оператор.

Чтобы узнать значение выражения, надо нажать знак равенства и клавишу `Enter`. После этого двойным щелчком по ответу левой кнопкой мыши можно вызвать диалоговое окно и задать количество отображаемых цифр.

1.10. Контрольные вопросы

1. Дать определения следующим понятиям, привести примеры: а) алгоритм; б) алгоритмический язык; в) язык программирования; г) синтаксическое правило; д) семантика; е) машинный код; ж) ассемблер и язык ассемблера; з) язык высокого уровня; и) парадигма программирования; к) декларативный язык; л) императивный язык; м) структурное программирование; н) подпрограмма (функция); о) компилятор; п) интерпретатор; р) лексема; с) алфавит языка; т) идентификатор; у) ключевое слово; ф) числовая константа; х) комментарий; ц) переменная; ч) именованная константа; ш) инициализатор; ы) тип данных; э) интегрированная среда разработки; ю) форматная строка; я) L-значение.

2. Как в языке C/C++ записываются числовые константы: а) десятичные; б) восьмеричные; в) шестнадцатеричные.
3. Переведите указанные шестнадцатеричные числа в десятичный вид: а) 0xC5; б) 0xA2; в) 0x7B; г) 0x3F; д) 0x5D; е) 0x9E; ж) 0xD3.
4. Переведите указанные десятичные числа в шестнадцатеричный вид: а) 37; б) 54; в) 29; г) 71; д) 57; е) 69; ж) 48.
5. Запишите указанные десятичные числа в нормализованном виде. Пример: $257.351 = 2.57351e2$. а) 0.037; б) 540; в) 0.29; г) 71; д) 5700; е) 69000; ж) 0.0048.
6. Можно ли в программе на языке C/C++ записать десятичное число 13 в виде константы 013?
7. С каких символов может начинаться идентификатор в программе?
8. Из каких символов может состоять идентификатор?
9. Как описать вещественную переменную двойной точности с именем *x* и инициализировать её значением 3.1415?
10. Как описать целую константу с именем *i* и инициализировать её значением 5?
11. В программе имеется строка:
`const k;`
Какие две ошибки допустил программист?
12. Из каких трёх этапов состоит процесс построения (build) приложения, написанного на языке C/C++?
13. Зачем в начале программы используют директиву `#include`?
14. Какая функция предназначена для вывода информации на экран?
15. Какая функция используется для ввода информации с клавиатуры?
16. В каком заголовочном файле описаны функции ввода-вывода?
17. Какое расширение имеют заголовочные файлы?
18. Какое расширение имеют файлы с исходными текстами программ на языке C и C++.

19. Объясните понятие «кодировка символов». Какие функции C/C++ используются для установки кодировки при вводе и выводе? В каком заголовочном файле они описаны?
20. Что делать, если в тексте программы правильно установлена кодировка 1251, но русские символы в консольном окне всё равно не отображаются?
21. Какой смысл имеет значение, которое пишется в операторе `return` в конце функции `main()`?
22. Какая функция предназначена для ввода кода нажатой клавиши? В каком заголовочном файле она описана?
23. Как в программе на языке C/C++ выполнить команду операционной системы?
24. Что произойдёт, если в функции `scanf()` перед вторым параметром не указать операцию взятия адреса `&`?
25. Допустим, в тексте программы первоначально использовалась переменная типа `float`, а соответствующая функция вывода имела вид:
- ```
printf("Результат = %7.2f");
```
- Потребовалось производить вычисления с большей точностью, поэтому тип переменной изменили на `double`. Какие изменения потребуются в функции вывода?
26. Чем отличаются спецификаторы формата `"%f"`, `"%e"` и `"%g"`?
27. Расшифруйте следующую запись:
- а) `x += y;`   б) `x *= y;`  
в) `x /= y;`   г) `x %= y;`
28. Каким будет результат вычисления выражения:
- а) `9 / 2;`   б) `9.0 / 2;`  
в) `9 / 2.0;`   г) `9.0 / 2.0;`
29. Каким будет результат:
- а) `x = 021 << 2;`  
б) `y = 0x3B << 2;`

в) `z = 13 >> 3;`

30. Найдите результат вычисления логического выражения:

а) `bool x = (3 < 1) || !(2 <= 7);`

б) `bool y = (3 >= 1) && !(~2 & 7);`

31. Что будет выведено на экран в результате выполнения данного фрагмента программы:

а)

```
int x = 014;
printf("%d", x);
```

б)

```
int x = 0x14;
printf("%d", x);
```

32. Сколько байт отводится компилятором для хранения переменных следующих типов: а) `short int`; б) `long int`; в) `int`; г) `char`; д) `bool`; е) `float`; ж) `double`; з) `long double` ?

33. В чём отличие типа `int` от `unsigned int` ?

34. С какого символа начинаются директивы препроцессора?

35. Чем область действия идентификатора отличается от области видимости?

36. Как описать глобальную переменную? Локальную?

37. Зачем используются статические переменные?

38. Как обозначается операция сравнения на равенство?

39. Чем поразрядные операции `&` и `|` отличаются от логических операций `&&` и `||` ?

40. Какое число будет выведено в результате выполнения следующего фрагмента программы:

а)

```
int a = 5, b = 12;
int x = a & b;
printf("%d", x);
```

б)

```
int a = 5, b = 12;
int x = a ^ b;
printf("%d", x);
```

В)

```
int a = 13, b = 5;
int x = a % b;
printf("%d", x);
```

Г)

```
int a = 9, b = 2;
int x = a / b;
printf("%d", x);
```

Д)

```
int a = 9, b = 2;
int x = a >> b;
printf("%d", x);
```

Е)

```
unsigned short int x = 0;
x--;
printf("%i", x);
```

Ж)

```
int i = 5, j = 9, k;
k = i++ * --j;
printf("%d%d%d", i, j, k);
```

З)

```
int i = 5, j = 9, k;
k = ++i * j--;
printf("%d%d%d", i, j, k);
```

## 2. Потоки ввода-вывода, условные операторы, циклы

### 2.1. Потоки ввода-вывода

Вместо функций `scanf()` и `printf()` в программах на C++ можно использовать специальные объекты — поток ввода `cin` и поток вывода `cout`. Они описаны в заголовочном файле `iostream.h` (или `iostream`), в пространстве имён `std`. Подробнее о них мы будем говорить в следующей части, при изучении объектно-ориентированного программирования (ООП), а пока нам достаточно уметь правильно их использовать.

Для ввода с клавиатуры значения переменной (символьного, целого или вещественного типа) следует писать:

```
std::cin >> x;
```

Для вывода на экран текстовой строки и значения переменной:

```
std::cout << "Ответ = " << x;
```

Как видим, спецификаторы формата здесь не используются. Заметим, что операции `>>` и `<<` имеют в языке C другой смысл: поразрядный сдвиг вправо и влево соответственно, но язык C++ позволяет произвольным образом переопределять (перегружать) операции для классов, описанных программистом. В данном случае эти две операции переопределены для потоков ввода и вывода. Пример программы приведён в листинге 2.1.

#### Листинг 2.1: Использование потоков ввода-вывода C++ (вариант 1)

```
#include <iostream> ①
#include <cmath> ②

int main() {
 double x, y;
 const double pi = 3.14159265358979323846; ③
 system("chcp 1251");
```

```
std::cout << "\nx = : "; ④
std::cin >> x; ⑤
y = cos(2 * pi * x); ⑥
std::cout << "y = " << y << std::endl; ⑦
system("pause");
return 0;
}
```

- ① — подключили заголовочный файл, в котором определены потоки ввода-вывода (в старых программах можно встретить `iostream.h`);
- ② — вместо `math.h` в новых программах используют `cmath`;
- ③ — определили константу  $\pi$ ;
- ④ — вывели на экран приглашение с помощью потока вывода `cout`;
- ⑤ — ввели с клавиатуры значение переменной `x` с помощью потока ввода `cin`;
- ⑥ — вычислили значение выражения;
- ⑦ — вывели на экран ответ с помощью потока вывода `cout` и перешли на новую строку (`endl` — end of line — конец строки).

Файл с исходным текстом программы должен иметь расширение `.cpp`.

Чтобы каждый раз не указывать название пространства имён `std`, можно в начале программы написать

```
using namespace std;
```

В этом случае операции ввода и вывода примут вид:

```
cin >> x;
cout << "Ответ = " << x;
```

а программа, приведённая в листинге 2.1, изменится, как показано в листинге 2.2.

#### Листинг 2.2: Использование потоков ввода-вывода C++ (вариант 2)

```
#include <iostream>
#include <cmath>
```

```
using namespace std;

int main() {
 double x, y;
 const double pi = 3.14159265358979323846;
 system("chcp 1251");
 cout << "\nx = ";
 cin >> x;
 y = cos(2 * pi * x);
 cout << "y = " << y << endl;
 system("pause");
 return 0;
}
```

Для изменения вида и задания точности выводимых чисел вместе с потоком вывода `cout` используют *манипуляторы форматирования*, описанные в заголовочном файле `iomanip`. Манипулятор `setw` позволяет задать ширину поля (количество позиций — актуально для вывода таблиц); манипулятор `setprecision` — точность выводимых чисел (количество знаков после десятичной точки); манипуляторы `scientific` и `fixed` изменяют формат выводимых чисел (экспоненциальный или с фиксированной точкой). В листинге 2.3 приведена короткая программа, в которой два вещественных числа сначала выводятся в экспоненциальном формате с точностью 4 знака после десятичной точки с шириной позиции 14 символов, а затем те же числа выводятся в формате с фиксированной точкой, с точностью 5 знаков и шириной 12 символов (рис. 2.1).

### Листинг 2.3: Использование манипуляторов форматирования

```
#include <iostream>
#include <iomanip>
using namespace std;

int main () {
```

```
double x =123.546789, y = 3.1415926;
cout << scientific << setw(14) << setprecision(4)
 << x << ", " << y << endl;
cout << fixed << setw(12) << setprecision(5)
 << x << ", " << y << endl;
system("pause");
return 0;
}
```

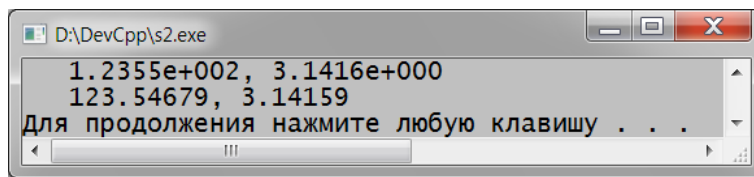


Рис. 2.1. Экран программы из листинга 2.3

## 2.2. Системы счисления. Кодирование чисел

**Позиционные системы счисления.** Для представления чисел обычно используются *позиционные системы счисления*, в которых значение каждой цифры в записи числа определяется позицией этой цифры:

$$A = \sum_{i=0}^{n-1} a_i \cdot d^i = a_{n-1} \cdot d^{n-1} + \dots + a_1 \cdot d^1 + a_0 \cdot d^0, \quad (2.1)$$

где  $a_i$  —  $i$ -я цифра числа,  $a_i = 0, 1, \dots, d-1$ ;  $n$  — разрядность числа;  $d$  — основание системы счисления (в двоичной системе  $d = 2$ ; в десятичной  $d = 10$ ; в восьмеричной  $d = 8$ ; в шестнадцатеричной  $d = 16$ ). Для представления чисел в позиционной системе по основанию  $d$  используется  $d$  различных цифр от 0 до  $(d-1)$ . Используя  $n$  разрядов, можно записать  $d^n$  различных чисел: от 0 до  $d^{n-1}$ .

**Двоичная система счисления.** В двоичной системе  $d = 2$ ; для записи чисел используются только две цифры: 0 и 1. Это очень удобно с точки зрения аппаратной реализации цифровых устройств: они должны находиться только

в двух возможных состояниях: включено и выключено. Каждая цифра двоичного числа (двоичный разряд) называется *битом*. Биты нумеруются справа налево, от 0 до  $n - 1$ . Нулевой (самый правый) бит называется *младшим* (least significant digit, LSD); последний (самый правый, с весом  $n - 1$ ) бит называется *старшим* (most significant digit, MSD).

Числа, представленные в двоичном виде, легко умножать или делить на 2 — эти операции эквивалентны сдвигу числа влево (младший бит получившегося числа принимает нулевое значение) или вправо (младший бит исходного числа при этом теряется).

**Перевод чисел из двоичной системы в десятичную.** Производится по формуле (2.1). Например:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = (8 + 0 + 2 + 1)_{10} = 11_{10}.$$

Пример перевода двоичного числа, содержащего как целую, так и дробную часть:

$$110,101_2 = 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 5,625_{10}.$$

**Перевод чисел из десятичной системы в двоичную.** Для перевода целого десятичного числа в двоичное необходимо выполнить последовательные деления на 2 до тех пор, пока остаток после очередного деления не станет меньше 2 (рис. 2.2 а); в качестве ответа берут все полученные остатки в обратном порядке: от последнего до первого (начиная с последнего результата деления).

Чтобы перевести дробное число (с нулевой целой частью, рис. 2.2 б), необходимо, наоборот, умножать исходное десятичное число на 2 и брать в качестве очередной цифры результата целую часть произведения, а дробную часть использовать для дальнейшего умножения до тех пор, пока не будет достигнута нужная точность или не получится нулевой результат.

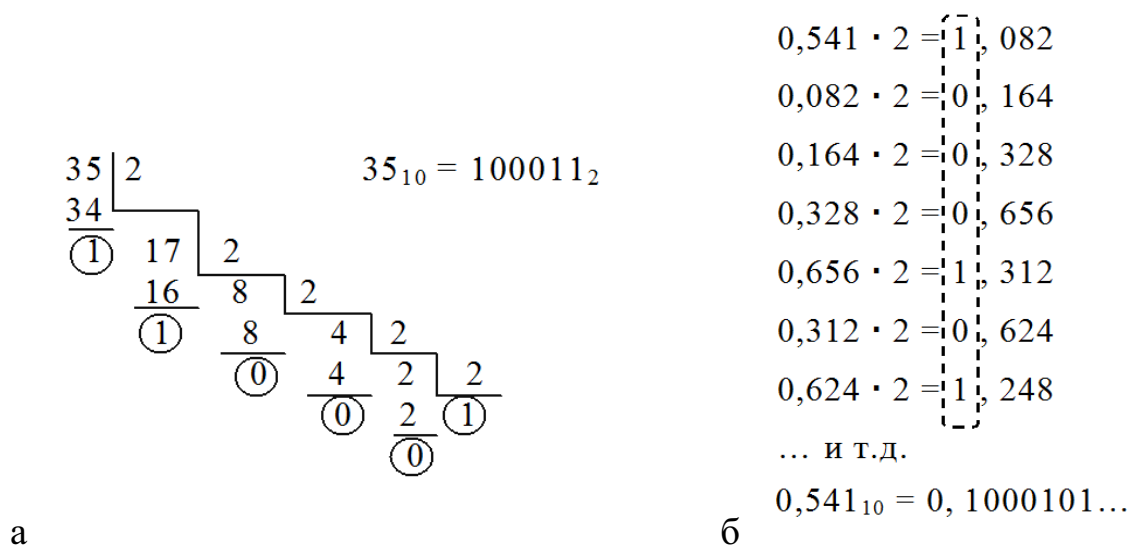
Заметим, что конечная десятичная дробь может (как в нашем примере) превратиться в бесконечную двоичную, но не наоборот.

Для перевода числа, содержащего как целую, так и дробную часть, необходимо осуществить перевод целой и дробной частей в отдельности.

**Восьмеричная система счисления.** В восьмеричной системе  $d = 8$ ; для записи чисел используются восемь цифр: от 0 до 7 включительно. Всё вышесказанное относительно перевода чисел относится и к восьмеричной системе счисления, достаточно только деление (умножение) на 2 заменить, соответственно, делением (умножением) на 8.

**Шестнадцатеричная система счисления.** В шестнадцатеричной системе  $d = 16$ ; для представления чисел необходимо использовать шестнадцать различных цифр. Поэтому для цифр после девяти применяют буквы латинского алфавита:  $a, b, c, d, e, f$ , соответственно, для 10, 11, 12, 13, 14 и 15.

**Метод подстановки.** Т. к. числа в двоичном представлении состоят из большого количества нулей и единиц, то для облегчения восприятия их удобнее записывать в другой системе счисления, основание которой является степенью двойки, например, в восьмеричной или шестнадцатеричной системе. Дело в том, что для взаимного перевода между этими системами можно использовать метод подстановки, не нуждающийся в последовательных делениях или умножениях. Достаточно запомнить таблицу соответствия для первых шестнадцати двоичных чисел (табл. 2.1).



**Рис. 2.2.** Перевод десятичного числа в двоичное

**Таблица 2.1.** Таблица соответствия чисел в различных системах счисления

| Двоичное | Десятичное | Шестнадцатеричное | Восьмеричное |
|----------|------------|-------------------|--------------|
| 0000     | 0          | 0                 | 0            |
| 0001     | 1          | 1                 | 1            |
| 0010     | 2          | 2                 | 2            |
| 0011     | 3          | 3                 | 3            |
| 0100     | 4          | 4                 | 4            |
| 0101     | 5          | 5                 | 5            |
| 0110     | 6          | 6                 | 6            |
| 0111     | 7          | 7                 | 7            |
| 1000     | 8          | 8                 | 10           |
| 1001     | 9          | 9                 | 11           |
| 1010     | 10         | <i>a</i>          | 12           |
| 1011     | 11         | <i>b</i>          | 13           |
| 1100     | 12         | <i>c</i>          | 14           |
| 1101     | 13         | <i>d</i>          | 15           |
| 1110     | 14         | <i>e</i>          | 16           |
| 1111     | 15         | <i>f</i>          | 17           |

Для перевода двоичного числа в шестнадцатеричное надо разбить исходное число на *тетрады* (группы по четыре двоичных разряда), при необходимости дополнив целую часть числа нулевыми битами слева, а дробную часть — справа, и осуществить перевод каждой тетрады в отдельности в соответствии с табл. 2.1. Например:

$$1010\ 1100\ 0010_2 = AC2_{16};$$

$$0010\ 0110, 0100\ 1000_2 = 26,48_{16}.$$

Для перевода в восьмеричную систему надо так же поступить с группами по три двоичных разряда (т. к.  $2^3 = 8$ ).

Обратный перевод осуществляется аналогично, например:

$$5C8_{16} = 0101\ 1100\ 1000_2;$$

$$327_8 = 010\ 010\ 111_2.$$

**Двоично-десятичное представление чисел** (binary coded decimal, BCD) — форма записи целых чисел, когда каждый десятичный разряд числа записывается в виде его четырёхбитного двоичного кода. Например, десятичное число  $37_{10}$  будет записано в двоичной системе счисления как  $0010\ 0101_2$ , а в двоично-десятичном коде — как  $0011\ 0111_{BCD}$ .

Двоично-десятичное представление занимает больше памяти, чем двоичное (каждые четыре бита кода BCD представляют всего 10 различных комбинаций вместо 16), но числа, представленные в формате BCD, проще перевести в десятичный вид (методом подстановки); при переводе чисел с дробной частью не теряется точность; BCD-числа проще умножать или делить на 10.

**Обратный код.** Обратный код применяется для представления отрицательных двоичных чисел. Обратный код отрицательного числа образуется путём инвертирования всех разрядов модуля исходного двоичного числа.

Например, если записать число 5 в двоичном 4-разрядном виде, то получится 0101. Следовательно, отрицательное число  $-5$  в обратном коде выглядит как 1010. Единица в старшем разряде — признак отрицательного числа. Для обратного перевода нужно снова инвертировать все биты, получится опять 0101.

Положительные числа в обратном коде не изменяют свой вид.

**Дополнительный код.** Дополнительный код применяется для представления отрицательных двоичных чисел. Чтобы записать дополнительный код отрицательного числа, надо инвертировать все разряды модуля исходного двоичного числа (как и в случае обратного кода), а затем прибавить к результату 1 (учитывая все переносы в старшие разряды).

Число  $-5$  в дополнительном коде:  $1010 + 1 = 1011$ . Единица в старшем разряде — признак отрицательного числа. Для обратного перевода нужно снова инвертировать все биты и прибавить 1, получится опять  $0100 + 1 = 0101$ .

Другой пример:  $-6_{10} \rightarrow 0110_2 \rightarrow 1001_2 + 1 = 1010$ .

Положительные числа в дополнительном коде не изменяют свой вид.

**Представление вещественных чисел.** Любое число  $x$  в системе счисления с основанием  $d$  можно записать в виде  $x = s \cdot m \cdot d^p$ , где  $s = 1$  или  $-1$  — задаёт знак числа;  $m$  — множитель, содержащий все цифры числа (*мантисса*);  $p$  — целое число, называемое *порядком*. Такой способ записи чисел называется *представлением числа с плавающей точкой*. Например, вещественное число  $123.4567 = 1.234567 \cdot 10^2 = 0.1234567 \cdot 10^3$ . Вещественные числа при вводе и выводе обычно представляются в десятичной форме и записываются в виде  $1.234567e2$ , но в памяти компьютера значения мантиссы и порядка хранятся в двоичной системе счисления.

Вещественное число называется *нормализованным*, если мантисса лежит в пределах  $1 \leq m < d$ . При  $d = 2$  первый бит мантиссы нормализованного числа всегда равен единице, поэтому он обычно не хранится в памяти компьютера (для экономии места).

Международный стандарт IEEE 754 (IEC 60559:1989) определяет три типа чисел с плавающей точкой: `real*4`, `real*8` и `real*10`, которые занимают в памяти 4, 8 и 10 байт соответственно (в языке C/C++ это типы `float`, `double` и `long double`).

Во всех трёх типах в самом старшем бите первого байта хранится знак числа (1 для отрицательных, 0 для положительных чисел).

Для типа `real*4` остальные семь бит первого байта и старший бит второго байта отводятся под хранение двоичного представления порядка десятичного числа. Порядок хранится в памяти со смещением 127, т. е. нулевое значение в памяти соответствует значению порядка  $-127$ . Оставшиеся 23 бита (7 бит второго байта и все биты третьего и четвёртого байтов) отводятся под мантиссу. Старший бит мантиссы подразумевается, но не хранится (т. е. реально мантисса в этом формате представляется 24 битами).

Для типа `real*8` порядок занимает 11 бит (7 бит первого байта и 4 бита второго). Порядок представляется со смещением 1023. Мантисса занимает 52 бита (плюс единичный старший бит, который не хранится, но подразумевается).

Для типа `real*10` порядок занимает 15 бит (7 бит первого байта и 8 бит второго). Смещение порядка 16383. Мантисса занимает 64 бита (единичный старший бит, в отличие от предыдущих форматов, хранится в памяти).

## 2.3. Логические выражения

**Логический тип данных.** Переменная логического типа `bool` по определению может принимать только два возможных значения: `true` (истина) и `false` (ложь). В памяти компьютера логическая переменная занимает 1 байт; любое ненулевое значение трактуется как `true`, нулевое значение обозначает `false`. Ниже для краткости будем вместо `true` писать 1, а вместо `false` — 0.

**Логические операции.** Элементарные логические операции:

1) Отрицание (унарная операция НЕ, NOT) *инвертирует* логическое значение, т. е. заменяет его на противоположное:  $\neg 0 = 1$ ;  $\neg 1 = 0$ ;

2) Дизъюнкция (логическое сложение, бинарная операция ИЛИ, OR) двух логических значений равна единице, если хотя бы одно из значений равно единице:  $0 \vee 0 = 0$ ;  $0 \vee 1 = 1$ ;  $1 \vee 0 = 1$ ;  $1 \vee 1 = 1$ ;

3) Конъюнкция (логическое умножение, бинарная операция И, AND) двух логических значений равна нулю, если хотя бы одно из значений равно нулю:  $0 \wedge 0 = 0$ ;  $0 \wedge 1 = 0$ ;  $1 \wedge 0 = 0$ ;  $1 \wedge 1 = 1$ .

Из перечисленных трёх операций можно определить любую другую логическую операцию, например, бинарную операцию «Исключающее ИЛИ» (XOR), которая даёт 0, если аргумента равны (т. е. оба имеют значение `true` или оба имеют значение `false`), и 1 — если не равны:

$$a \text{ XOR } b = (\neg a \wedge b) \vee (a \wedge \neg b).$$

В языке C/C++ логические операции NOT, OR и AND обозначаются, соответственно, «!», «||» и «&&». Например:  $1 \ \&\& \ 0 = 0$ ,  $1 \ \&\& \ !0 = 1$ ,  $1 \ || \ 0 = 1$ .

**Работа с битами на языке C/C++.** При работе с аппаратным обеспечением или для создания графических приложений часто приходится обращаться к отдельным битам переменных. Например, часто требуется установить в 1 или сбросить в 0 некоторый бит, не изменив при этом состояние остальных битов. Напомним, что биты в байте нумеруются справа налево, начиная с нуля.

Краткая сводка битовых операций на языке C/C++ приведена в табл. 2.2, а в табл. 2.3 показаны результаты выполнения операций И, ИЛИ и Исключающее ИЛИ для всех возможных значений битов аргументов.

Битовые (поразрядные) логические операции выполняются отдельно над каждой парой соответствующих битов своих операндов. Например, в результате выполнения операции

```
x = 0b00101001 & 0b10100101;
```

переменная `x` примет значение 00100001.

Для установки в 1, например, 2-го бита переменной `x` можно использовать операцию поразрядного ИЛИ:

```
x = x | 0x04;
```

или, что то же самое:

```
x |= 0x04;
```

Действительно, шестнадцатеричное число 4 в двоичном виде записывается как 00000100 (т. е. 1 во 2-м бите); операция ИЛИ с битом 1 даёт в результате

**Таблица 2.2.** Битовые операции

| Обозначение | Название                       | Примеры                                    |
|-------------|--------------------------------|--------------------------------------------|
| ~           | Инверсия битов                 | <code>x = ~x;</code>                       |
|             | Поразрядное ИЛИ                | <code>x = a   b;    y  = x;</code>         |
| &           | Поразрядное И                  | <code>x = a &amp; b;    y &amp;= x;</code> |
| ^           | Поразрядное<br>ИСКЛЮЧАЮЩЕЕ ИЛИ | <code>x = a ^ b;    y ^= x;</code>         |
| <<          | Сдвиг влево                    | <code>x = a &lt;&lt; b;</code>             |
| >>          | Сдвиг вправо                   | <code>x = a &gt;&gt; b;</code>             |

**Таблица 2.3.** Таблица истинности битовых операций

| a | b | a   b | a & b | a ^ b |
|---|---|-------|-------|-------|
| 0 | 0 | 0     | 0     | 0     |
| 0 | 1 | 1     | 0     | 1     |
| 1 | 0 | 1     | 0     | 1     |
| 1 | 1 | 1     | 1     | 0     |

1, независимо от состояния бита переменной  $x$ ; операция ИЛИ с битом 0 не изменяет соответствующий бит переменной  $x$ .

Наоборот, для сброса в 0, например, 5-го бита переменной  $x$  можно использовать операцию поразрядного И:

```
 $x = x \& 0xDF;$
```

или, что то же самое:

```
 $x \&= 0xDF;$
```

Действительно, шестнадцатеричное число DF в двоичном виде записывается как 11011111 (т. е. 0 в 5-м бите); операция И с битом 0 даёт в результате 0, независимо от состояния бита переменной  $x$ ; операция И с битом 1 не изменяет соответствующий бит переменной  $x$ .

Инвертирование битов производится с помощью операции поразрядное ИСКЛЮЧАЮЩЕЕ ИЛИ (XOR), она обозначается  $\wedge$ . Например, чтобы инвертировать 0-й и 2-й бит переменной  $x$ , надо написать:

```
 $x = x \wedge 0x05;$
```

или, что то же самое:

```
 $x \wedge= 0x05;$
```

Действительно, число 5 в двоичном виде записывается как 00000101; операция XOR с битом 1 инвертирует исходный бит; операция XOR с битом 0 не изменяет соответствующий бит переменной  $x$ .

Инвертировать сразу все биты переменной  $x$  можно ещё проще:

```
 $x = \sim x;$
```

Для сдвига всех битов вправо на нужное число разрядов используется операция `>>`. Например, сдвиг вправо на один бит:

```
x = x >> 1;
```

При этом младший бит теряется, а самый старший принимает значение 0 для положительных (или беззнаковых) переменных и 1 для отрицательных.

Сдвиг влево обозначается `<<`. При этом самый старший бит теряется, а младший бит принимает значение 0.

Например, маску 00101001 для установки 0-го, 3-го и 5-го бита можно записать следующим образом:

```
a = (1 << 5) | (1 << 3) | (1 << 0);
```

**Пример 2.1. Вычисление значения логического выражения.** В листинге 2.4 приведён пример консольного приложения, которое запрашивает с клавиатуры значения двух вещественных чисел и одно логическое значение, после чего вычисляет логическое выражение

$$a \wedge (x \leq 2) \vee (2y = 10) \wedge \left(\frac{x}{4} \neq 0.25\right) \vee (xy \geq 0).$$

**Листинг 2.4: Вычисление значения логического выражения**

```
1 #include<stdio.h>
2 int main(){
3 float x, y;
4 bool a, f;
5 printf("\n==== Логическое выражение =====\n");
6 printf("Введите число x = ");
7 scanf("%f",&x);
8 printf("Введите число y = ");
9 scanf("%f",&y);
10 printf("Введите логическое значение a = ");
11 scanf("%i", &a);
12 f= a && (x <= 2) || (2*y == 10) && (x/4 != 0.25)
13 || (x*y >= 0);
14 printf("\nРезультат = %i\n", f);
```

```
15 return 0;
16 }
```

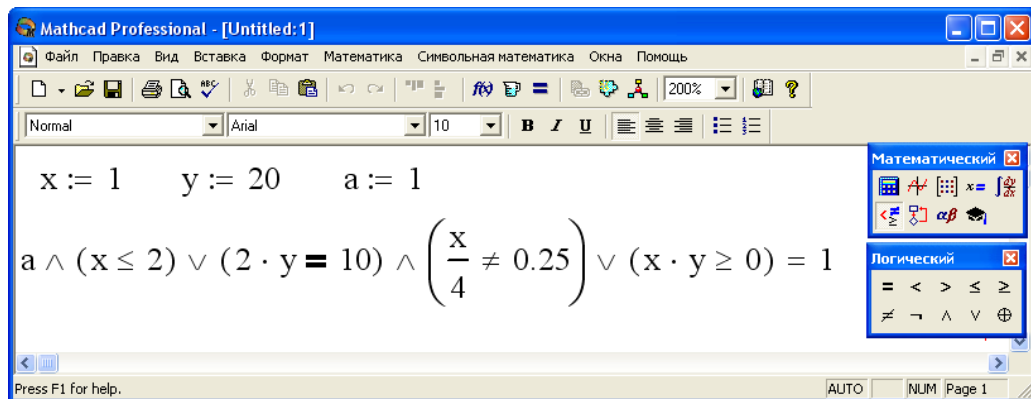


Рис. 2.3. Пример вычисления логического выражения в программе MathCAD

**Использование MathCAD для логических вычислений.** Для проверки правильности работы программы удобно использовать MathCAD (рис. 2.3). В MathCAD для представления логических значений true и false используются числовые значения 1 и 0. Для ввода логических операций можно использовать кнопки панели инструментов «Логическая» (если её не видно, надо выполнить команду меню Вид ⇒ Панели инструментов ⇒ Логическая) или комбинации клавиш Ctrl+Shift+6 для операции ИЛИ; Ctrl+Shift+6 для операции И; Ctrl+Shift+5 для операции Исключающее ИЛИ; Ctrl+Shift+1 для операции НЕ; Ctrl + = для сравнения на равенство.

## 2.4. Условные операторы

Вычислительный процесс называют *разветвляющимся*, если в зависимости от выполнения некоторого условия он реализуется по одному из нескольких направлений. Напомним, что ветвление является одной из базовых алгоритмических конструкций структурного программирования (см. рис. 1.1 б).

Для реализации разветвляющихся алгоритмов в языке C/C++ используется условный оператор if (если):

```
if (условие) {
 ... ;
 ... ;
} else {
 ... ;
 ... ;
}
```

Условие задаётся логическим или целочисленным выражением и записывается обязательно в скобках. Если результат не равен нулю (т. е. имеет значение `true` — истина), то выполняется первый блок кода. В противном случае выполняется второй блок, указанный после ключевого слова `else` (иначе). Если первый или второй блок состоит всего из одного оператора, то фигурные скобки вокруг него можно не писать.

Ветвь `else` не обязательна; если её опустить, то при нулевом значении условного выражения (`false` — ложь) блок кода, указанный в операторе `if`, не выполняется, а управление передаётся следующим операторам.

**Пример 2.2.** Если значение `k` равно 5, то присвоить `a = 10`, `b = 20`, а иначе наоборот:

```
if (k == 5) { // если k равно 5
 a = 10;
 b = 20;
}
else { // иначе
 a = 20;
 b = 10;
}
```

Обратите внимание, что сравнение на равенство записывается в виде двух знаков «`==`». Если вместо этого указать один символ «`=`», то получится оператор присваивания. Компилятор не выдаст сообщения об ошибке, поскольку в данном контексте оператор присваивания также допустим: кроме самого присваивания он возвращает присвоенное значение, в результате такое условие в операторе `if` будет считаться ис-

тиной, если выражение, стоящее справа от знака «=», не равно нулю. Чтобы уберечь себя от подобной ошибки, иногда рекомендуют записывать сравнение на равенство в виде `if ( 5 == k )`. Если здесь забыть о втором знаке «=», то получим ошибку во время компиляции.

**Пример 2.3.** То же самое можно записать другим способом:

```
if (k != 5) { // если k не равно 5
 a = 20;
 b = 10;
}
else { // иначе, т.е. если равно
 a = 10;
 b = 20;
}
```

**Пример 2.4.** В условном выражении можно использовать связки И (&&), ИЛИ (||) и другие логические операции (см. табл. 1.2 на с. 29):

```
if ((x <= 5) && (x > -2))
 a++; // инкремент, т.е. a = a + 1
else
 a--; // декремент, т.е. a = a - 1
```

**Пример 2.5.** Здесь ветвь `else` отсутствует:

```
if (x > 5)
 a = 10;
```

**Пример 2.6.** Вложенный `if`:

```
if (x < -1)
 a = 10;
else if (x >= 5)
 a = 20;
else
 a = 30;
```

Более компактное форматирование:

```
if (x < -1) a = 10;
else if (x >= 5) a = 20;
else a = 30;
```

Можно вкладывать друг в друга произвольное число операторов `if`.

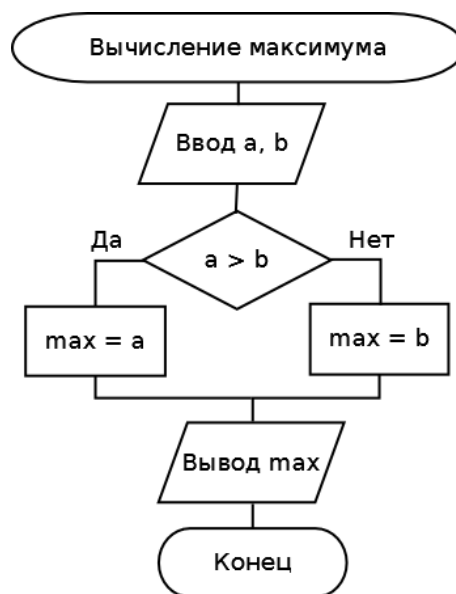
Для проверки значения нужного бита переменной используются рассмотренные выше поразрядные логические операции. Например, приведённый ниже условный оператор будет выполнен, если 5-й бит переменной `x` имеет значение 1:

```
if (x & 0xDF) { ... }
```

Аналогично, если требуется проверить, не равен ли 2-й бит переменной `x` нулю:

```
if (! (x | 0x04)) { ... }
```

**Максимальное из двух чисел.** В листинге 2.5 приведён пример программы, которая запрашивает ввод с клавиатуры двух вещественных чисел и выводит на экран максимальное из них. На рис. 2.4 показана схема алгоритма.



**Рис. 2.4.** Алгоритм нахождения максимального из двух чисел

#### Листинг 2.5: Максимальное из двух чисел (вариант 1)

```
1 #include <stdio.h>
2 #include <windows.h>
```

```
3
4 int main(){
5 double a, b, max;
6 printf("a = ");
7 scanf("%lg", &a);
8 printf("b = ");
9 scanf("%lg", &b);
10 if (a > b)
11 max = a;
12 else
13 max = b;
14 printf("max = %lg\n", max);
15 system("pause");
16 return 0;
17 }
```

В листинге 2.6 показано, как изменится предыдущая программа, если вместо функций `scanf()` и `printf()` использовать потоки ввода-вывода `cin` и `cout`.

#### Листинг 2.6: Максимальное из двух чисел (вариант 2)

```
1 #include <iostream>
2 using namespace std;
3
4 int main(){
5 double a, b, max;
6 cout << "a = ";
7 cin >> a;
8 cout << "b = ";
9 cin >> b;
10 if (a > b)
11 max = a;
12 else
```

```
13 max = b;
14 cout << "max = " << max << endl;
15 system("pause");
16 return 0;
17 }
```

**Оператор switch.** Оператор множественного выбора `switch` используется для перебора большого числа вариантов:

```
switch (выражение) {
case константное_выражение1: [список_операторов1]
case константное_выражение2: [список_операторов2]
case константное_выражениеN: [список_операторовN]
[default: список_операторов]
}
```

Здесь и далее в квадратных скобках указываются фрагменты программы, которые можно опустить. В данном случае после любого константного выражения допускается не указывать список операторов (т. е. некоторый список операторов может соответствовать нескольким вариантам значения константного выражения); блок `default` также необязателен.

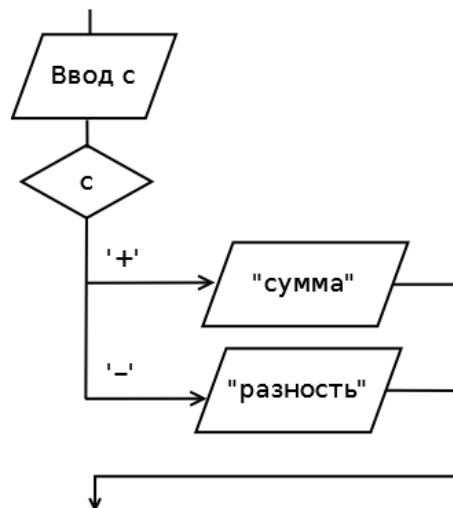
**Пример 2.7.** Ввести с клавиатуры значение `i`; если ввели 1, то присвоить  $x = 5$ ; если ввели 2, то присвоить  $x = 10$ ; во всех остальных случаях присвоить  $x = 0$ :

```
int i, x;
cin >> i;
switch (i) {
case 1: x = 5; break;
case 2: x = 10; break;
default: x = 0;
}
```

Не забывайте `break` после каждого варианта!

**Пример 2.8.** Если с клавиатуры ввели символ '+', то вывести слово «сумма»; если ввели '-', то вывести слово «разность»; во всех остальных случаях ничего не делать:

```
char c;
cin >> c;
switch (c) {
case '+': cout << "сумма"; break;
case '-': cout << "разность";
}
```



**Рис. 2.5.** Схема алгоритма для примера 2.8.

Если опустить оператор `break`, то будут выведены оба слова подряд (даже без пробела между ними). На рис. 2.5 показано обозначение оператора множественного выбора на схемах алгоритмов.

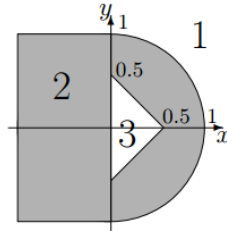
**Условная операция.** В языке C/C++ имеется тернарная (имеющая три операнда) условная операция:

`условие ? выражение1 : выражение2`

Если выражение, заданное в качестве условия, не равно нулю (т. е. имеет значение `true`), то результат операции — первое выражение, в противном случае — второе. Например, чтобы найти максимальное из двух чисел:

```
double max = (a > b)? a : b;
```

**Пример 2.9. Области на плоскости.** По координатам точки на плоскости необходимо определить, к какой из областей принадлежит точка (рис. 2.6). Для этого необходимо сначала записать уравнения для границ областей.



**Рис. 2.6.** Области на плоскости

Уравнение прямой на плоскости:  $y = ax + b$ , где параметр  $a$  равен тангенсу угла наклона прямой ( $a = 0$  для горизонтальных прямых), а параметр  $b$  — это ордината точки пересечения прямой с осью  $y$ .

Уравнение вертикальной прямой:  $x = x_0$ , где  $x_0$  — абсцисса точки пересечения прямой с осью  $x$ .

Если известны координаты двух точек  $x_1, y_1$  и  $x_2, y_2$ , через которые проходит прямая, то параметры  $a$  и  $b$  легко находятся решением системы двух линейных уравнений:

$$\begin{cases} y_1 = a \cdot x_1 + b; \\ y_2 = a \cdot x_2 + b. \end{cases}$$

Выражая  $b$  из второго уравнения и подставляя результат в первое, получим:

$$a = \frac{y_1 - y_2}{x_1 - x_2}; \quad b = y_2 - a \cdot x_2.$$

Множество точек, лежащих ниже прямой, задаётся неравенством  $y < ax + b$ ; выше прямой — неравенством  $y > ax + b$ .

Уравнение окружности радиуса  $r$ , центр которой имеет координаты  $(x_0, y_0)$ , выглядит следующим образом:  $(x - x_0)^2 + (y - y_0)^2 = r^2$ .

Множество точек, лежащих внутри окружности:  $(x - x_0)^2 + (y - y_0)^2 < r^2$ ; снаружи:  $(x - x_0)^2 + (y - y_0)^2 > r^2$ .

Рассмотрим для примера задание из варианта 12. Уравнение наклонной прямой, проходящей через точки с координатами  $(0, 0.5)$  и  $(0.5, 0)$  (верхняя граница области 3):  $y = -x + 0.5$ , а уравнение прямой, проходящей через точки  $(0, -0.5)$  и  $(0.5, 0)$  (нижняя граница области 3):  $y = x - 0.5$ . Поэтому множество точек, лежащих внутри области 3 задаётся системой неравенств:

$$\begin{cases} x > 0; \\ y < -x + 0.5; \\ y > x - 0.5. \end{cases}$$

Множество точек, лежащих внутри окружности единичного радиуса (часть области 2), задаётся неравенством  $x^2 + y^2 \leq 1$ . Для оставшейся части области 2 выполняются неравенства:

$$\begin{cases} 0 \leq x \leq 1; \\ -1 \leq y \leq 1. \end{cases}$$

Точки, координаты которых не удовлетворяют неравенствам, записанным для областей 2 или 3, очевидно, лежат в области 1.

Схема алгоритма решения задачи показана на рис. 2.7. Образец программы приведён в листинге 2.7, а на рис. 2.8 демонстрируется реализация условных вычислений в MathCAD.

#### Листинг 2.7: Определение номера области по координатам точки

```

1 #include<stdio.h>
2 int main(){
3 float x, y;
4 int region;
5 printf("\n x = "); scanf("%f", &x);
6 printf(" y = "); scanf("%f", &y);
7 if ((x > 0) && (y < -x+0.5) && (y > x-0.5))
8 region = 3;
9 else
10 if (x*x + y*y <= 1)
11 region = 2;
```

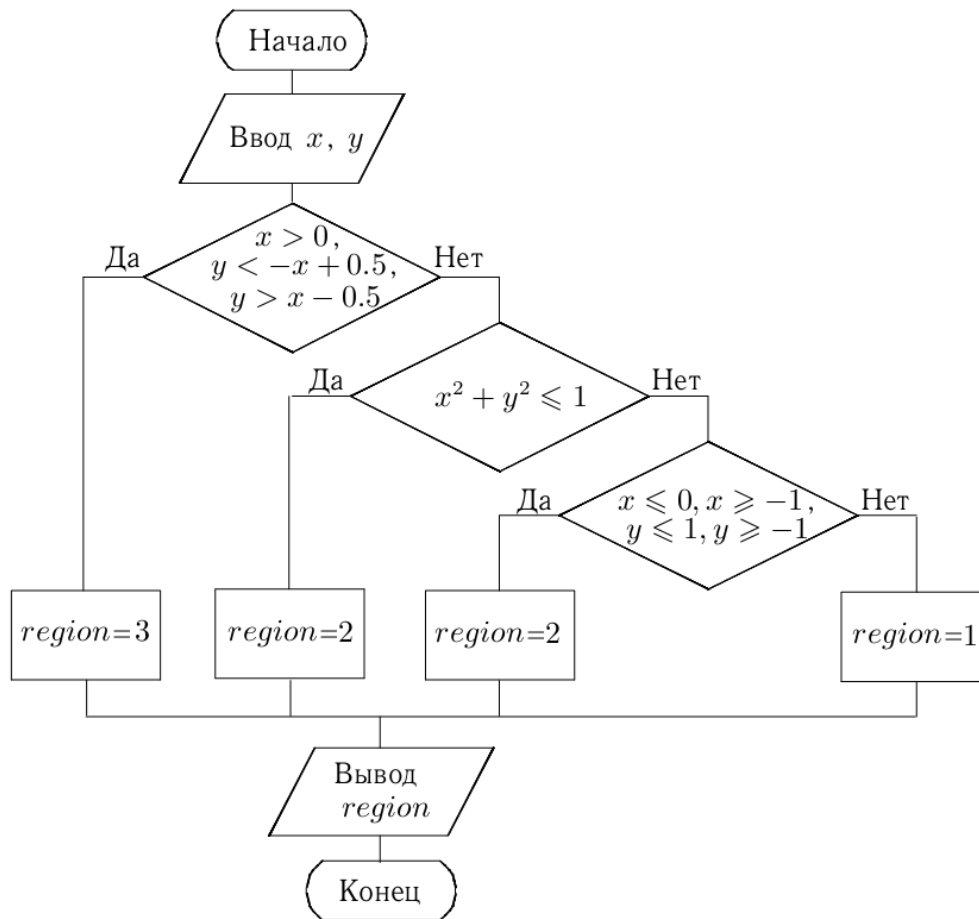


Рис. 2.7. Алгоритм определения номера области

$x := 1 \quad y := 0$

$region := \text{if}(x > 0 \wedge x - 0.5 < y < -x + 0.5, 3, \text{if}(x^2 + y^2 \leq 1, 2, \text{if}(-1 \leq x \leq 0 \wedge -1 \leq y \leq 1, 2, 1)))$   
 $region = 2$

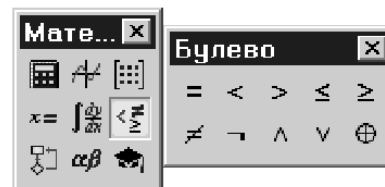


Рис. 2.8. Условный оператор в MathCAD

```

12 else
13 if ((x <= 0) && (x >= -1) && (y <= 1) && (y >= -1))
14 region = 2;
15 else
16 region = 1;
17 printf("\nRegion = %i\n", region);
18 return 0;
19 }

```

## 2.5. Операторы цикла

В языке C/C++ имеется три вида оператора цикла: с предусловием, с постусловием и цикл с параметром.

**Цикл с предусловием.** Если указанное условное выражение истинно, то выполняется тело цикла и управление передаётся снова на проверку условия. Как только условие окажется ложным, выполнение цикла завершится:

```
while (условие) {
 ... ;
 ... ;
}
```

На схемах алгоритмов цикл с предусловием обозначается, как показано на рис. 2.9 а, б.

**Пример 2.10.** Вывести числа от 5 до 1:

```
int i = 5;
while (i > 0) {
 cout << i << ", ";
 i--; // т.е. i = i - 1;
}
```

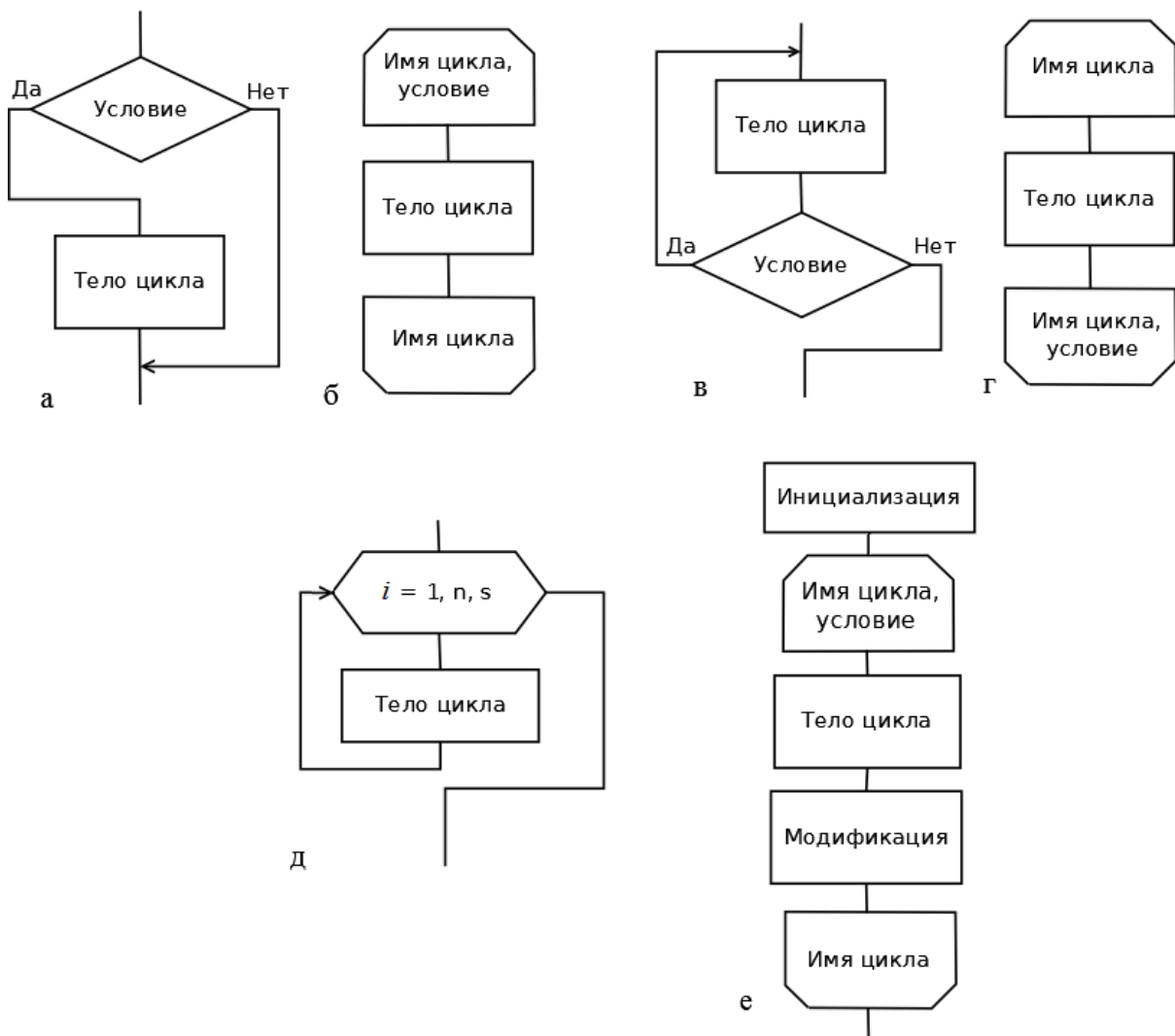
**Цикл с постусловием.** Сначала выполняется тело цикла, затем проверяется условие. Если оно истинно, то процесс повторяется:

```
do {
 ... ;
 ... ;
} while (условие);
```

Два варианта обозначения цикла с постусловием показано на рис. 2.9 в, г.

**Пример 2.11.** Вывести числа от 5 до 1:

```
int i = 5;
do {
 cout << i << ", ";
```



**Рис. 2.9.** Обозначения циклов на схемах алгоритмов: а, б) цикл с предусловием; в, г) цикл с постусловием; д, е) цикл с параметром

```
i--; // т.е. i = i - 1;
} while (i > 0);
```

**Сумма цифр числа.** В листинге 2.8 приведена программа, которая вычисляет сумму цифр заданного числа. Для нахождения цифр в десятичной записи числа используются последовательные деления на 10: остаток даёт очередную цифру, а результат запоминается для следующего деления. Так продолжается до тех пор, пока результат деления не станет нулевым. Алгоритм:

1. Вводим целое число  $x$ .
2. Инициализируем переменную для накопления суммы:  $sum = 0$ .

3. Находим остаток от деления  $x$  на 10;  
накапливаем сумму  $sum = sum + \text{остаток}$ .
4. Находим результат целочисленного деления  $x = x / 10$ .
5. Если  $x$  не равно 0 возвращаемся к п.3.
6. Выводим значение  $sum$ .

На рис. 2.10 показан результат работы программы.

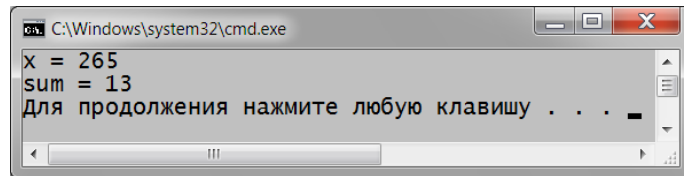


Рис. 2.10. Результат работы программы из листинга 2.8

#### Листинг 2.8: Вычисление суммы цифр числа

```
1 #include <iostream>
2 using namespace std;
3 int main(){
4 int x;
5 cout << "x = ";
6 cin >> x;
7 int sum = 0;
8 do{
9 sum += x % 10; // т.е. sum = sum + (x % 10);
10 x /= 10; // т.е. x = x / 10;
11 } while(x != 0);
12 cout << "sum = " << sum << endl;
13 system("pause");
14 return 0;
15 }
```

**Цикл с параметром.** Сначала выполняется инициализация, затем, как и в цикле с предусловием, осуществляется проверка истинности условного выражения. Если оно истинно, то выполняется тело цикла, после чего выпол-

няется модификация параметра(ов) цикла и управление снова передаётся на проверку условия:

```
for (инициализация; условие; модификация) {
 ... ;
 ... ;
}
```

На старых схемах алгоритмов цикл с параметром иногда обозначают, как показано на рис. 2.9 д (задано начальное значение параметра, конечное значение  $n$  и шаг изменения  $s$ ). Новое обозначение по ГОСТ 19.701-90 приведено на рис. 2.9 е. Если тело цикла состоит из одного оператора, то фигурные скобки можно опустить.

**Пример 2.12.** Вывести числа от 5 до 1:

```
int i;
for(i=5; i>0; i--) {
 cout << i << ", ";
}
```

**Пример 2.13.** В блоках оператора `for` можно записывать несколько операторов через запятую. Кроме того, в языке C++ (в отличие от C) можно объявить параметр(ы) цикла внутри оператора `for`:

```
for (int i=0, j=5; i < 3; i++, j--){
 printf("%d, %d, ", i, j);
}
```

Будет напечатано: 0, 5, 1, 4, 2, 3, Здесь блоки инициализации и модификации состоят из двух операторов.

**Пример 2.14.** Чтобы в каждой итерации цикла вывод на экран начинался с новой строки, надо в блоке модификации перейти на новую строку, например:

```
for(int i=10, k=5; i>0; i--, k++, cout<<endl)
 cout << i << ", " << k ;
```

**Сумма чётных чисел в заданном диапазоне.** В листинге 2.9 приведена программа, которая вычисляет сумму всех чётных чисел в заданном диапазоне. На рис. 2.11 показан результат работы программы.

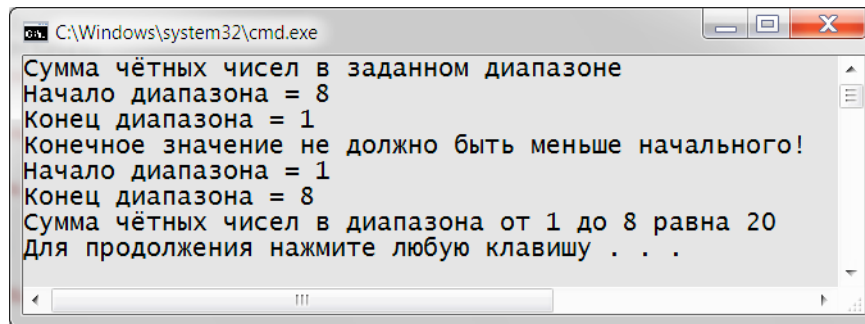


Рис. 2.11. Экран программы из листинга 2.9

#### Листинг 2.9: Сумма чётных чисел в заданном диапазоне

```
1 #include <iostream>
2 #include <windows.h>
3 using namespace std;
4 int main(){
5 int a, b;
6 long int sum;
7 bool flag;
8 SetConsoleOutputCP(1251);
9 cout << "Сумма чётных чисел в заданном диапазоне\n";
10 do{ // Цикл, пока не будет введён правильный диапазон
11 cout << "Начало диапазона = "; cin >> a;
12 cout << "Конец диапазона = "; cin >> b;
13 flag = a > b; // true, если диапазон задан неверно
14 if (flag)
15 cout << "Конечное значение не должно быть "
16 << "меньше начального!\n";
17 } while(flag);
18
19 sum = 0; // Инициализация переменной, в которой накапливаем
 сумму
```

```
20 for(int i = a; i <= b; i++) // Перебираем все числа в
 диапазоне
21 if(i % 2 == 0) // Если остаток от деления на 2 равен 0
22 sum += i; // Накапливаем сумму
23 cout << "Сумма чётных чисел в диапазона от "
24 << a << " до " << b << " равна " << sum << endl;
25 system("pause");
26 return 0;
27 }
```

**Вложенные циклы.** В листинге 2.10 приведена программа, выводящая на экран все трёхзначные числа, сумма цифр которых равна заданному числу. На рис. 2.12 показан результат её работы.

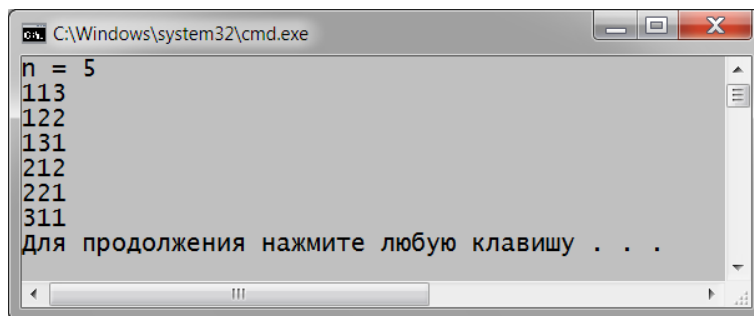


Рис. 2.12. Экран программы из листинга 2.10

#### Листинг 2.10: Трёхзначные числа с заданной суммой цифр

```
1 #include <iostream>
2 using namespace std;
3 int main(){
4 int n;
5 cout << "n = ";
6 cin >> n;
7 for (int i=1; i<10; i++)
8 for (int j=1; j<10; j++)
9 for (int k=1; k<10; k++)
10 if (i+j+k == n)
```

```
11 cout << i << j << k << endl;
12
13 system("pause");
14 return 0;
15 }
```

**Оператор передачи управления.** Имеется (но не рекомендуется для использования, т. к. нарушает принципы структурного программирования) оператор `goto`, который позволяет перейти в указанное место программы:

```
goto метка;
...
метка: оператор;
...
```

Имя метки должно удовлетворять обычным требованиям к идентификаторам.

**Оператор `break`.** Оператор `break` (прервать) используется внутри операторов цикла или `switch` для обеспечения перехода в точку программы, находящуюся непосредственно за оператором, внутри которого находится `break`.

**Пример 2.15.** Рассмотрим следующий фрагмент:

```
const int n = 8;
int a[n] = {5, 1, 2, 14, 25, 6, 0, 9};
int i, j=-1, k;
cout << "Введите число: ";
cin >> k;
for(i = 0; i < n; i++){
 if(a[i] == k){ // нужное значение
 j = i; // номер найденного элемента
 cout << k << " на " << j << " месте\n";
 break; // выйти из цикла
 }
}
```

```
if(j < 0) // элемент не найден
 cout << k << " не найден\n";
```

Здесь в цикле `for` производится поиск элемента в массиве. Как только найден элемент с заданным значением, поиск прекращается.

**Оператор `continue`.** Оператор перехода к следующей итерации цикла `continue` (продолжить) пропускает все операторы, оставшиеся до конца тела цикла, и передаёт управление на начало следующей итерации.

**Пример 2.16.** Рассмотрим, что будет выведено в результате выполнения следующего фрагмента программы:

```
for (int i=0, j=3; i < j; j--) {
 cout << i++;
 if (i < j)
 continue;
 cout << '.' << j;
}
```

**Таблица 2.4.** Трассировка цикла из примера 2.16.

| Итерация | i | j | for(..., i<j, ...) | cout | i | if(i<j) | cout |
|----------|---|---|--------------------|------|---|---------|------|
| 1        | 0 | 3 | true               | 0    | 1 | true    |      |
| 2        | 1 | 2 | true               | 1    | 2 | false   | .2   |
| 3        |   | 1 | false              |      |   |         |      |

Распишем по шагам процесс выполнения данного цикла:

- Выполняется блок инициализации оператора `for`: `i = 0; j = 3`.
- Проверяется условие `i < j` цикла `for`. Результат `true`, поэтому выполняется тело цикла.
- На экран выводится значение `i` — число 0, затем `i` увеличивается на 1 (постинкремент), т. е. теперь `i = 1`.
- Проверяется условие `i < j` в операторе `if`, получается `true` (т. к. `1 < 3`), поэтому выполняется оператор `continue`, символ «точка» не выводится, управление передаётся на блок модификации оператора `for`.

— Значение переменной `j` уменьшается на 1 и становится равным 2. Проверяется условие `i < j` в операторе `for`, получается `true` (т. к. `1 < 2`), поэтому снова выполняется тело цикла.

— На экран выводится значение `i` — число 1, затем `i` увеличивается `i` на 1, т. е. теперь `i = 2`.

— Проверяется условие `i < j` в операторе `if`, получается `false` (т. к. 2 не меньше 2), поэтому оператор `continue` не выполняется. Выводится символ «точка» и значение `j` — число 2. управление передаётся на блок модификации оператора `for`.

— Значение переменной `j` уменьшается на 1 и становится равным 1. Проверяется условие `i < j` в операторе `for`, получается `false` (т. к. 3 не меньше 1), поэтому выполнение цикла завершается.

Трассировка цикла показана в табл. 2.4. Таким образом, в результате выполнения данного фрагмента будет выведена строка

01.2

### Правила хорошего стиля программирования:

1. Не использовать `goto`.
2. Соблюдать отступы.
3. Не смешивать стили форматирования операторных скобок `{` и `}`.
4. Не смешивать методы ввода-вывода `C` и `C++`.
5. Писать содержательные комментарии.

**Табулирование функции.** В листинге 2.11 приведена программа, которая выводит на экран таблицу значений заданной функции (рис. 2.13). Позже (см. с. 122) мы переделаем эту программу для использования с несколькими различными функциями.

#### Листинг 2.11: Табулирование функции

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <windows.h>
4 int main(){
```

```

5 double a, b, y, x, d;
6 long i, n;
7 SetConsoleOutputCP(1251);
8 printf("\n ===== Табулирование функции =====\n");
9 printf("Начало = "); scanf("%lg", &a);
10 printf("Конец = "); scanf("%lg", &b);
11 printf("Кол-во строк = "); scanf("%li", &n);
12 d = (b - a) / (n - 1); // Шаг изменения аргумента
13 printf("\n-----\n");
14 printf(" x\t\tty\n"); // \t - символ табуляции
15 printf("-----\n");
16 x = a;
17 for (i=0; i<n; i++){
18 y = (2.0 - 2.0/3 + exp(log(x/2.7)/3.0)
19 - sqrt(exp(-3.0*x))) * fabs(sin(5.0*x));
20 printf("%5.2lg \t %8.3lg\n", x, y);
21 x += d;
22 }
23 system("pause");
24 return 0;
25 }

```

```

D:\DevCpp\s2.exe
===== Табулирование функции =====
Начало = 0
Конец = 1
Кол-во строк = 6

 x y

 0 0
 0.2 0.852
 0.4 1.19
 0.6 0.216
 0.8 1.29
 1 1.75
Для продолжения нажмите любую клавишу . . .

```

Рис. 2.13. Результат работы программы из листинга 2.11

**Накопление сумм и произведений.** В листинге 2.12 показан пример программы, которая вычисляет произведение

$$\prod_{k=1}^n \frac{a^k}{(a+1)^{2k-1}} = \frac{a}{a+1} \cdot \frac{a^2}{(a+1)^3} \cdots \frac{a^n}{(a+1)^{2n-1}} = a^{n(n+1)/2} \cdot (a+1)^{-(n-1)(n+1)-1}.$$

Здесь использована рекуррентная формула для вычисления очередного значения сомножителя:

$$x_{i+1} = x_i \cdot \frac{a}{(a+1)^2}.$$

На рис. 2.14 показан результат работы программы.

#### Листинг 2.12: Накопление произведения

```

1 #include <iostream>
2 #include <cmath>
3 #include <windows.h>
4
5 using namespace std;
6
7 int main () {
8 double a, x, y, s, s0, d;
9 long n, i;
10 SetConsoleOutputCP(1251);
11 cout << "Введите параметр a = ";
12 cin >> a;
13 cout << "Введите число сомножителей n = ";
14 cin >> n;
15 s = 1;
16 x = a/(a+1); // первый сомножитель
17 y = x/(a+1); // коэффициент для следующих сомножителей
18 for(i = 0; i<n; i++){
19 cout << i+1 << "-й сомножитель = " << x << endl;
20 s *= x; // накапливаем произведение
21 x *= y; // следующий сомножитель
22 }
23 cout << "Произведение = " << s << endl;

```

```

24 s0 = pow(a, n*(n+1.0)/2.0) * pow(a+1, -(n-1.0)*(n+1.0)-1);
25 cout << "Контрольное значение = " << s0 << endl;
26 d = s - s0;
27 if(fabs(d) < 1e-4)
28 cout << "Ответ правильный, ошибка = " << d << endl;
29 else
30 cout << "Ответ неверный, ошибка = " << d << endl;
31
32 system("pause");
33 return 0;
34 }

```

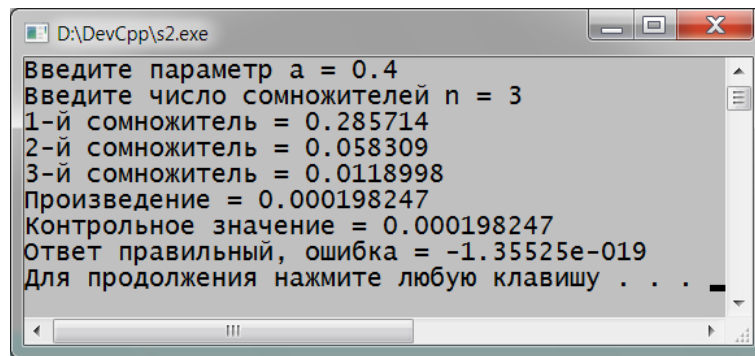


Рис. 2.14. Экран программы из листинга 2.12

**Суммирование бесконечных рядов.** Найдём сумму бесконечного ряда

$$x \left( \frac{1}{1!} - \frac{1}{2!} \right) - x^2 \left( \frac{1}{2!} - \frac{1}{4!} \right) + x^3 \left( \frac{1}{3!} - \frac{1}{6!} \right) - \dots \pm x^i \left( \frac{1}{i!} - \frac{1}{(2i)!} \right) \mp \dots$$

Из курса математического анализа известно, что данный ряд является разложением функции

$$\cos \sqrt{x} - e^{-x}.$$

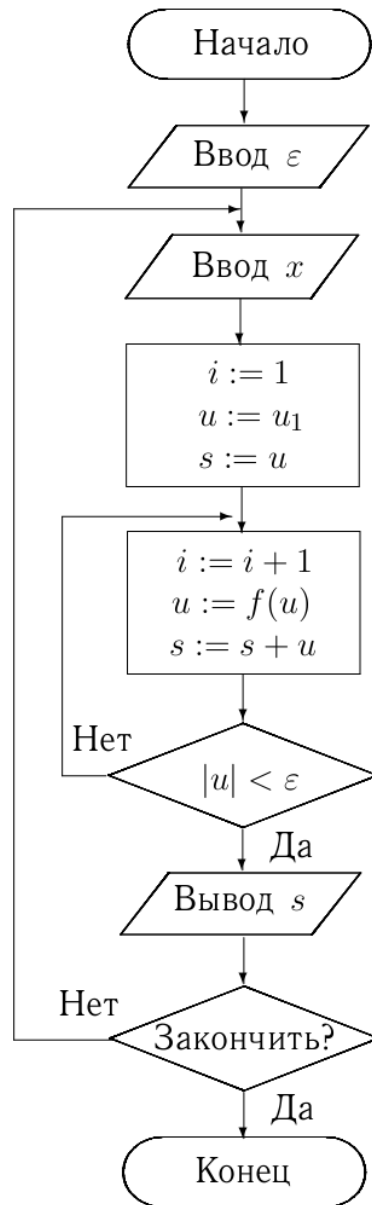
При вычислении суммы бесконечного ряда на компьютере необходимо задать точность  $\varepsilon$  — некоторое малое число ( $0 < \varepsilon \ll 1$ ). Когда очередное слагаемое станет по модулю меньше  $\varepsilon$ , то вычисления можно прекратить.

Словесное описание алгоритма суммирования бесконечного ряда:

- 1) ввести точность  $\varepsilon$ ;
- 2) ввести значение переменной  $x$ ;



```
printf("\n-----\n");
printf("Введите x = ");
scanf("%g", &x);
i = 1; a = 1.0; b = 0.5; c = x;
u = c*(a-b);
s = u; // сначала сумма равна первому слагаемому
```



**Рис. 2.15.** Схема алгоритма суммирования бесконечного ряда:  $\varepsilon$  — точность;  $x$  — значение аргумента;  $i$  — счётчик просуммированных членов ряда;  $u_1$  — первый член ряда;  $f(u)$  — рекуррентная формула для вычисления очередного члена ряда;  $u$  — очередной член ряда;  $s$  — сумма ряда

```
do{ // цикл для слагаемых ряда
 i++; // счётчик слагаемых
 a = a / i;
 b = b / (2*i-1) / (2*i);
 c = -c * x;
 u = c * (a-b); // очередное слагаемое
 s += u; // накапливаем сумму
}while((fabs(u) > eps) && (i < 10000));
f = cos(sqrt(x)) - exp(-x); // контрольная функция
e = s - f; // ошибка
printf("Сумма ряда при x=%g равна %lg\n", x, s);
printf("истинное значение = %lg, ошибка = %lg\n", f, e);
printf("просуммировано %li слагаемых\n", i);
printf("Будем продолжать (y/n)? ");
}while(getch() == 'y');
return 0;
}
```

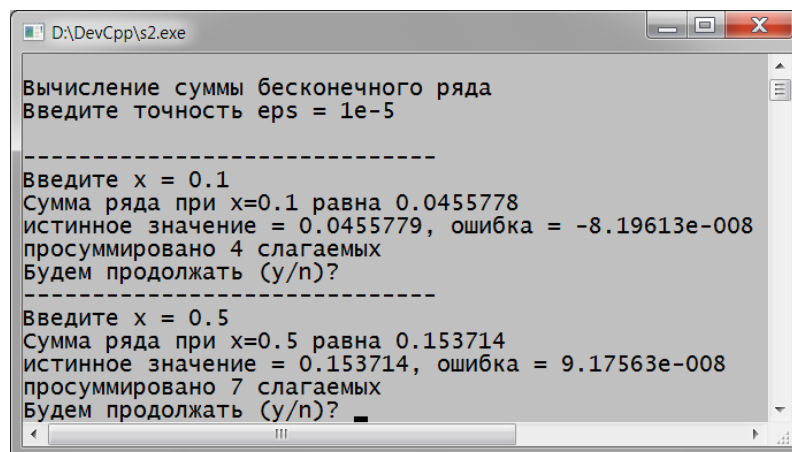


Рис. 2.16. Результат работы программы из листинга 2.13

## 2.6. Контрольные вопросы

1. Пояснить смысл следующих понятий, привести примеры: а) условный оператор; б) цикл с предусловием; в) цикл с постусловием; г) цикл с параметром; д) условная операция.
2. Почему не рекомендуется использовать оператор перехода `goto`?
3. Чем различаются операторы ветвления `if` и `switch`?
4. Какая строка будет выведена в результате выполнения следующего фрагмента программы:

```
int a = 5, b = 12;
int x = a && b;
if (x)
 printf("true");
else
 printf("false");
```

5. Какое число будет выведено в результате выполнения следующего фрагмента программы:

а)

```
int a = 9, b = 2;
int x = a <= b ? ++a : --b;
printf("%d", x);
```

б)

```
int a = 9, b = 2;
int x = a <= b ? a++ : b--;
printf("%d", x);
```

в)

```
int x = 5, y = 10;
y -= x--;
if (x < y) cout << x;
else cout << y;
```

г)

```
int x = 5, y = 10;
y -= ++x;
switch (x) {
 case 5: cout << 'a';
 case 6: cout << 'b';
 default: cout << 'c';
}
```

6. Что будет выведено в результате выполнения следующего фрагмента программы:

а)

```
int x = 5;
while (x < 8) {
 cout << x++;
}
```

б)

```
int x = 5;
do {
 cout << ++x;
} while (x < 8);
```

в)

```
for(int i=0, j=3;
 i<j; ++i, j--){
 cout << i << j;
}
```

г)

```
for(int i=0, j=3;
 i<j;) {
 cout << ++i << j--;
}
```

д)

```
for(int i=0, j=3;
 i<j; i--){
 cout << ++i << j--;
}
```

е)

```
for(int i=0, j=4;
 j; ++i, --j){
 cout << i << j--;
}
```

ж)

```
for(int i=0, j=3;
 i<j; j--) {
 cout << i++;
 if (i < j) continue;
 cout << '.' << j;
}
```

7. Запишите уравнение прямой на плоскости. Какой смысл имеют входящие в него параметры?
8. Как аналитически задать множество точек, лежащих ниже (выше) прямой на плоскости?
9. Запишите уравнение окружности с заданными координатами центра и радиусом.
10. Как аналитически задать множество точек, лежащих внутри круга с заданными координатами центра и радиусом? Вне его?
11. Сформулируйте правило окончания вычислений при суммировании бесконечных рядов.

12. Какой ряд называется знакопостоянным? Знакопередающим?
13. Можно ли утверждать, что отбросив при суммировании все члены ряда, начиная с первого, который по модулю меньше  $\varepsilon$ , мы получим ошибку вычисления суммы ряда  $E \leq \varepsilon$ ?
14. В листинге 2.13 через переменные  $a$  и  $b$  обозначены величины  $1/i!$  и  $1/(2i)!$  соответственно. Что изменится, если в переменных  $a$  и  $b$  хранить значения знаменателей  $i!$  и  $(2i)!$ , а очередной член ряда вычислять по формуле  $d = c \cdot (1/a - 1/b)$ ?
15. Напишите формулу Тейлора разложения функции в ряд вблизи произвольной точки  $x = a$ .
16. Можете ли вы с помощью своей программы определить, является ли сходимость ряда равномерной?
17. Попробуйте сформулировать правило окончания цикла при вычислении бесконечного произведения  $\prod_{k=1}^{\infty} u_k(x)$ .
18. Как можно свести задачу вычисления бесконечного произведения к задаче вычисления бесконечной суммы?

## 3. Массивы и указатели

► *Массив* — это упорядоченный набор однотипных элементов, доступ к которым осуществляется по их номеру (*индексу*).

### 3.1. Одномерные статические массивы

При описании переменной-массива в квадратных скобках указывается количество элементов. Стандарты C89 и C90 языка C требуют, чтобы количество элементов задавалось в виде константы или константного выражения (значение которого можно вычислить на этапе компиляции). Пример описания одномерного массива из десяти целых элементов:

```
int a[10];
```

или

```
const int n = 10;
int a[n];
```

или

```
#define N 10
int a[N];
```

Рекомендуется использовать второй вариант.

Элементы массивов нумеруются, начиная с 0; поэтому в приведённых выше примерах элементы будут иметь индексы от 0 до 9.

Элементы одномерного массива располагаются в памяти последовательно, друг за другом, в соответствии с их индексами, без пропусков.

Новый стандарт<sup>1</sup> C99 разрешает в качестве размера *локального* массива (т.е. объявленного внутри некоторого блока, ограниченного фигурными скобками, например, внутри какой-либо функции, в том числе функции `main`) указывать переменную целого типа, значение которой уже известно:

---

<sup>1</sup>Компилятор Microsoft Visual C++ пока не поддерживает стандарт C99. Массивы с переменной длиной (Variable Length Array, VLA) можно использовать с компиляторами GNU Compiler Collection (GCC) или MinGW, например, в Dev C++.

```
long n;
cout >> "Введите число элементов = ";
cin >> n;
double arr[n];
```

Разумеется, если значение переменной `n` после этого изменится, то размер массива `arr` всё равно останется прежним.

В описании массива можно использовать инициализатор:

```
int a[10] = { 2, 6, 3, 1, 4, 2, 1 } ;
```

Здесь первым семи элементам массива (с индексами от 0 до 6) присвоены начальные значения, остальные три элемента (с индексами от 7 до 9) по умолчанию инициализируются нулями. При использовании инициализатора размер массива в квадратных скобках можно опустить (оставив сами скобки) — число элементов массива будет совпадать с числом значений, указанных в фигурных скобках. Также допускается не указывать размер переменной-массива, если она является формальным параметром функции (см. раздел 4.1).

Для доступа к отдельным элементам массива в квадратных скобках указывают нужный индекс, например:

```
a[7] = 10;
a[8] = a[0] + a[4] * a[7];
```

При выполнении программы не производится никаких автоматических проверок выхода индекса за допустимые пределы, что может привести к ошибкам при небрежном программировании.

**Пример 3.1. Сумма элементов массива.** В листинге 3.1 приведён пример программы, которая запрашивает с клавиатуры фактический размер массива, значения его элементов и вычисляет их сумму и среднее значение; на рис. 3.1 показан результат работы программы.

#### Листинг 3.1: Вычисление суммы элементов массива

```
1 #include <iostream>
2 #include <windows.h>
3 using namespace std;
```

```
4
5 int main(){
6 const int m=10; // размер массива
7 double a[m], x; // массив и среднее значение элементов
8 int n; // число элементов
9 SetConsoleOutputCP(1251);
10
11 do{
12 cout << "Введите число элементов "
13 "(не больше " << m << ") = ";
14 cin >> n;
15 } while (n > m); // пока не будет введено правильное значение
16
17 double sum = 0.0; // здесь будем накапливать сумму элементов
18 for (int i=0; i<n; i++) {
19 cout << " введите элемент [" << i << "] = ";
20 cin >> a[i]; // вводим i-й элемент
21 sum += a[i]; // сумма элементов
22 }
23 cout << "Сумма введенных элементов = " << sum << endl;
24 x = sum / n; // среднее арифметическое
25 cout << "Среднее значение = " << x << endl;
26 return 0;
27 }
```

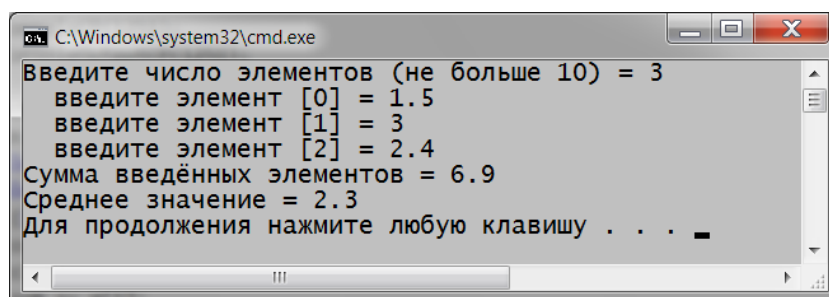


Рис. 3.1. Экран программы из листинга 3.1

### Пример 3.2. Максимальный и минимальный элементы массива.

В листинге 3.2 приведён пример программы, в которой находятся максимальный и минимальный элементы массива. Сначала переменная `max` инициализируется наименьшим, а переменная `min` — наибольшим возможным числом для типа `double`. Константа `DBL_MAX` определена в заголовочном файле `<climits>` (`limits.h`). Затем мы перебираем все элементы массива в цикле, и, если очередной элемент больше текущего значения максимума (или меньше текущего значения минимума), то запоминаем новое значение максимума (минимума), а также номер позиции `imax` (или `imin`).

Листинг 3.2: Максимальный и минимальный элементы массива

```

1 #include <iostream>
2 #include <cfloat>
3 using namespace std;
4
5 int main(){
6 const int m=10; // размер массива
7 double a[m]; // массив
8 int n; // число элементов
9 setlocale(LC_ALL, "rus");
10 cout << "Введите число элементов "
11 "(не больше " << m << ") = ";
12 cin >> n;
13 if (n > m){
14 cout << "Введённое число слишком велико,\n"
15 "будет использовано значение " << m << endl;
16 n = m;
17 }
18 double max = -DBL_MAX, min = DBL_MAX;
19 int imax = -1, imin = -1;
20 for (int i=0; i<n; i++) {
21 cout << " введите элемент [" << i << "] = ";
22 cin >> a[i]; // вводим i-й элемент

```

```
23 if (a[i] > max) {
24 max = a[i]; imax = i;
25 }
26 if (a[i] < min) {
27 min = a[i]; imin = i;
28 }
29 }
30 cout << "Максимальный элемент " << max
31 << " с номером " << imax << endl;
32 cout << "Минимальный элемент " << min
33 << " с номером " << imin << endl;
34 system("pause");
35 return 0;
36 }
```

## 3.2. Указатели. Динамические массивы

Для создания различных структур данных произвольного размера (в том числе, массивов) используют специальную *динамическую память*, которая предоставляется программам операционной системой. Переменные, значения которых хранятся в динамической памяти, называются *динамическими*. Они не имеют имени, доступ к ним производится с помощью *указателей*.

**Указатели.** *Указатель* — это переменная, которая хранит адрес некоторой области памяти. При описании указателя задаётся тип значения, на которое он должен указывать, и ставится звёздочка. Например, описание указателя p1 на целое значение:

```
int *p1;
```

описание указателя p2 на вещественное число двойной точности:

```
double *p2;
```

В одной строке можно описать и обычную переменную, и указатель, например:

```
int *p1, *p3, i, j;
```

т. е. звёздочка относится к имени переменной, а не к названию типа.

Независимо от типа указателя, он занимает в памяти фиксированный размер (4 байта).

Описав переменную-указатель, мы можем присвоить ей адрес другой переменной, используя оператор взятия адреса `&`. Например:

```
p1 = &i;
```

Указателю также можно присвоить адрес, хранящийся в другом указателе, если их типы совместимы:

```
p3 = p1;
```

Указателю можно присвоить произвольный адрес в памяти, например:

```
char *p = (char*)0xB8000000;
```

Считается, что никакая переменная не может иметь нулевой адрес, поэтому указатель, имеющий значение 0, никуда не указывает:

```
p = 0; //или p = NULL;
```

Чтобы получить доступ к области памяти, адрес которой хранит указатель, используется операция *разыменования* (*разадресации*) — звёздочка перед именем переменной-указателя:

```
int i, j;
int *p1(&i); // p1 указывает на i
*p1 = 5; // эквивалентно i = 5;
j = *p1; // эквивалентно j = i;
```

Можно описать указатель без определённого типа (если тип значения заранее не известен или указатель требуется использовать для работы со значениями разного типа):

```
void *p;
```

но тогда каждый раз при разыменовании придётся использовать операцию приведения типа. Например:

```
(int*)p = &i; j = (int*)p;
```

Пример использования модификатора `const` при описании указателей:

```
const int k = 5; // целая константа
const int *p1; // указатель на целую константу
int * const p2 = &i; // константный указатель на целую
 переменную
const int * const p3 = &k; // константный указатель на целую
 константу
```

К указателям можно применять арифметические операции сложения, вычитания, инкремента и декремента. Если, например, описать указатель на длинное целое и присвоить ему адрес какой-либо переменной:

```
long int p = &i;
```

то после выполнения операции

```
p += 2;
```

адрес, хранящийся в указателе, увеличится на  $4 \cdot 2 = 8$  (т. к. одна переменная типа `long int` занимает в памяти 4 байта).

**Указатели и одномерные массивы.** Имя массива рассматривается компилятором как константный указатель на нулевой элемент массива. Поэтому после описания, например, массива `int a[10]` запись `*(a+i)` эквивалентна обращению к *i*-му элементу массива, т. е. `a[i]`.

**Пример 3.3.** Копирование элементов из одного массива в другой.

```
int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, b[10];

// Первый способ:
for(int i = 0; i < 10; i++)
 b[i] = a[i];

// Второй способ:
for(int i = 0; i < 10; i++)
 *(b + i) = *(a + i);

// Третий способ:
```

```
int *p1, *p2;
for(int i = 0, p1 = a, p2 = b; i < 10; i++)
 *(p2++) = *(p1++);
```

Присваивание массивов не допускается. Если мы хотим скопировать элементы одного массива в другой, то должны сами сделать это в цикле.

**Ссылки.** *Ссылка* — это синоним имени, которое указывается при её инициализации. Другими словами, ссылка — это указатель, который не требуется разыменовывать. Например:

```
int k; // целая переменная
int & r = k; // ссылка на переменную k
int i = r + 5; // т.е. i = k + 5;
r = 10; // т.е. k = 10;
const char & cr = '\n'; // ссылка на константу-символ
```

Обычно ссылки используются в качестве выходного параметра функции (см. раздел 4.1, с. 117).

**Выделение и освобождение динамической памяти.** В приведённых выше примерах переменные-указатели хранили адреса других именованных переменных. Но обычно указатели используются для работы с безымянными данными, которые хранятся в динамической памяти.

Для выделения в динамической памяти области для хранения безымянной переменной используется операция `new` (в C++) или функция `malloc` (в языке C).

Например, для создания динамической переменной целого типа:

```
int *p1 = new int;
```

для создания динамической переменной типа `double`:

```
double *p2 = new double;
```

Здесь совмещено описание переменной-указателя и выделение памяти; эти два действия можно разделить:

```
double *p2;
```

```
p2 = new double;
```

Можно использовать инициализатор в круглых скобках, например:

```
double *p2 = new double(3.5);
```

В результате выполнения операции `new` в динамической памяти резервируется область нужного размера (в зависимости от указанного типа) и адрес начала этой области помещается в указатель. Если используется инициализатор, то заданное в скобках значение помещается в зарезервированную область памяти и может быть прочитано при разыменовании указателя.

Пример выделения динамической памяти с помощью функции `malloc`:

```
int *p3;
p3 = (int*)malloc(sizeof(int));
double *p4;
p4 = (double*)malloc(sizeof(double));
```

В качестве параметра указывается размер области памяти. Поскольку функция `malloc()` возвращает значение типа `void*` (указатель без типа), требуется явное приведение типов.

Если при попытке выделения динамической области памяти выясняется, что свободная память отсутствует, то в старых версиях компиляторов языка С указатель принимает нулевое значение и память не выделяется, а в новых версиях генерируется так называемое *исключение*, которое может быть обработано в программе (работа исключениями рассматривается во второй части настоящего пособия).

Когда динамическая переменная становится ненужной, занимаемую ею память надо освободить. Переменные, созданные с помощью операции `new`, уничтожаются операцией `delete`. Если использовалась функция `malloc()`, то для уничтожения динамической переменной применяют функцию `free()`:

```
delete p1;
delete p2;
free(p3);
free(p4);
```

Если указатель нулевой (никуда не указывает), то операция `delete` и функция `free` ничего не делают.

После освобождения памяти указатель по-прежнему хранит старый адрес, поэтому иногда рекомендуют обнулить указатель, чтобы случайно не обратиться к участку памяти, который уже не должен использоваться:

```
p1 = 0; p2 = 0; p3 = 0; p4 = 0;
```

Если программист забыл освободить динамическую память, то происходит, так называемая, *утечка памяти* — занятая область динамической памяти больше не используется программой, но не может быть выделена для других динамических переменных (в том числе, при работе других программ), пока данная программа (в которой происходит утечка) не будет завершена. При завершении программы все созданные в ней динамические переменные уничтожаются автоматически и память освобождается.

**Одномерные динамические массивы.** Обычно в динамической памяти хранят не одиночные переменные, а структуры данных большого размера, в том числе, массивы.

Пример создания одномерного динамического массива из  $n$  элементов типа `double`:

```
double *p1 = new double[n];
```

Здесь описана переменная-указатель на массив вещественных чисел; в динамической памяти зарезервирована область для хранения  $n$  элементов массива; адрес начала этой области помещён в переменную-указатель.

То же самое, но с использованием функции `malloc()`:

```
double *p2;
p2 = (double*)malloc(n*sizeof(double));
```

Инициализатор в фигурных скобках при определении динамического массива использовать нельзя. Кроме того, в отличие от статических массивов, элементы которых автоматически инициализируются нулевыми значениями, элементы динамических массивов имеют произвольные начальные значения.

При освобождении памяти, отведённой под массив, в операции `delete` требуется указывать квадратные скобки:

```
delete [] p1;
```

Если программист забудет указать квадратные скобки, то освободится область динамической памяти, занятая только нулевым элементом массива; а место, занимаемое остальными элементами, будет недоступно для всех программ, пока наша программа не завершится (утечка памяти).

Функция `free` работает с указателями на массив, как и с любыми другими:

```
free(p2);
```

Варианты обращения к  $i$ -му элементу одномерного динамического массива, на начало которого указывает переменная-указатель `p`, ничем не отличаются от использования указателей со статическими массивами: `p[i]` или `*(p+i)`.

Перепишем программу из листинга 3.1, используя динамические массивы (листинг 3.3).

### Листинг 3.3: Вычисление суммы элементов динамического массива

```
1 #include <iostream>
2 #include <windows.h>
3 using namespace std;
4
5 int main(){
6 // Ограничение на число элементов:
7 const unsigned long int m = 100000000;
8
9 double x; // среднее значение
10 unsigned long int n; // число элементов
11 SetConsoleOutputCP(1251);
12
13 do{
14 cout << "Введите число элементов "
```

```
15 "(не больше " << m << ") = ";
16 cin >> n;
17 } while(n > m); // пока не будет введено правильное значение
18
19 double *a = new double[n]; // создали динамический массив
20 double sum = 0.0; // здесь будем накапливать сумму элементов
21 for (unsigned long int i=0; i<n; i++) {
22 cout << " введите элемент [" << i << "] = ";
23 cin >> a[i]; // вводим i-й элемент
24 sum += a[i]; // сумма элементов
25 }
26 cout << "Сумма введённых элементов = " << sum << endl;
27 x = sum / n; // среднее арифметическое
28 cout << "Среднее значение = " << x << endl;
29 delete[] a;
30 return 0;
31 }
```

Как видим, текст программы практически не изменился: мы исправили только описание массива (добавили операцию `new`), переместив её в то место программы, где уже известно число элементов; также добавилась операция освобождения памяти. В результате мы используем память экономно (в отличие от варианта со статическим массивом, где место для него резервировалось с запасом); кроме того, теперь мы можем работать с массивами очень большой длины (в пределах возможностей динамической памяти).

**Модификация массивов.** При использовании массивов программист имеет возможность непосредственно обращаться для чтения или записи к любому элементу. Но если, например, требуется вставить новый элемент в начало или середину массива, сохранив все уже имеющиеся, или нужно удалить некоторый элемент из начала или середины массива, то приходится сдвигать

все элементы, которые следуют после вставляемого или удаляемого элемента, соответственно, вправо или влево.

**Пример 3.4.** Вставка нового элемента  $x$  в позицию  $k$  массива  $a[n]$  и сдвиг всех оставшихся элементов вправо (последний элемент теряется):

```
for(unsigned long i=n-2; i>k; i--)
 a[i+1] = a[i];
a[k] = x;
```

**Пример 3.5.** Удаление  $k$ -го элемента массива  $a[n]$  и сдвиг всех оставшихся элементов влево (последний элемент инициализируется нулём):

```
for(unsigned long i=k; i<n-1; i++)
 a[i] = a[i+1];
a[n-1] = 0;
```

### 3.3. Сортировка элементов массива

*Алгоритм сортировки* — это алгоритм упорядочения элементов в одномерном массиве (векторе, списке).

**Перестановка элементов.** В приведённых ниже алгоритмах сортировки используется операция перестановки элементов массива. В простейшем случае значения двух элементов  $a[i]$  и  $a[j]$  можно поменять местами с помощью служебной переменной:

```
double t = a[i];
a[i] = a[j];
a[j] = t;
```

Если элементы массива имеют целый тип, то можно обойтись и без дополнительных переменных, например, с помощью поразрядной операции «исключающее или»:

```
a[i] = a[i] ^ a[j];
a[j] = a[i] ^ a[j];
a[i] = a[i] ^ a[j];
```

Если операция перестановки используется в алгоритме часто, то можно определить функцию (см. раздел 4.1) или макрос:

```
#define SWAP(A, B) { A = A ^ B; B = A ^ B; A = A ^ B; }
```

Чтобы переставить вещественные числа, вместо поразрядных операций придётся использовать арифметические:

```
#define SWAP(A, B) { A = A + B; B = A - B; A = A - B; }
```

Чтобы избежать ошибок округления (особенно заметных, если переставляемые значения отличаются на несколько порядков), лучше всё-таки использовать временную переменную:

```
#define SWAP(A, B) { double T = A; A = B; B = T; }
```

**Гномья сортировка.** Данный алгоритм предложен в 2000 году и состоит (в отличие от других) всего из одного цикла. Это метод, которым садовый гном сортирует линию цветочных горшков. Он смотрит на два соседних горшка: если они в правильном порядке, он шагает на один горшок вперёд, иначе он меняет их местами и шагает на один горшок назад. Граничные условия: если нет предыдущего горшка — гном шагает вперёд; если нет следующего горшка — сортировка завершена.

Фрагмент программы для сортировки по возрастанию массива из  $n$  элементов методом гнома (нумерация элементов начинается с 0):

```
int i = 0;
while (i < n){
 if (i == 0 || a[i-1] <= a[i]) {
 i++;
 } else {
 double t = a[i];
 a[i] = a[i-1];
 a[--i] = t;
 }
}
```

Подумайте, как изменить текст программы, чтобы сортировка происходила по убыванию.

**Сортировка пузырьком.** Сортировка простыми обменами (пузырьком) — также один из самых простых, но не эффективных алгоритмов; часто упоминается в учебной литературе, но не рекомендуется профессиональным программистам:

```
for (i = n - 1; i > 0; i--) {
 for (j = 0; j < i; j++) {
 if (a[j] > a[j + 1])
 SWAP(a[j], a[j+1])
 }
}
```

**Сортировка перемешиванием (шейкерная сортировка)** представляет собой улучшенный вариант пузырьковой сортировки.

```
int last = n-1, left = 1, right = n-1;
do {
 for(j = right; j >= left; j--) {
 if(a[j-1] > a[j]) {
 SWAP(a[j-1], a[j])
 last = j;
 }
 }
 left = last + 1;
 for(j = left; j <= right; j++) {
 if(a[j-1] > a[j]) {
 SWAP(a[j-1], a[j])
 last = j;
 }
 }
 right = last-1;
} while(left < right);
```

**Сортировка вставками.** На каждом шаге алгоритма мы выбираем один из элементов входных данных и вставляем его на нужную позицию в уже отсортированном списке, до тех пор, пока набор входных данных не будет исчерпан.

```
for (i = 1; i < n; i++) {
 t = a[i];
 for (j = i - 1; j >= 0; j--) {
 if (a[j] < t) {
 break;
 }
 a[j+1] = a[j];
 }
 a[j+1] = t;
}
```

**Сортировка выбором.** Находим минимальное значение в текущем списке; производим обмен этого значения со значением на первой неотсортированной позиции; сортируем хвост списка, исключив из рассмотрения уже отсортированные элементы:

```
for(int i=0; i<n-1; ++i){
 int min = i;
 for(int j=i+1; j<n; ++j){
 if(a[j]< a[min])
 min = j;
 }
 if(min != i)
 SWAP(a[i], a[min]);
}
```

### 3.4. Двумерные массивы

**Статические двумерные массивы.** При описании статических двумерных массивов (матриц) в квадратных скобках указываются количество строк и столбцов, например:

```
double a[5][3]; // матрица из 5 строк и 3 столбцов
```

Согласно стандарту C90 обе размерности должны быть константами или константными выражениями, известными на момент компиляции. Стандарт C99 допускает использование переменных, если переменная-массив является локальной.

Аналогично описывается массив любой другой размерности, например, трёхмерный массив:

```
double a[2][5][3];
```

В памяти элементы массива располагаются построчно (при переходе к следующему элементу изменяется последний индекс).

Определение массива может сопровождаться его инициализацией; при этом каждую строку можно либо заключать в отдельные фигурные скобки (при этом можно не указывать самую первую размерность), либо задавать единый список всех элементов в том порядке, в каком они должны размещаться в памяти компьютера:

```
int a[][3] = {{1, 5, 3}, {-5, 4, 1}};
int a[2][3] = {1, 5, 3, -5, 4, 1};
```

Если значения последних элементов не указаны, то они будут инициализированы нулями.

Доступ к элементу двумерного массива осуществляется по двум индексам; варианты обращения к элементу, находящемуся в *i*-й строке и *j*-м столбце:

```
a[i][j]
*(a[i] + j)
*(*(a + i) + j)
```

Во втором случае мы сначала обращаемся к адресу начала  $i$ -й строки, а от него переходим к  $j$ -му элементу. В третьем случае мы берём адрес начала массива, затем переходим к началу  $i$ -й строки, после чего переходим к  $j$ -му элементу.

**Динамические двумерные массивы.** Для создания двумерного массива в динамической памяти при использовании операции `new` необходимо указать обе его размерности, причём переменной может быть только первая размерность. Данное ограничение связано с тем, что компилятору необходимо знать все размерности многомерного массива, кроме первой, чтобы генерировать код для доступа к нужным элементам. Ниже указаны фрагмент кода, иллюстрирующий работу с динамическим двумерным массивом:

```
int n;
const int m = 10;
cout << "Введите количество строк матрицы: ";
cin >> n;
double (*p)[m] = new double[n][m];
for(int i=0; i<n; i++){
 for(int j=0; j<m; j++){
 cout << "arr[" << i << "][" << j << "] = ";
 cin >> p[i][j];
 }
}
```

Здесь мы определили указатель `p` на массив из `m` элементов типа `double` (если не поставить скобки, то получится массив указателей) и выделили динамическую память под `n` элементов типа «массив из `m` элементов типа `double`». После этого в цикле организовали ввод элементов массива с клавиатуры.

Напомним, что элементы динамических массивов не инициализируются компилятором. Для освобождения динамической памяти, занятой массивами с любым количеством измерений, используется операция

```
delete [] p;
```

Чтобы иметь возможность работать с динамическими массивами, имеющими произвольное количество строк и столбцов, обычно организуют в динамической памяти иерархическую структуру, состоящую только из одномерных массивов (рис. 3.2):

```
int n, m;
cout << "Введите количество строк и столбцов: ";
cin >> n >> m;
double **p = new double *[n]; ①
for(int i = 0; i < n; i++) ②
 p[i] = new double[m]; ③
```

- ① — описываем «указатель на указатель» на элемент массива типа `double` и выделяем в динамической памяти место для хранения одномерного массива указателей на строки матрицы;
- ② — цикл по строкам матрицы;
- ③ — выделяем в динамической памяти место для хранения элементов  $i$ -й строки и присваиваем  $i$ -му элементу массива указателей адрес начала данного участка памяти.

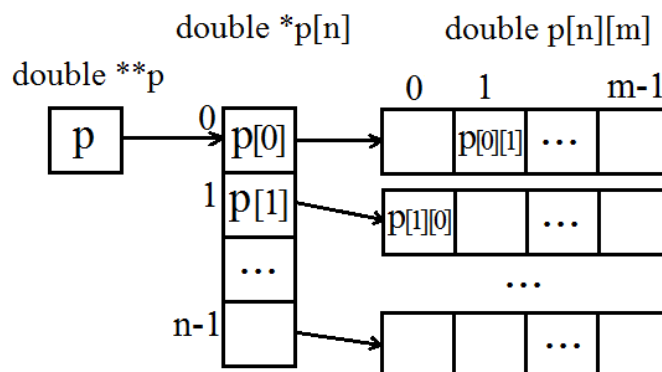


Рис. 3.2. Двумерный массив в динамической памяти

Освобождение памяти в этом случае выглядит следующим образом:

```
for(int i = 0; i < n; i++){
 delete [] p[i];
}
delete [] p;
```

т. е. сначала в цикле освобождается память, занятая каждой строкой матрицы, а затем уничтожается массив указателей на (уже уничтоженные) строки.

### 3.5. Контрольные вопросы

1. Пояснить смысл следующих понятий, привести примеры: а) массив; б) указатель; в) ссылка; г) динамическая память; д) утечка памяти.
2. Можно ли хранить в массиве элементы различных типов?
3. Доступ к отдельным элементам массива осуществляется по имени или по номеру?
4. Какой индекс имеет самый первый элемент одномерного массива в языке C/C++?
5. Как располагаются в памяти элементы одномерного массива?
6. Когда при описании массива допускается не указывать его размер?
7. Зачем используется функция `sizeof()`? Какие она имеет входные параметры? Какой результат и какого типа возвращает?
8. В программе требуется обратиться к 50-му элементу массива. Можно ли это сделать непосредственно, или придётся по очереди перебрать все предыдущие 49 элементов?
9. Какими способами можно создать динамическую переменную типа `double`?
10. Как создать одномерный динамический массив из  $n$  элементов типа `long int`?
11. Какой смысл имеют следующие строки в программе:  
а) `int *p = new int(5);`  
б) `int *p = new int[5];`
12. Программист создал в динамической памяти массив из 100 элементов. В ходе выполнения программы потребовалось добавить к этому массиву ещё 50 элементов. Как это сделать?
13. Описан массив из  $M$  элементов. Уже используются (имеют нужные значения) первые  $N$  элементов. В ходе выполнения программы потребова-

лось между  $i$ -м и  $(i + 1)$ -м элементом массива вставить новый элемент, сохранив значения всех уже имеющихся элементов. Как это сделать? Изменится ли ответ, если  $M = N$ ?

14. Зачем нужно освобождать динамическую память после использования?
15. В программе описан статический массив из 10 элементов. В некоторой строке программы происходит обращение к 11-му элементу. К чему это может привести? Возникнут ли ошибки во время компиляции и/или выполнения программы? Изменится ли ответ, если массив будет не статическим, а динамическим?
16. Перечислите преимущества и недостатки динамических массивов по сравнению со статическими.
17. Какими значениями инициализируются элементы статических массивов? А динамических?
18. В программе объявлены массив и указатель:

```
int a[10], *p;
```

Как присвоить указателю адрес самого первого элемента массива?

19. В программе создан динамический массив:

```
int *a = new int[m];
```

Как освободить выделенную область динамической памяти?

20. В программе создан динамический массив:

```
int *a = (int*)malloc(m*sizeof(int));
```

Как освободить выделенную область динамической памяти?

21. Что происходит, если при попытке выделения динамической области памяти выясняется, что свободная память отсутствует?
22. Какой тип данных описан в данной строке — указатель на массив или массив указателей:

```
int *a[10];
```

23. Как элементы многомерного массива хранятся в памяти компьютера?

24. Укажите три варианта обращения к элементу  $i$ -й строки  $j$ -го столбца двумерного массива  $a$ .

25. В программе имеются следующие строки:

```
int n = 2, m = 3;
int a[n][m];
```

Какую ошибку допустил программист?

26. Могут ли строки или столбцы двумерного массива содержать разное количество элементов?

## 4. Модульное программирование

### 4.1. Функции

Обычно при проектировании алгоритма сложную задачу делят на более простые. Каждую простую задачу решают отдельно.

► *Функция* — это именованная последовательность описаний и операторов, выполняющая какое-либо законченное действие.

Функция может принимать параметры и возвращать значение.

**Объявление и определение функции.** Программа на языке C/C++ состоит из функций, одна из которых должна иметь имя `main` — именно с неё начинается выполнение программы. Остальные функции начинают выполняться в момент *вызова*. Любая функция должна быть *объявлена* и *определена*. Объявлений функции в различных местах программы может быть несколько, но определение должно быть только одно. Определение функции должно находиться раньше её вызова (чтобы однопроходный компилятор смог проверить правильность вызова и сгенерировать нужный код).

*Объявление функции* (называемое также *прототипом* или *заголовком*) задаёт её имя, параметры и тип возвращаемого значения. *Определение функции* содержит, кроме объявления, *тело функции* — последовательность операторов и описаний, заключенные в фигурные скобки:

```
[класс] тип имя ([список_параметров]) [throw (исключения)]
{ тело_функции }
```

Необязательные класс и исключения будут рассмотрены ниже, без них определение функции выглядит проще:

```
тип имя ([список_параметров])
{ тело_функции }
```

Тип возвращаемого значения может быть любым, кроме массива и функции (но может являться указателем на массив или функцию). Если функция не возвращает никакого значения, то пишется `void`.

Список параметров определяет величины, которые передаются функции при её вызове. Элементы списка пишутся через запятую. Для каждого параметра указывается его тип и имя (в объявлении имя можно опустить).

В объявлении, определении и при вызове одной и той же функции типы и порядок следования параметров должны совпадать. В определении функции указываются так называемые *формальные параметры*, а при вызове — *фактические параметры*. Имена формальных и фактических параметров могут не совпадать: соотношение между ними устанавливается порядком следования.

В программе можно определить несколько функций с одинаковыми именами, если они имеют разный набор типов входных параметров.

**Пример 4.1.** Рассмотрим программу (листинг 4.1), в которой требуется найти модуль комплексного числа. Напомним, что комплексное число — это упорядоченная пара вещественных чисел (действительная и мнимая часть), а модуль (длина вектора на комплексной плоскости) вычисляется по теореме Пифагора: как корень из суммы квадратов действительной и мнимой частей комплексного числа. На рис. 4.1 показан результат работы программы.

**Листинг 4.1: Вычисление модуля комплексного числа с помощью функции**

```
1 #include <iostream> // для cin и cout
2 #include <cmath> // для sqrt()
3 #include <windows.h> // для system()
4
5 using namespace std;
6
7 // Функция для вычисления модуля комплексного числа
8 // Вход: x1 - действительная часть;
9 // x2 - мнимая часть числа.
10 // Выход: модуль комплексного числа
11 double complAbs(double x1, double x2) {
12 double z;
```

```
13 z = sqrt(x1*x1 + x2*x2);
14 return z;
15 }
16
17 int main () {
18 double a1, a2, x;
19 SetConsoleOutputCP(1251);
20 cout << "Введите действительную и мнимую часть\n"
21 "комплексного числа:\n";
22 cin >> a1 >> a2;
23 x = complAbs(a1, a2); // Вызов функции
24 cout << "Модуль = " << x << endl;
25 system("pause");
26 return 0;
27 }
```

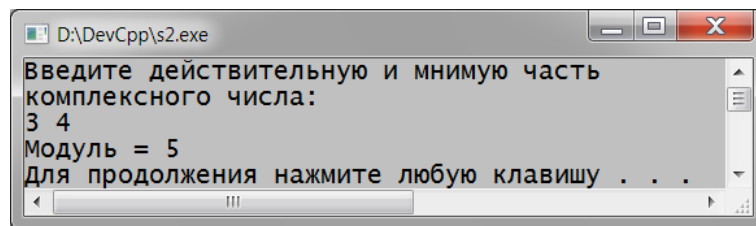


Рис. 4.1. Экран программы из листинга 4.1

Здесь сначала определена функция `complAbs` (строка 11), имеющая два вещественных параметра и возвращающая вещественное значение. Хорошим стилем программирования считается подробное документирование всех функций, т. е. указание комментариев, поясняющих смысл каждой функции, её входных параметров и возвращаемого результата.

В строке 12 описана локальная переменная; область её действия — от момента описания до конца функции `complAbs`.

В строке 14 используется оператор возврата `return`, в нём указывается значение, которая функция `complAbs` должна вернуть в место её вызова.

В строке 18 описаны локальные переменные, используемые в функции `main`.

В строке 23 происходит вызов функции `complAbs`, при этом имена фактических параметров `a1` и `a2` никак не связаны с именами формальных параметров `x1` и `x2`, важен только их тип и порядок.

**Пример 4.2.** В предыдущем примере функция сначала определена, а затем используется (вызывается). В некоторых случаях сохранить данный порядок невозможно: например, одна функция может вызывать другую, а та, в свою очередь, — первую. Тогда приходится использовать предварительные объявления функций (листинг 4.2).

#### Листинг 4.2: Предварительное объявление функции

```
1 #include <iostream>
2 #include <cmath>
3 #include <windows.h>
4
5 using namespace std;
6
7 double complAbs(double, double);
8
9 int main () {
10 double a1, a2, x;
11 SetConsoleOutputCP(1251);
12 cout << "Введите действительную и мнимую часть\n"
13 "комплексного числа:\n";
14 cin >> a1 >> a2;
15 x = complAbs(a1, a2);
16 cout << "Модуль = " << x << endl;
17 system("pause");
18 return 0;
19 }
20
```

```
21 double complAbs(double x1, double x2) {
22 double z;
23 z = sqrt(x1*x1 + x2*x2);
24 return z;
25 }
```

В строке 7 объявлена функция `complAbs` (при этом имена её формальных параметров писать не обязательно — важны только их типы). В строке 15 она вызывается с указанием фактических параметров. И только в строке 21 начинается определение этой функции (здесь имена формальных параметров обязательны).

**Рекурсивные функции.** Функция называется *рекурсивной*, если она вызывает сама себя (прямо или косвенно). Пример рекурсивной функции приведён в листинге 4.3.

#### Листинг 4.3: Рекурсивная функция для вычисления факториала

```
#include <iostream>
using namespace std;

// Рекурсивная функция - вычисление факториала числа n:
long fact(int n){
 if(n < 0) // Если отрицательное число -
 return 0; // возвращаем 0.
 if (n == 0)
 return 1; // 0! = 1
 else // В остальных случаях -
 return n * fact(n - 1); // рекурсия
}

int main(){
 int n;
 setlocale(0, ""); // Включаем кириллицу
```

```
cout << "Введите число для вычисления факториала = ";
cin >> n;
cout << n << "! = " << fact(n) << endl;
system("pause");
return 0;
}
```

Программист должен следить за тем, чтобы рекурсивная функция обязательно имела терминальную ветвь: рано или поздно процесс рекурсивных вызовов должен закончиться.

Выполнение рекурсивной функции связано с дополнительным расходом времени и памяти для манипуляций со стеком и может вызвать переполнение стека, поэтому обычно стараются изменить алгоритм так, чтобы заменить рекурсию обычным циклом, например, как показано в листинге 4.4.

#### Листинг 4.4: Нерекурсивная функция для вычисления факториала

```
// Нерекурсивная функция - вычисление факториала n:
long fact(int n){
 if(n < 0) // Если отрицательное число -
 return 0; // возвращаем 0.

 long k = 1;
 for(int i=1; i < n;)
 k *= ++i;
 return k;
}
```

**Ханойские башни.** В листинге 4.5 приведён пример использования рекурсивной функции для решения задачи о Ханойских башнях.

Даны три стержня, на один из которых нанизаны  $n$  колец, причём кольца отличаются размером и лежат в порядке уменьшения диаметра (меньшее на большем). Задача состоит в том, чтобы перенести пирамиду из  $n$  колец на

третий стержень. Вторым стержнем можно пользоваться для временного размещения колец. Надо решить задачу за наименьшее число ходов. За один ход разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее.

#### Листинг 4.5: Ханойские башни

```
1 #include <iostream>
2 using namespace std;
3
4 static long count; // счётчик ходов (глобальная переменная)
5
6 void hanoi_towers(int n, // число колец
7 int from, // номер начального стержня
8 int to, // номер конечного стержня
9 int buf // номер промежуточного стержня
10) {
11 if (n != 0) {
12 hanoi_towers(n-1, from, buf, to);
13 ::count++;
14 cout << from << " -> " << to << endl;
15 hanoi_towers(n-1, buf, to, from);
16 }
17 }
18
19 int main() {
20 int n, start, dest, buf;
21 setlocale(LC_ALL, "rus"); // кодировка для кириллицы
22
23 cout << "Номер первого стержня (от 1 до 3) = ";
24 cin >> start;
25 cout << "Номер конечного стержня (от 1 до 3) = ";
26 cin >> dest;
27 cout << "Номер промежуточного стержня (от 1 до 3) = ";
```

```
28 cin >> buf;
29 cout << "Количество дисков = ";
30 cin >> n;
31
32 ::count = 0;
33 hanoi_towers(n, start, dest, buf);
34 cout << "Сделано " << ::count << " ходов\n";
35 system("pause");
36 return 0;
37 }
```

Доказано, что  $n$  колец можно переложить за  $2^n - 1$  ходов. Согласно буддийской легенде, конец мира наступит, когда будет окончена игра с 64 стержнями. К счастью, для этого потребуется 18 446 744 073 709 551 615 ходов, что займёт 580 миллиардов лет, если мы делаем один ход за секунду.

**Встраиваемые функции.** При определении функции компилятор создаёт соответствующий машинный код в области памяти с некоторым адресом. При каждом вызове функции в машинный код вставляются команды для копирования в стек параметров (а также адреса возврата) и команда перехода на тот адрес, по которому расположен машинный код функции. После выполнения функции компилятор генерирует машинные команды, которые очищают стек и возвращают управление на команду, следующую за точкой вызова. Все эти операции связаны с расходом времени и памяти. Если функция небольшая (соответствует малому числу команд процессора), то её можно оформить как *встраиваемую*. Тогда компилятор по возможности не будет размещать машинный код функции в отдельной области памяти, а будет генерировать для каждого вызова функции соответствующие ей машинные команды; стек при этом не используется.

Чтобы сделать функцию встраиваемой, в её заголовке надо написать ключевое слово `inline`, например:

```
inline double sum(double a, double b){
```

```
 return a+b;
}
```

Объявление функции встраиваемой выражает лишь пожелание программиста: компилятор оценивает размеры функции и сам решает, выполнять это пожелание или игнорировать.

Спецификатор `inline` будет проигнорирован, если функция является рекурсивной.

**Перегрузка функций.** Часто бывает необходимо реализовать один и тот же алгоритм для разного набора типов данных. В таких случаях удобно, чтобы соответствующие функции имели одинаковое имя. Использование нескольких функций с одним и тем же именем, но разным набором типов параметров, называется *перегрузкой*. Компилятор определяет, какую именно функцию следует вызвать, по типу фактических параметров (*разрешение перегрузки*). Тип возвращаемого значения при этом не используется.

Например, в программе имеются перегруженные функции `max` для нахождения максимального из двух чисел:

```
int max(int, int);
long max(long, long);
float max(float, float);
double max(double, double);
```

Тогда при вызове

```
double a=1.3, b=2.5, c;
c = max(a, b);
```

будет использована последняя функция `max`. Если компилятор не найдёт точного соответствия между типами фактических и формальных параметров перегруженных функций, то будет сделана попытка преобразования типов по стандартным правилам. Если с помощью различных вариантов преобразований будет обнаружена возможность использования нескольких разных функций (неоднозначность разрешения перегрузки), то компилятор выдаст сооб-

щение об ошибке. Тогда программист должен будет использовать явное приведение типов параметров. В листинге 4.6 показан пример неоднозначности разрешения перегрузки.

**Листинг 4.6: Неоднозначность разрешения перегрузки**

```
1 #include <iostream>
2 using namespace std;
3
4 int max(int a, int b){
5 return a > b ? a : b;
6 }
7 long max(long a, long b){
8 return a > b ? a : b;
9 }
10 float max(float a, float b){
11 return a > b ? a : b;
12 }
13 double max(double a, double b){
14 return a > b ? a : b;
15 }
16
17 int main(){
18 double x1=3, x2=4.5, y;
19 y = max(x1, x2);
20 int i=1, j=-3, k;
21 k = max(i, j);
22 double a=1.0, z;
23 int b = 7;
24 z = max(a, b); // Здесь ошибка компиляции!
25 //
26 return 0;
27 }
```

Здесь для вызова функции `max` в строке 24 имеются три кандидата:

```
long max(long, long);
float max(float, float);
double max(double, double);
```

поэтому компилятор выдаст сообщение об ошибке.

Однако, если написать:

```
z = max(a, (double)b);
```

то программа будет скомпилирована без ошибок.

## 4.2. Параметры функций

**Стек и параметры функции.** Вызовы функций в сложных программах могут быть вложены в друг друга: одна функция вызывает другую, та — третью и т.д. При каждом вызове необходимо передать значения параметров и запомнить адрес возврата (куда вернуться после того, как выполнение функции завершится). Для управления вызовами функций используется специальным образом организованная область памяти, называемая *стеком*.

*Стек* — это буфер, который функционирует по правилу: последним вошёл — первым вышел (LIFO — Last In, First Out). Именно в стеке размещает компилятор адрес возврата и значения фактических параметров функции при её вызове. Когда выполнение функции завершается (после выполнения её последнего оператора или при выполнении команды `return`), параметры убираются из стека вместе с адресом возврата и управление передаётся по этому адресу, т.е. к оператору, находящемуся в тексте программы после вызова только что отработавшей функции.

Параметры функции могут передаваться в неё по значению, адресу (указателю) или ссылке. Способ передачи задаётся типом формального параметра в заголовке функции при её объявлении и определении.

**Передача параметров по значению.** При передаче *по значению* в стек заносятся копии значений параметров. Операторы вызванной функции рабо-

тают с этими копиями, а не с переменными вызвавшей программы, поэтому любые изменения значений этих копий, размещённых в стеке, никак не отражаются на значениях исходных переменных в вызвавшей программе.

**Пример 4.3. Передача параметров по значению.** В листинге 4.7 определена функция `fun1`, имеющая три вещественных параметра и не возвращающая результата (`void`). Все три параметра переданы по значению (такой режим действует в C/C++ по умолчанию), но внутри функции значение третьего параметра изменяется. В конце данной функции оператор `return` можно опустить (или указать его без параметра).

Листинг 4.7: Передача параметров по значению

```
1 void fun1(double x, double y, double z) {
2 z = x + y;
3 return; // можно опустить
4 }
5
6 int main () {
7 double a, b, c=0;
8 cin >> a >> b;
9 fun1(a, b, c); // вызов функции
10 cout << c << endl; // будет выведен 0
11 return 0;
12 }
```

Далее функция `fun1` вызывается с фактическими параметрами `a`, `b` и `c`. Третий параметр при вызове имеет нулевое значение. Оно не изменится и после вызова (на экран будет выведено число 0), хотя соответствующий формальный параметр `z` изменил своё значение внутри функции `fun1`.

Если мы хотим, чтобы изменения значений параметров функции переходили в вызвавшую программу, надо использовать передачу параметров по адресу или по ссылке.

**Передача параметров по адресу (указателю).** Чтобы передать какой-либо параметр функции *по адресу*, используются указатели. При этом в стек копируется не значение переменной, а её адрес.

**Пример 4.4. Передача параметра по адресу.** В листинге 4.8 первые два параметра *a* и *b* передаются, как и прежде, по значению, а третий параметр *c* — по адресу (формальный параметр имеет тип указателя). В теле функции для третьего параметра используется операция разыменования (звёздочка). При вызове функции *fun1* на месте третьего параметра должно быть указано не имя или значение, а адрес (т. е. фактическая переменная-указатель или, как в данном случае, операция взятия адреса переменной *&*). Теперь на экран будет выведена сумма двух введённых чисел (заданных при вызове функции на месте первых двух параметров).

**Листинг 4.8: Передача параметра по адресу**

```
1 void fun1(double x, double y, double *z) {
2 *z = x + y;
3 }
4
5 int main () {
6 double a, b, c=0;
7 cin >> a >> b;
8 fun1(a, b, &c); // вызов функции
9 cout << c << endl; // будет выведена сумма
10 return 0;
11 }
```

**Передача параметров по ссылке** отсутствовала в языке C, но в C++ именно её рекомендуется использовать вместо указателей, когда некоторые параметры функции являются выходными, т. е. должны передавать вычисленные значения в вызвавшую программу. Как и в случае с указателями, при передаче параметра по ссылке в стек копируется адрес переменной, а не её значение.

**Пример 4.5. Передача параметра по ссылке.** В листинге 4.9 при определении функции третий параметр является ссылкой, поэтому автоматически разыменовывается в теле функции (звёздочка не нужна). При вызове функции на месте третьего параметра указывается фактическая переменная (операция взятия адреса также не требуется).

**Листинг 4.9: Передача параметра по ссылке**

```
1 void fun1(double x, double y, double & z) {
2 z = x + y;
3 }
4
5 int main () {
6 double a, b, c=0;
7 cin >> a >> b;
8 fun1(a, b, c); // вызов функции
9 cout << c << endl; // будет выведена сумма
10 return 0;
11 }
```

Как видим, при использовании ссылок текст программы выглядит более понятным, чем с указателями.

**Константные параметры функции.** В некоторых случаях параметр функции приходится передавать по адресу или ссылке, но нам требуется, чтобы его значение не изменялось при выполнении функции. Перед таким формальным параметром в заголовке функции надо поставить ключевое слово `const`.

**Передача массива в качестве параметра функции.** Напомним, что имя массива — это адрес нулевого элемента. Поэтому массивы не могут передаваться в функцию по значению (это разумно, т.к. при копировании в стек всех элементов большого массива возникло бы переполнение стека). Кроме

адреса массива в функцию требуется передавать все его размерности. Пример функции для суммирования элементов массива показан в листинге 4.10.

#### Листинг 4.10: Суммирование элементов одномерного массива

```
#include <iostream>
using namespace std;

// Ввод элементов массива:
void arrinput(long n, double arr[]){
 for(long i=0; i<n; i++){
 cout << "arr[" << i << "] = ";
 cin >> arr[i];
 }
}

// Вывод элементов массива:
void arrprint(long n, const double arr[] const){
 for (long i=0; i<n; i++)
 cout << "arr[" << i << "] = " << arr[i] << endl;
}

// Суммирование элементов массива:
double arrsum(long n, const double arr[] const){
 double s = 0;
 for (long i=0; i<n; i++)
 s += arr[i];
 return s;
}

int main(){
 long n;
 setlocale(0, ""); // Включаем кириллицу
 cout << "Количество элементов = ";
```

```

cin >> n;
double *a = new double[n];
arrinput(n, a); // Ввод элементов массива
cout << "Введён массив: \n";
arrprint(n, a); // Вывод элементов массива
double x = arrsum(n, a); // Суммирование элементов
cout << "Сумма элементов = " << x << endl;
delete [] a;
system("pause");
return 0;
}

```

Здесь параметр-массив `arr` во всех трёх функциях передаётся по адресу, но в функции `arrinput` элементы массива изменяются и это изменение передаётся в главную программу. В остальных двух функциях `arrprint` и `arrsum` перед именем массива в списке формальных параметров стоит ключевое слово `const`. Это означает, что элементы массива не должны изменяться внутри функции. Если мы, например, попробуем написать `arr[i] = 0`, то получим ошибку во время компиляции программы.

Вместо `double arr[]` для одномерного массива в списке формальных параметров можно писать `double *arr`. Если элементы массива не должны изменяться, то пишут `const double *arr`. Чтобы нельзя было изменить и сам указатель на начало массива, пишут `const double * const arr`.

В листинге 4.11 показана аналогичная программа для двумерного массива.

#### Листинг 4.11: Суммирование элементов двумерного массива

```

#include <iostream>
using namespace std;

// Ввод элементов массива:
void arrinput2(long n, long m, double *arr[]){
 for(long i=0; i<n; i++)

```

```
 for(long j=0; j<m; j++){
 cout << "arr[" << i << ", "
 << j << "] = ";
 cin >> arr[i][j];
 }
 }

// Вывод элементов массива:
void arrprint2(long n, long m, const double * const arr[]){
 for (long i=0; i<n; i++)
 for(long j=0; j<m; j++)
 cout << "arr[" << i << ", "
 << j << "] = " << arr[i][j]
 << endl;
}

// Суммирование элементов массива:
double arrsum2(long n, long m, const double * const arr[]){
 double s = 0;
 for (long i=0; i<n; i++)
 for (long j=0; j<m; j++)
 s += arr[i][j];
 return s;
}

int main(){
 long n, m;
 setlocale(0, ""); // Включаем кириллицу
 cout << "Количество строк = ";
 cin >> n;
 cout << "Количество столбцов = ";
 cin >> m;

 // Выделение памяти (массив указателей на строки):
```

```
double **a = new double *[n];
for(long i = 0; i < n; i++)
 a[i] = new double[m];

arrinput2(n, m, a); // Ввод элементов массива
cout << "Введён массив: \n";
arrprint2(n, m, a); // Вывод элементов массива
double x = arrsum2(n, m, a); // Суммирование элементов
cout << "Сумма элементов = " << x << endl;
for(long i = 0; i < n; i++) // Освобождение памяти
 delete [] a[i];

delete [] a;
system("pause");
return 0;
}
```

Вместо `double *arr[]` для двумерного массива в списке формальных параметров можно писать `double **arr`.

Что касается передачи массива по ссылке `double (&arr)[]`, то этот способ обычно не используют, поскольку он не работает с динамическими массивами произвольной длины.

**Указатель на функцию как параметр функции.** Функция не может являться параметром другой функции, но при необходимости можно передать указатель на функцию. В листинге 4.12 показан пример функции, предназначенной для вывода на экран таблицы любой другой функции, переданной первой в качестве указателя.

#### Листинг 4.12: Указатель на функцию как параметр функции

```
#include <stdio.h>
#include <math.h>
#include <windows.h>
```

```
// Указатель на функцию, имеющую два параметра
// типа int и double
// и возвращающую значение типа double:
typedef double (*Fun)(int, double);

// Первая функция:
double f1(int k, double x){
 return x * (x - k);
}

// Вторая функция:
double f2(int k, double x){
 return x / k;
}

// Третья функция (с другой сигнатурой):
double f3(double x){
 return x * x;
}

// Вывод таблицы заданной функции f:
void tabfun(long n, double a, double b, Fun f, int k){
 double d = (b - a)/(n - 1); // Шаг изменения x
 printf("\n-----\n");
 printf(" x\t\tty\n"); // \t - символ табуляции
 printf("-----\n");
 double x = a;
 for (long i=0; i<n; i++){
 double y = f(k, x);
 printf("%5.2lg \t %8.3lg\n", x, y);
 x += d;
 }
}
```

```

}

int main(){
 double a, b, y, x, d;
 long n;
 int k;
 SetConsoleOutputCP(1251);
 printf("\n ===== Табулирование функции =====\n");
 printf("Начало = "); scanf("%lg", &a);
 printf("Конец = "); scanf("%lg", &b);
 printf("Кол-во строк = "); scanf("%li", &n);
 printf("Целый параметр = "); scanf("%i", &k);
 printf("\nФункция 1:");
 tabfun(n, a, b, f1, k);
 printf("\nФункция 2:");
 tabfun(n, a, b, f2, k);
 // tabfun(n, a, b, f3, k); - здесь была бы ошибка!
 system("pause");
 return 0;
}

```

Здесь мы описали новый тип `Fun` — указатель на функцию, имеющую один целый и один вещественный параметр и возвращающую вещественный результат. Определили функцию `tabfun`, которая реализует вывод на экран значения любой функции в заданном диапазоне изменений аргумента  $x \in [a, b]$ . Затем вы вызвали функцию `tabfun` два раза для вывода таблицы двух разных математических функций. В качестве фактического параметра `f` функции `tabfun` можно задать указатель только на такие функции, которые имеют нужную сигнатуру (набор типов параметров).

**Значения параметров по умолчанию.** При объявлении функции в её заголовке можно указывать значения по умолчанию для последних параметров. Тогда при вызове функции можно опустить те параметры, которые имеют

значение по умолчанию. В листинге 4.13 показан пример функции, которая может суммировать два или три аргумента. Третий параметр по умолчанию (если не указан при вызове) принимает нулевое значение.

#### Листинг 4.13: Значения параметров по умолчанию

```
#include <iostream>
using namespace std;

double sum23(double x1, double x2, double x3=0){
 return x1 + x2 + x3;
}

int main(){
 double x = sum23(3, 5, 2);
 double y = sum23(1, 4);
 cout << x << ", " << y << endl;
 system("pause");
 return 0;
}
```

Если значение по умолчанию задано для нескольких параметров (они должны быть последними в списке формальных параметров), а при вызове некоторый параметр опущен, то должны быть опущены и все остальные параметры, стоящие в списке после него.

**Функции с переменным числом параметров.** Если список формальных параметров функции заканчивается многоточием, то при вызове такой функции на месте многоточия можно указать несколько параметров любого типа. Таким способом определены некоторые системные функции, имеющие произвольное число аргументов, например, `printf`:

```
int printf(const char*, ...);
```

Для доступа к фактическим параметрам, переданным при вызове вместо многоточия, используются макросы из заголовочного файла `<stdarg.h>`.

**Возвращение указателя из функции.** Если функция должна возвращать другую функцию, массив или сложную структуру данных, до приходится использовать указатели. При этом надо следить за временем жизни той переменной, указатель на которую мы возвращаем из функции (независимо от того, указана она в операторе `return` или является параметром функции). В листинге 4.14 приведён пример распространённой ошибки: из функции возвращается указатель на локальную переменную, описанную в самой функции. Поскольку после выхода из функции все её локальные (не статические) переменные уничтожаются, то возвращаемый указатель уже никуда не указывает: результат выполнения программы становится непредсказуемым. При компиляции данной программы будет выдано предупреждение (warning).

**Листинг 4.14: Возвращение указателя из функции: ошибка**

```
1 #include <iostream>
2 using std::cout;
3 using std::endl;
4
5 double * fun1(double);
6
7 int main (void){
8 double num = 7.0;
9 double * ptr =0;
10 ptr = fun1 (num);
11 cout << *ptr << endl;
12 system("pause");
13 return 0;
14 }
15
16 double * fun1 (double x){
17 double result = 0.0; // Локальная переменная
18 result = 3.0 * x;
19 return &result; // Warning!
```

20 }

### 4.3. Многофайловые проекты

**Размещение определений функций в отдельных файлах.** Сложные программы содержат большое число функций и часто разрабатываются командой программистов, поэтому удобно разделять весь текст программы на отдельные модули — файлы, содержащие определения функций, связанных по смыслу. Чтобы гарантировать, что до вызова любой функции компилятору уже будет известно количество и типы её параметров, а также тип возвращаемого значения, — объявления всех функций, определённых в каждом файле, помещают в отдельный *заголовочный файл* (обычно с расширением `.h`). Этот файл подключают к тексту программы с помощью знакомой нам директивы препроцессора `#include`.

**Пример 4.6.** Вынесем функцию из примера 4.1. (см. с. 106) в отдельный модуль (файл с именем `compl.cpp`), а заголовок функции (её объявление) поместим в заголовочный файл `compl.h`. Файл, в котором определена главная функция, обычно так и называют: `main.cpp`. В итоге получится три файла (листинг 4.15).

**Листинг 4.15: Определение функции в отдельном файле**

```

1 // ----- файл main.cpp: -----
2 #include <iostream> // для cin и cout
3 #include <windows.h> // для system
4 #include "compl.h" // для complAbs ①
5
6 using namespace std;
7
8 int main () {
9 double a1, a2, x;
10 SetConsoleOutputCP(1251);
11 cout << "Введите действительную и мнимую часть\n"
```

```
12 "комплексного числа:\n";
13 cin >> a1 >> a2;
14 x = complAbs(a1, a2); // вызов функции
15 cout << "Модуль = " << x << endl;
16 system("pause");
17 return 0;
18 }
19
20 // ----- заголовочный файл compl.h: -----
21 double complAbs(double x1, double x2); ②
22
23
24 // ----- файл compl.cpp: -----
25 #include <cmath> // для sqrt ③
26
27 double complAbs(double x1, double x2) {
28 double z;
29 z = sqrt(x1*x1 + x2*x2);
30 return z;
31 }
```

① — в главном модуле `main.cpp` должны быть подключены необходимые заголовочные файлы. Обратите внимание, что в отличие от системных, они заключаются в кавычки, а не в угловые скобки;

② — заголовочный файл содержит только объявление нашей функции;

③ — подключение объявлений математических функций переключали из главной программы (где они больше не нужны) в тот модуль, где они теперь используются.

## 4.4. Шаблоны функций

## 5. Символы, строки, структуры, файлы

### 5.1. Символы

Символьные константы, определённые в программе на языке C, заключаются в одиночные кавычки, например: 'а'.

Значением символьной константы является численное значение (код) этого символа, определённый в кодовой таблице ASCII (American Standard Code for Information Interchange — американский стандартный код для обмена информацией). Код каждого символа в коде ASCII занимает один байт, т.е. всего можно кодировать  $2^8 = 256$  различных символов (от 0 до 255). Часто коды символов записывают в шестнадцатеричном виде, т.е. от 0x00 до 0xFF.

**Таблица 5.1.** Первая часть кодовой таблицы ASCII

(строка — первая 16-ричная цифра кода; столбец — вторая цифра)

|    | 00  | 01  | 02  | 03  | 04  | 05  | 06  | 07  | 08  | 09 | 0A  | 0B  | 0C | 0D | 0E | 0F  |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|-----|-----|----|----|----|-----|
| 00 | NUL | SOH | STX | ETX | EOT | ENQ | ACK | BEL | BS  | HT | LF  | VT  | FF | CR | SO | SI  |
| 10 | DLE | DC1 | DC2 | DC3 | DC4 | NAK | SYN | ETB | CAN | EM | SUB | ESC | FS | GS | RS | US  |
| 20 |     | !   | "   | #   | \$  | %   | &   | '   | (   | )  | *   | +   | ,  | -  | .  | /   |
| 30 | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9  | :   | ;   | <  | =  | >  | ?   |
| 40 | @   | A   | B   | C   | D   | E   | F   | G   | H   | I  | J   | K   | L  | M  | N  | O   |
| 50 | P   | Q   | R   | S   | T   | U   | V   | W   | X   | Y  | Z   | [   | \  | ]  | ^  | _   |
| 60 | `   | a   | b   | c   | d   | e   | f   | g   | h   | i  | j   | k   | l  | m  | n  | o   |
| 70 | p   | q   | r   | s   | t   | u   | v   | w   | x   | y  | z   | {   |    | }  | ~  | DEL |

В кодовой таблице ASCII первые 32 символа (с кодами от 0 до 31) являются управляющими, например, символ с кодом 7 — звуковой сигнал (bell); 8 — стирание предыдущего символа (backspace); 9 — горизонтальная табуляция (horizontal tab); 0x0A — перевод строки (line feed); 0x0D — возврат каретки (carriage return). Управляющим также является символ с кодом 0xFF — удаление (delete).

**Таблица 5.2.** Вторая часть кодовой таблицы cp1251

(строка — первая 16-ричная цифра кода; столбец — вторая цифра)

|    | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 80 | Ђ  | Ѓ  | И  | Ѕ  | „  | …  | †  | ‡  | €  | ‰  | Љ  | <  | Њ  | Ќ  | Љ  | Ц  |
| 90 | ђ  | ѓ  | и  | ѕ  | ”  | •  | —  | —  |    | ™  | љ  | >  | њ  | ќ  | ћ  | ц  |
| A0 |    | Ў  | Ў  | Ј  | Ѡ  | Г  | І  | Ѕ  | Ё  | ©  | Є  | «  | ¬  |    | ®  | Ї  |
| B0 | °  | ±  | І  | і  | Г  | μ  | ¶  | ·  | ё  | №  | є  | »  | ј  | Ѕ  | ѕ  | ї  |
| C0 | А  | Б  | В  | Г  | Д  | Е  | Ж  | З  | И  | Й  | К  | Л  | М  | Н  | О  | П  |
| D0 | Р  | С  | Т  | У  | Ф  | Х  | Ц  | Ч  | Ш  | Щ  | Ъ  | Ы  | Ь  | Э  | Ю  | Я  |
| E0 | а  | б  | в  | г  | д  | е  | ж  | з  | и  | й  | к  | л  | м  | н  | о  | п  |
| F0 | р  | с  | т  | у  | ф  | х  | ц  | ч  | ш  | щ  | ъ  | ы  | ь  | э  | ю  | я  |

Первая (стандартная) часть кодовой таблицы ASCII включает 128 символов (с кодами от 0 до 127): цифры, заглавные и строчные буквы латинского алфавита, знаки препинания (табл. 5.1). Символ пробела имеет код 0x20, т.е. 32. Заглавные символы латинского алфавита: коды с 0x41 по 0x5A; строчные: с 0x61 по 0x7A; цифры: с 0x30 по 0x39.

Вторая часть (расширенная таблица) содержит символы национальных алфавитов и другие специальные знаки. Существует несколько вариантов кодирования второй части кодовой таблицы. Например, в русской версии операционной системы Microsoft Windows используется кодировка cp1251 (табл. 5.2), а в консольных окнах — кодовая таблица 866.

Для работы с символами в языке C используются функции из стандартной библиотеки `<cctype>` (или `<ctype.h>`).

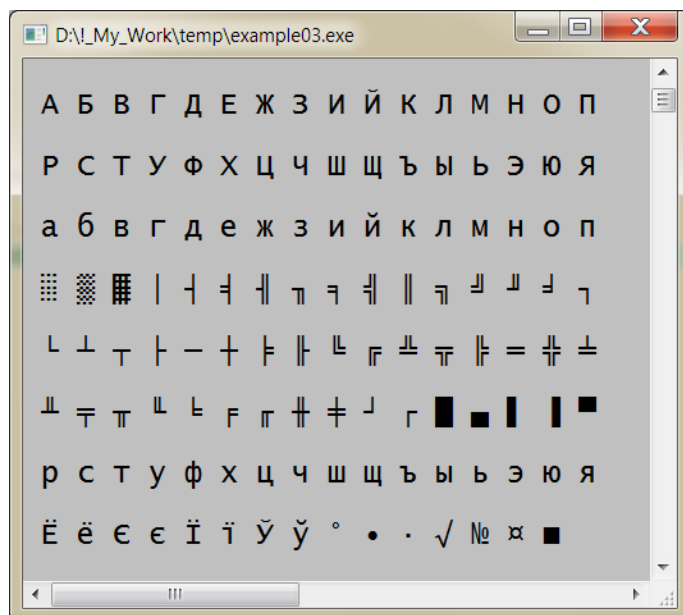
Функция `tolower()` преобразует символ к нижнему регистру, если это возможно (в противном случае возвращает неизменённое значение). Функция `toupper()` преобразует символ к верхнему регистру.

Функция `isupper()` возвращает ненулевое значение, если аргумент является заглавным символом; функция `islower()` возвращает ненулевое зна-

чение, если аргумент является строчным символом. Функция `isdigit()` возвращает ненулевое значение, если аргумент является цифрой.

Для вывода символьной переменной на экран с помощью функции `printf` используется спецификатор формата `%c`.

**Пример 5.1. Вывод на экран таблицы символов.** Напишем программу, которая позволяет вывести на экран все символы кодовой таблицы (листинг 5.1). На рис. 5.1 показано консольное окно с результатом работы программы. Обращаем внимание, что выводимая на экран кодовая таблица соответствует текущей кодировке, установленной в операционной системе, т. е. если бы перед выполнением программы мы изменили кодировку с помощью команды `chcp`, то результат был бы другим.



**Рис. 5.1.** Кодовая таблица 866 (экран программы из листинга 5.1)

#### Листинг 5.1: Таблица символов cp866

```
#include <iostream>
using namespace std;

int main(){
 for(int i=0x80; i<=0xFF; i++){
 if(!(i % 16)) // в одной строке 16 символов
 cout << endl << endl;
 }
}
```

```
 cout << " " << char(i); // printf(" %c", i);
}
cout << endl << endl;
system("PAUSE");
return 0;
}
```

Вводить и выводить символы можно с помощью функций `getchar` и `putchar`. При использовании `getchar` в конце ожидается нажатие <Enter>, вводимые символы отображаются на экране. Функция `getch`, определённая в заголовочном файле `<conio.h>`, позволяет вводить символы без отображения их на экране и без использования <Enter>.

**Пример 5.2. Отображение кода введённого символа.** В листинге 5.2 приведена программа, которая ожидает нажатие клавиши и выводит на экран соответствующий символ и его код (в текущей кодировке). После нажатия клавиши <ESC> выполнение программы завершается.

#### Листинг 5.2: Отображение кода введённого символа

```
#include <stdio.h>
#include <conio.h>

int main(){
 char c;
 do {
 c = getch(); // ввод символа без эха
 printf("%c - %d\n", c, c);
 } while(c != 0x1B); // пока не нажали ESC
 return 0;
}
```

Функция `getche` работает аналогично `getch`, но вводимый символ отображается на экране, поэтому тело цикла `do` из листинга 5.2 можно переписать следующим образом:

```
c = getche();
printf(" - %d\n", c);
```

Если в программе используются потоки ввода-вывода, то для ввода любого символа (включая символ пробела) рекомендуется использовать метод `cin.get()`.

## 5.2. Строки

Строка представляет собой массив символов, заканчивающийся нулевым байтом. Варианты описания строковой переменной:

```
const int n = 10;
char s1[n]; ①
char s2[n] = "Hello!"; ②
char s3[] = "Привет!"; ③
```

- ① — длина строки должна быть константным выражением (значение которого известно на момент компиляции). Здесь для переменной `s1` выделено 10 байт, 9 из которых можно занять символами, а последний — нулевой.
- ② — можно использовать инициализатор в виде строкового литерала. Здесь для переменной `s2` выделено 10 байт. Первые семь байт инициализированы символами 'н', 'е', 'л', 'л', 'о', '!' и '\0'.
- ③ — при использовании инициализатора допускается не указывать длину строки (компилятор отведёт ровно столько места, сколько необходимо для хранения всех символов плюс нуль-символ в конце). Здесь в памяти для переменной `s3` выделено 7 байт (номера символов — от 0 до 6).

Для размещения строки в динамической памяти надо описать указатель на `char`, выделить память и присвоить адрес начала выделенного участка памяти данному указателю:

```
int m;
cin >> m;
```

```
char *p1 = new char[m]; // вариант 1
char *p2 = (char*)malloc(m*sizeof(char)); // вариант 2
```

Строки в динамической памяти, как и массивы, нельзя инициализировать. Например, если написать:

```
char *p3 = "Привет!";
```

то будет создана не динамическая строка, а строка-константа, изменить которую невозможно; адрес её начала будет присвоен указателю.

Для вывода строковой переменной на экран с помощью функции `printf` используется спецификатор формата `%s`:

```
char s1[] = "Hello!";
char *s2 = "Привет!";
printf("%s\n%s", s1, s2);
```

Пример ввода строки с клавиатуры с помощью функции `scanf`:

```
const int n = 20;
char s1[n];
scanf("%s", s1);
printf("%s", s1);
```

Строка считается массивом символов, а имя массива в языке C представляет адрес его самого первого байта, поэтому традиционный `'&'` во втором параметре функции `scanf` не требуется. Следует иметь в виду, что функция `scanf` читает только часть строки, до первого пробельного символа<sup>1</sup>, остальное игнорируется.

Чтобы ввести всю строку, вместе со всеми пробелами, придётся использовать спецификатор `'c'`, указав максимальное число ожидаемых символов:

```
const int n = 20;
char s1[n];
scanf("%20c", s1);
printf("%s", s1);
```

---

<sup>1</sup>Пробельным символом считается сам пробел, символ табуляции и символ конца строки.

Но лучше использовать функции языка C, специально предназначенные для ввода и вывода строк: `gets` и `puts`:

```
const int n = 20;
char s1[n];
gets(s1); // ввели строку
puts(s1); // вывели строку
```

Потоки ввода-вывода `cin` и `cout` также могут быть использованы при работе со строками:

```
cin >> s1;
cout << s1 << endl;
```

но при вводе будет прочитана только часть строки до первого пробельного символа. Поэтому для ввода рекомендуется применять специальный метод `getline`:

```
const int n = 20;
char s1[n];
cin.getline(s1, n);
cout << s1 << endl;
```

Функция `cin.getline(s1, n)` читает из входного потока  $(n-1)$  символов (или менее, если символ перевода строки встретится раньше) и записывает их в строковую переменную `s1`, в конце добавляется нулевой символ.

Присваивание строк не допускается (поскольку они рассматриваются компилятором как массивы символов). Для работы со строками в языке C следует использовать функции стандартной библиотеки `<cstring>` (или `<string.h>`)<sup>1</sup>. Для копирования строк предназначены функции `strcpy` и `strncpy`. Функция `strcpy(s2, s1)` копирует все символы строки `s1` (включая последний нулевой символ) в строку `s2`. Функция `strncpy(s2, s1, n)` делает то же самое, но копирует не более  $n$  символов. Пример:

```
const int n = 20;
```

---

<sup>1</sup>В языке C++ удобнее использовать класс `string`, рассматриваемый во второй части пособия.

```
char s1[n], s2[n];
gets(s1);
strcpy(s2, s1);
cout << s2 << endl;
```

Если значение `n` меньше или равно длине копируемой строки, то остаток строки не копируется, завершающий нуль-символ в конце не добавляется. При использовании обеих функций области памяти, отводимые для строки-источника и строки-приёмника, не должны пересекаться. Переполнение строк при копировании (как и при операциях с массивами), компилятором не контролируется.

Функция `strlen(s1)` возвращает длину строки (количество символов), не учитывая последний нулевой символ.

Конкатенация (сцепление) двух строк выполняется с помощью функции

```
char *strcat(char *str1, const char *str2);
```

Для сравнения строк используется функция

```
int strcmp(const char *str1, const char *str2);
```

Она возвращает число, большее (меньшее) нуля, если строка `str1` расположена после (до) строки `str2` по алфавиту. Если строки равны, то возвращается 0.

Для поиска символа в строке используется функция

```
char *strchr(const char *str, int ch);
```

Возвращается указатель на первое вхождение символа, заданного младшим байтом параметра `ch`.

Для преобразования строк в числа используются функции из библиотеки `<stdlib.h>` (или `<cstdlib>`):

```
double atof(const char*); // преобразование строки в double
int atoi(const char*); // преобразование строки в int
long atol(const char*); // преобразование строки в long
```

Если строку нельзя преобразовать в число, то возвращается 0.

Для обратного перевода из числа в строку используется функция `sprintf`.

### 5.3. Пользовательские типы данных

**Переименование типов.** Иногда удобно определить синоним для какого-либо типа данных; для этого используется ключевое слово `typedef`:

```
typedef тип_данных новое_имя ;
```

Для определения массива:

```
typedef тип новое_имя [размерность] ;
```

Здесь квадратные скобки являются частью синтаксиса (а не обозначают необязательный элемент, как обычно).

Примеры:

```
typedef unsigned long ULONG; // длинное целое без знака
typedef char MyStr[50]; // строка из 49 символов
```

После определения новый тип можно использовать, как обычный:

```
ULONG a, b; // Числа
MyStr s1; // Строка
MyStr ss[10]; // Массив из 10 строк
```

Определение типов помогает сократить текст программы и повысить его наглядность, а также делает программу переносимой: если все типы, зависящие от платформы, заменить синонимами, то при переходе на другую платформу достаточно будет скорректировать определения.

**Перечисления.** При моделировании объектов любой предметной области часто требуется кодировать различные состояния (номер ошибки, номер этапа некоторого процесса и т. д.). Для этого удобно использовать перечисляемый тип данных, который определяется с помощью служебного слова `enum`:

```
enum имя_типа {список_констант} ;
```

Обозначения констант указываются через запятую. Для каждой константы может быть указан инициализатор. Те константы, для которых инициализатор не указан, будут кодироваться целыми числами в порядке возрастания. Имена перечисляемых констант должны быть уникальными, а значения могут совпадать.

Пример:

```
enum MyErr {
 ERR_READ = 1,
 ERR_WRITE,
 ERR_CONVERT
};
MyErr error1;
// ...
switch (error1) {
 case ERR_READ:
 // ...
 break;
 case ERR_WRITE:
 // ...
 break;
 case ERR_CONVERT:
 // ...
}
```

Здесь мы определили новый тип `MyErr` и описали переменную `error1`. Данная переменная может принимать три возможных значения: `ERR_READ`, `ERR_WRITE` и `ERR_CONVERT`. Эти три константы компилятор будет заменять значениями 1, 2 и 3 соответственно.

**Структуры.** Набор элементов различных типов, доступ к которым осуществляется по имени, называется *структурой*. Для определения структуры используется ключевое слово `struct`:

```
struct [имя_типа] {
```

```
тип_1 элемент_1;
тип_2 элемент_2;
...
тип_n элемент_n;
} [список_описателей];
```

Если отсутствует имя типа, то указывается список определений переменных, указателей или массивов. Если указано имя нового типа, то список описателей может отсутствовать. Элементы структуры называются *полями*.

Примеры:

```
struct Student {
 char fio[50]; // Фамилия, имя, отчество
 int group; // Номер группы
 float rate; // Средний балл
};
Student s1, *ps, ss[25];
Student s2 = {"Петров Максим Сергеевич", 12, 4.7};

struct MyComplex {
 double re, im;
};
MyComplex x, y, *pc;
MyComplex a[3] = {{1, 2.5}, {-2, 1.2}, {3.2, 1.7}};
MyComplex a[2][3] = {
 {{3, 1.5}, {1, 1.2}, {-2, 4.5}},
 {{1, 2.5}, {-2, 1.2}, {5.2, 1}}
};
```

Если переменные имеют один и тот же структурный тип, то для них определена операция присваивания (поэлементного копирования). Доступ к полям структуры осуществляется с помощью операции выбора (точка для обычных переменных, стрелка для указателей):

```
s1.fio = "Сергеев Николай Петрович";
```

```
s2.rate = 3.75;
ss[7].group = 10;
(*ps).group = 8;
ps->group = 8;
```

Элементом (полем) структуры может являться другая структура или указатель на эту же структуру. Имена полей структуры должны быть уникальными. Поля разных структур могут иметь совпадающие имена.

Размер структуры в памяти, возвращаемый функцией `sizeof()`, не обязательно равен сумме размеров её полей, т. к. начало каждого поля может быть выровнено компилятором по границе машинного слова.

**Битовые поля.** Элементами структур могут являться битовые поля. *Битовое поле* — это набор двоичных разрядов. Каждый разряд (бит) имеет два возможных значения: 0 или 1. При определении структур размер каждого поля в битах указывается через двоеточие, например:

```
struct MyFlags {
 bool centerX :1;
 bool centerY :1;
 unsigned int color :4;
};
```

Здесь определена структура `MyFlags`, в состав которой входят три поля: два однобитных поля `centerX`, `centerY` и одно 4-битное поле `color`.

**Объединения.** *Объединение* — это структура, все поля которой располагаются по одному и тому же адресу. Объединение требует столько места в памяти, сколько занимает наибольший из его элементов. Описание объединения имеет тот же синтаксис, что и структура, но вместо `struct` используется ключевое слово `union`. Пример:

```
union MyType {
 double x;
 int i;
};
```

```
MyType a;
a.x = 4.75;
a.i = 3;
```

В переменной `a` можно хранить вещественное или целое число, обращаясь к полю `x` или `i` соответственно. Разумеется, после второго присваивания предыдущее значение `4.75` теряется.

Объединение не может содержать битовые поля. При описании переменной-объединения может быть проинициализировано значение только первого поля:

```
MyType a = {4.75};
```

## 5.4. Работа с файлами

Для работы с файлами в языке C надо подключить заголовочный файл `<stdio.h>` или `<cstdio>`.

Перед тем, как работать с файлом, его требуется сначала открыть для чтения или записи с помощью функции `fopen`:

```
FILE * fopen (const char * filename, const char * mode);
```

Здесь `filename` — имя файла (необходимо помнить о том, что символ `'\'` обозначает начало управляющей последовательности, поэтому его необходимо удваивать); `mode` — режим:

`"r"` — файл открывается для чтения (`read`);

`"w"` — файл открывается для записи (`write`), если такой файл уже существует, то вся информация в нём стирается;

`"a"` — файл открывается для добавления информации в его конец (`append`);

`"r+"` — файл открывается для чтения и записи (файл уже должен существовать);

`"w+"` — открывается пустой файл для чтения и записи (если такой файл существует, то хранящаяся в нём информация стирается);

"a+" — файл открывается для чтения и добавления информации в его конец (append).

Строка `mode` также может содержать символ `t` или `b`, что означает соответственно текстовый (text) или двоичный (binary) режим. По умолчанию действует текстовый режим.

При успешном открытии функция `fopen` возвращает указатель на структуру типа `FILE`, в которой содержится вся информация, необходимая для работы с файлом. Если при попытке открыть файл происходит ошибка, то возвращается нулевой указатель. Ошибка может возникнуть из-за того, что файл заблокирован (открыт для записи) другой программой, из-за отсутствия свободного места на диске и т. д.

Пример открытия для чтения и записи двоичного файла `myfile1.dat`, расположенного в каталоге `Work` на диске `C`:

```
FILE *f = fopen("C:\\Work\\myfile1.dat", "rb+");
```

Если указывается только имя файла и расширение (без полного пути), то имеется в виду текущий каталог текущего диска.

По окончании работы с файлом его необходимо закрыть с помощью функции `fclose(f)`. Это разблокирует файл для доступа из других программ, а также освободит область памяти, занимаемую служебной информацией, связанной с этим файлом.

Чтение (`fread`) и запись (`fwrite`) файлов, открытых в двоичном режиме, происходит блоками заданного размера:

```
size_t fread(void *buf, size_t m, size_t k, FILE *f);
size_t fwrite(const void *buf, size_t m, size_t k, FILE *f);
```

Здесь `buf` — указатель на буфер в памяти (из которого читаются или в который записываются байты); `m` — размер одного элемента данных (количество байт); `k` — количество элементов для чтения или записи. Обе функции возвращают число прочитанных или записанных элементов. При чтении возвращаемое значение может оказаться меньше заданного `k`, если достигнут конец

файла или произошла ошибка чтения. При записи — из-за ошибки записи (например, нет свободного места на диске). Напомним, что `size_t` является базовым беззнаковым целочисленным типом (32-разрядное целое без знака для 32-разрядных систем).

## 6. Основы объектно-ориентированного программирования

### 6.1. Классы

*Класс* — пользовательский тип данных, представляющий собой модель некоторого объекта предметной области, инкапсулирующий данные и функции для работы с этими данными.

Данные класса называются *полями* или *данными-членами* (класса), а функции класса — *методами* или *функциями-членами* (класса).

Поля класса могут иметь любой тип, кроме типа этого же класса (но могут быть указателями или ссылками на этот класс); могут быть описаны с модификатором `const`, при этом они инициализируются только один раз (с помощью инициализатора конструктора, см. ниже) и не могут изменяться; могут быть описаны с модификатором `static`, но не как `auto`, `extern` или `register`.

Инициализация полей класса при описании не допускается.

Классы могут быть глобальными (объявленными вне любого блока) и локальными (объявленными внутри некоторого блока, например, внутри функции или другого класса).

Класс описывается следующим образом:

```
class имя_класса {
 элементы_класса;
};
```

Точка с запятой в конце ставится обязательно. Здесь `имя_класса` — любой допустимый идентификатор; `элементы_класса` — объявления и/или определения данных-членов (полей класса) и функций-членов (методов класса). Поля и/или методы могут отсутствовать. Класс, не имеющий ни полей, ни методов, называется *пустым классом*.

Спецификаторы доступа `private` (частный, закрытый) и `public` (общедоступный, публичный) управляют видимостью элементов класса. Элементы (поля и методы), описанные после служебного слова `private`, видимы только внутри класса. Этот вид доступа применяется по умолчанию, т.е. спецификатор `private` подразумевается во всех случаях, когда спецификатор доступа не указан явно. После служебного слова `public` описывается так называемый *интерфейс класса*, т.е. открытые (публичные) поля и методы, доступные вне класса:

```
class имя_класса {
 public:
 публичные_элементы_класса;
 private:
 скрытые_элементы_класса;
};
```

Действие любого спецификатора распространяется до следующего спецификатора или до конца класса. Можно задавать несколько секций `private` и `public`, порядок их следования значения не имеет.

**Пример 6.1.** Опишем класс «комплексное число» (напомним, что комплексное число — это упорядоченная пара вещественных чисел):

```
class Complex { // начало определения класса
 public: // поля и методы будут общедоступными
 // поля класса:
 double re, im; // вещественная и мнимая части

 // метод:
 double abs() { // вычисление модуля комплексного числа
 return sqrt(re*re+im*im);
 }
}; // конец определения класса
```

Описанный класс `Complex` имеет два поля `re` и `im` для хранения действительной и мнимой частей комплексного числа, а также один метод `abs()` для

вычисления модуля комплексного числа. Все поля и метод являются общедоступными, т. к. описаны после спецификатора `public`.

Объекты (экземпляры) класса можно создавать, как обычные переменные, просто указывая имя класса как имя нового типа:

```
Complex a, b, *p, arr[10];
```

Здесь для примера определены два комплексных числа `a` и `b`, указатель `p` на комплексное число и массив `arr`, состоящий из десяти комплексных чисел.

Обращение к полям класса внутри методов данного класса, как видно из приведённого примера, производится простым указанием идентификатора поля. Во всех остальных блоках программы для обращения к полю какого-либо экземпляра класса надо указать идентификатор экземпляра и имя поля через точку. В нашем примере можно было бы записать, к примеру:

```
a.re = 5.0; b.im = -a.re / 2;
arr[3].re = 5.0; arr[4].im = -arr[3].re / 5;
```

Вместо `(*p).re` и `(*p).im` обычно используют «стрелку»:

```
p = new Complex; p->re = 5.0; p->im = -p->re / 2;
```

## Список литературы

### Книги по C

1. Керниган Б., Ритчи Д. Язык программирования C. — 2-е изд. Пер. с англ. — М.: Вильямс, 2009. — 304 с.
2. Подбельский В.В., Фомин С.С. Программирование на языке Си: Учеб. пособие. — 2-е изд. — М.: Финансы и статистика, 2004. — 600 с.
3. *ISO/IEC International Standard 9899. Programming Languages — C. The current draft Standard (C99 with Technical corrigenda TC1, TC2, and TC3 included, 7 September 2007).*

### Книги по C++

4. Павловская Т.А. C/C++. Программирование на языке высокого уровня. — СПб.: Питер, 2003. — 461 с.
5. Павловская Т.А., Щупак Ю.А. C/C++. Структурное программирование: Практикум. — СПб.: Питер, 2003. — 240 с.
6. Павловская Т.А., Щупак Ю.А. C++. Объектно-ориентированное программирование: Практикум. — СПб.: Питер, 2006. — 265 с.
7. Лаптев В.В. C++. Объектно-ориентированное программирование: Учебное пособие. — СПб.: Питер, 2008. — 464 с.
8. Лаптев В.В., Морозов А.В., Бокова А.В. C++. Объектно-ориентированное программирование: Задачи и упражнения. — СПб.: Питер, 2007. — 288 с.
9. Подбельский В.В. Язык Си++: Учеб. пособие. — 5-е изд. — М.: Финансы и статистика, 2003. — 560 с.
10. Лафоре Р. Объектно-ориентированное программирование в C++. — 4-е изд. — СПб.: Питер, 2004. — 924 с.
11. Шилдт Г. Полный справочник по C++. — 4-е изд. — М.: Вильямс, 2006. — 800 с.

12. *Шилдт Г.* С++: Базовый курс. — 3-е изд. — М.: Вильямс, 2010. — 624 с.
13. *Шилдт Г.* С++: Руководство для начинающих. — 2-е изд. — М.: Вильямс, 2005. — 672 с.
14. *Шилдт Г.* Самоучитель С++. — 3-е изд. — СПб.: БХВ-Петербург, 2002. — 688 с.
15. *ISO/IEC International Standard 14882. Programming Languages — C++. Final Committee Draft (26 March 2010).*

### **Сборники задач**

16. *Юркин А.* Задачник по программированию. — СПб.: Питер, 2002. — 192 с.
17. Сборник заданий к практическим и лабораторным занятиям. / Под ред. И.Н. Котаровой. — М.: Изд-во МЭИ, 1991. — 111 с.
18. *Абрамов С.А., Гнездилова Г.Г., Капустина Е.Н., Селюн М.И.* Задачи по программированию. — М.: Наука, 1988. — 224 с.
19. *Пильщиков В.Н.* Сборник упражнений по языку Паскаль. — М.: Наука, 1989. — 160 с.

# Предметный указатель

ASCII, 129

BCD, 49

IDE, 16

L-значение, 29

LIFO, 115

LSD, 46

MSD, 46

алгоритм, 5

    сортировки, 95

алфавит, 9

ассемблер, 6

бит, 46

    младший, 46

    старший, 46

битовое поле, 140

ветвление, 7

декремент, 32

директива препроцессора, 20

идентификатор, 10

инициализатор, 13

инкремент, 32

интегрированная среда разработки, 16

интерпретатор, 8, 9

исключение, 91

код

    дополнительный, 49

    машинный, 5

    обратный, 49

    объектный, 8

кодировка символов, 21

команда

    машинная, 6

комментарии, 12

компилятор, 8, 9

компиляция, 20

компоновка, 20

компоновщик, 21

константа, 11, 12

лексема, 10

литерал, 11

манипулятор форматирования, 44

мантисса, 50

массив, 83

    двумерный, 99

    индекс элемента, 83

    одномерный, 83

модуль

    объектный, 20

объединение, 140

объявление, 12, 26

оператор, 6

операция

    арифметическая, 31

    бинарная, 29

    битовая, 52

    приоритет, 33

    сдвига, 33

    унарная, 29

описание, 12

- определение, 12, 26
- парадигма, 6
- перегрузка функций, 113
- переменная, 12
  - время жизни, 24
  - глобальная, 24
  - динамическая, 87
  - класс памяти, 24
  - локальная, 23
  - область видимости, 24
  - область действия, 23
  - объявление, 12, 26
  - описание, 12
  - определение, 12, 26
- перечисление, 137
- подпрограмма, 7
- поле
  - битовое, 140
- препроцессор, 20
- приложение
  - консольное, 19
- присваивание, 29
  - составное, 32
- программа, 5
- программирование, 5
  - объектно-ориентированное, 7
  - процедурное, 7
  - структурное, 7
- разадресация, 88
- разыменование, 88
- семантика, 9
- синтаксис, 9
- система счисления
  - восьмеричная, 45, 47
  - двоичная, 45
  - позиционная, 45
  - шестнадцатеричная, 45, 47
- слово
  - ключевое, 10
- сортировка
  - гномья, 96
  - пузырьком, 97
- ссылка, 90
- стек, 115
- структура, 138
- суффикс числовой константы, 15
- тип
  - приведение, 31
- тип переменной, 12
- типы данных, 14
- трансляция, 9
- файл
  - заголовочный, 127
- функция, 7, 105
  - встраиваемая, 112
  - заголовок, 105
  - объявление, 105
  - определение, 105
  - прототип, 105
  - рекурсивная, 109
  - тело, 105
  - фактические параметры, 106
  - формальные параметры, 106
- цикл, 7
- число
  - восьмеричное, 47
  - двоично-десятичное, 49

двоичное, 45

шестнадцатеричное, 47

#### язык

алгоритмический, 5

ассемблера, 5

высокого уровня, 6

декларативный, 6

императивный, 6

программирования, 5